Design Patterns Of Distributed Systems

Foundation For Expert Generalists

Thoughtworks

Course Overview

Design Patterns Of Distributed Systems

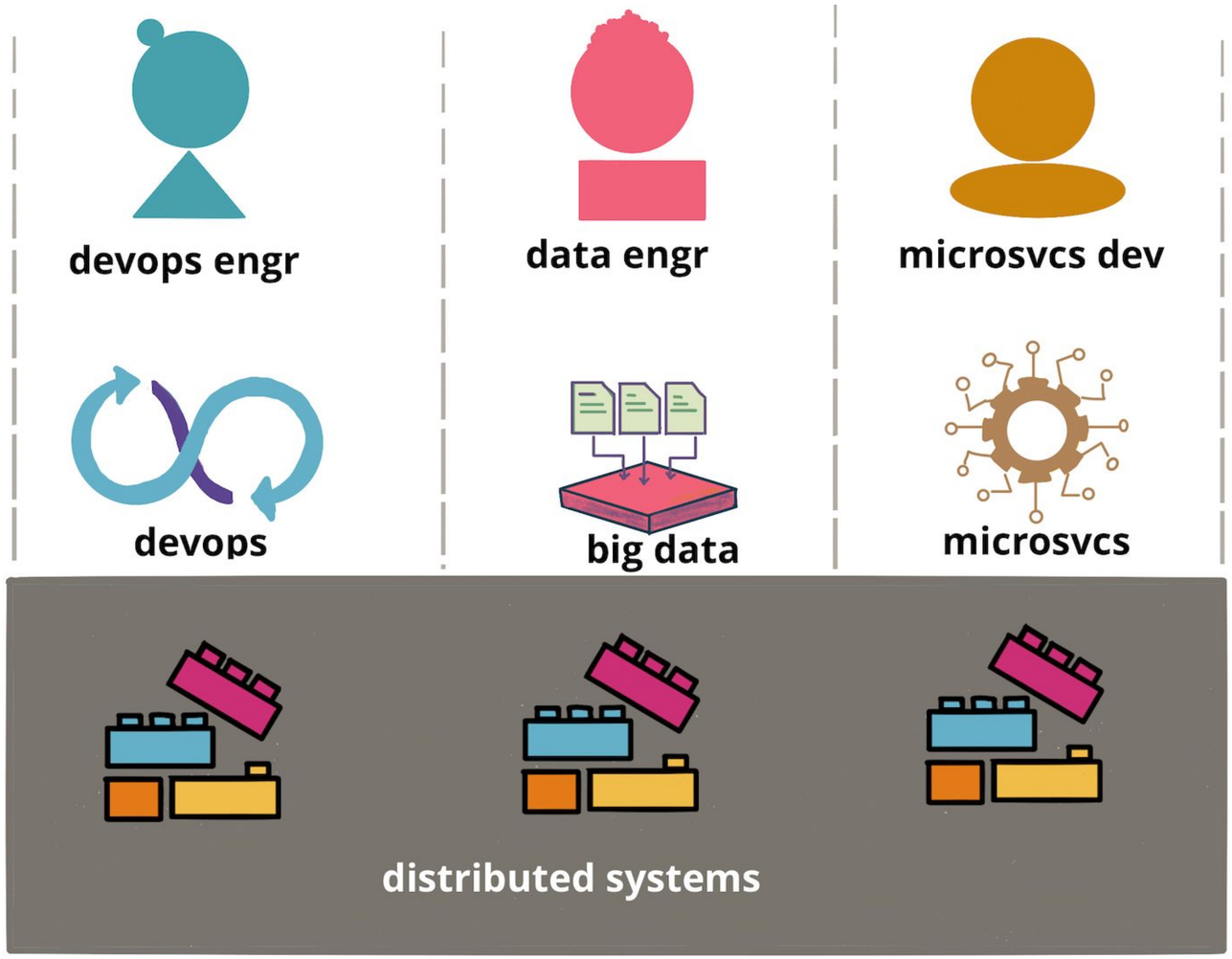
- Foundation For Expert Generalists
- Core building blocks across real-world distributed architectures
- Bridging cluster coordination, data placement, and transactional storage

The Characteristics of an Expert Generalist



- Curiosity — Discover how technologies work under the hood before applying them
- Collaborativeness — Partner with specialists; know your limits and exercise humility
- Customer Focus — Channel curiosity toward user value; avoid chasing shiny tools
- Favor Fundamental Knowledge — Prioritize timeless patterns over ephemeral syntax
- Blend of Skills — Comb-shaped profile with broad foundation & multiple deep spikes
- Sympathy for Related Domains — Understand how adjacent architectural layers interact
- Article: https://martinfowler.com/articles/expert-generalist.html#TheCharacteristicsOfAnExpertGeneralist

Sympathy for Related Domains



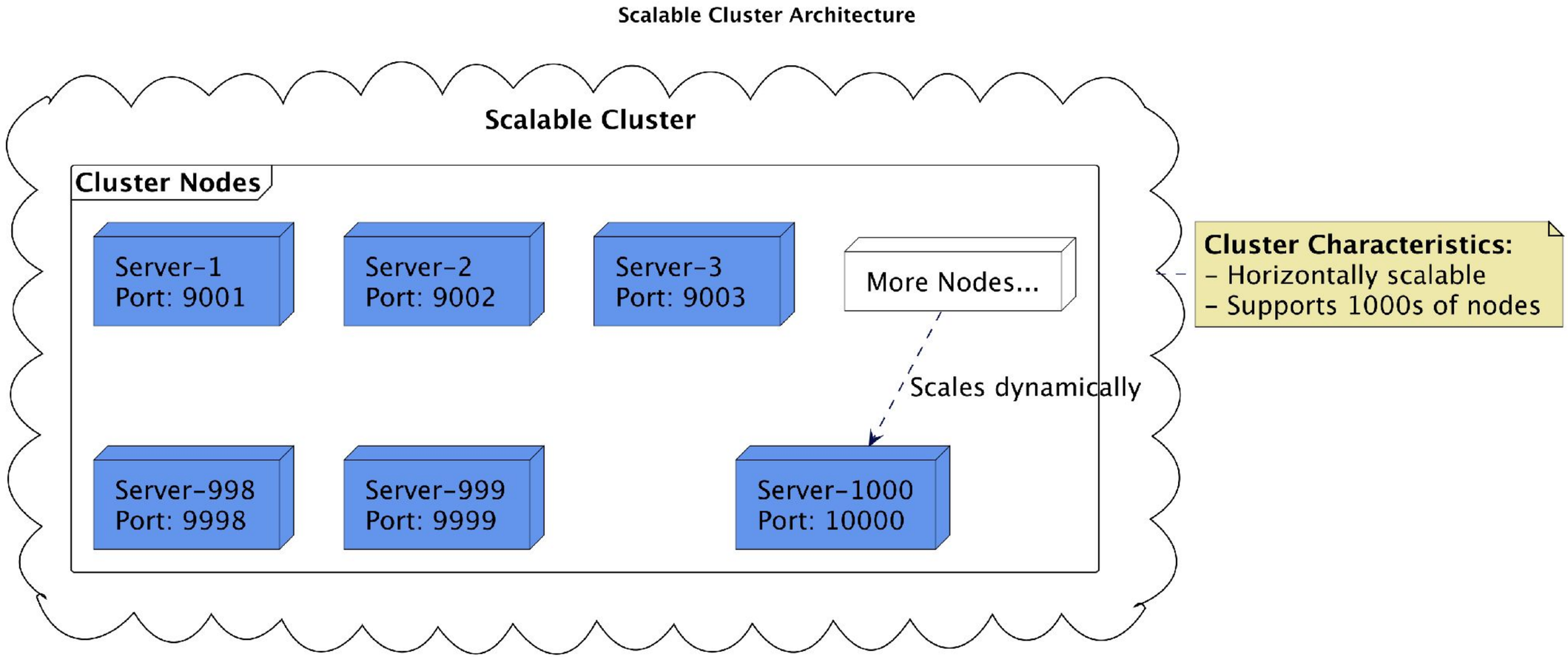
- Mechanical Sympathy: An intuitive feel for how the underlying platform operates under stress
- Beyond Leaky Abstractions: APIs hide details, but understanding the physics prevents outages
- Connecting the Silos: AppDev, Platform/DevOps, and Data teams share the same distributed fundamentals
- Shared Physics: Kafka, Kubernetes, Spark, and Delta Lake all solve the same core coordination and data challenges
- Article: https://martinfowler.com/articles/expert-generalist.html#SympathyForRelatedDomains

How this course is structured?

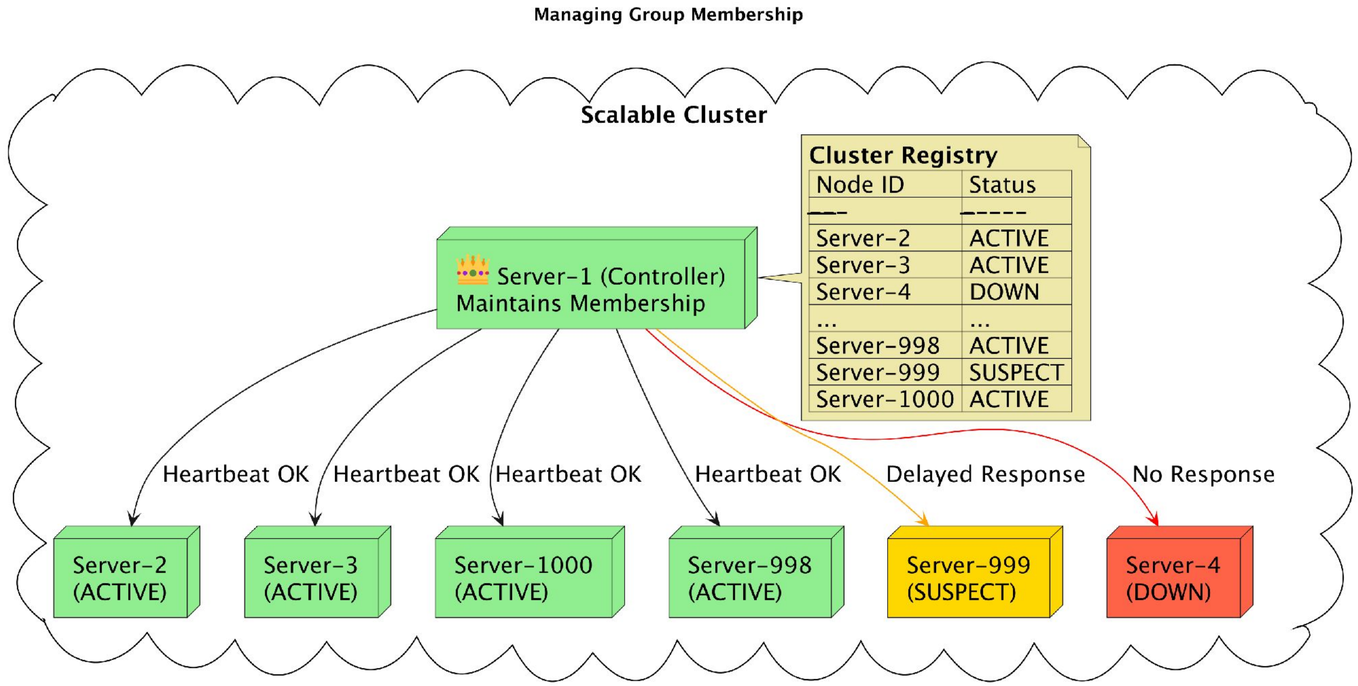
- Module 1: The Consistent Core, Group Membership & Failure Detection (Leases, State Watches)
- Module 2: Cluster Coordination & Workload Placement (Controllers, Epoch Fencing, Reconciliation Loops, Scheduling)
- Module 3: Partitioning Work & Data (Fixed Partitions, Key Hashing, Columnar Row Groups to RDDs)
- Module 4: Consistency Across Replicas & Partitions (Leader–Follower High-Watermark, Delta Lake ACID Log & OCC)

Module 1 · The Consistent Core, Group Membership & Failure Detection

Scalable clusters have up to a few thousand servers



Common Requirements for a Cluster



- Group membership and failure detection
- Metadata storage
- Cluster coordination

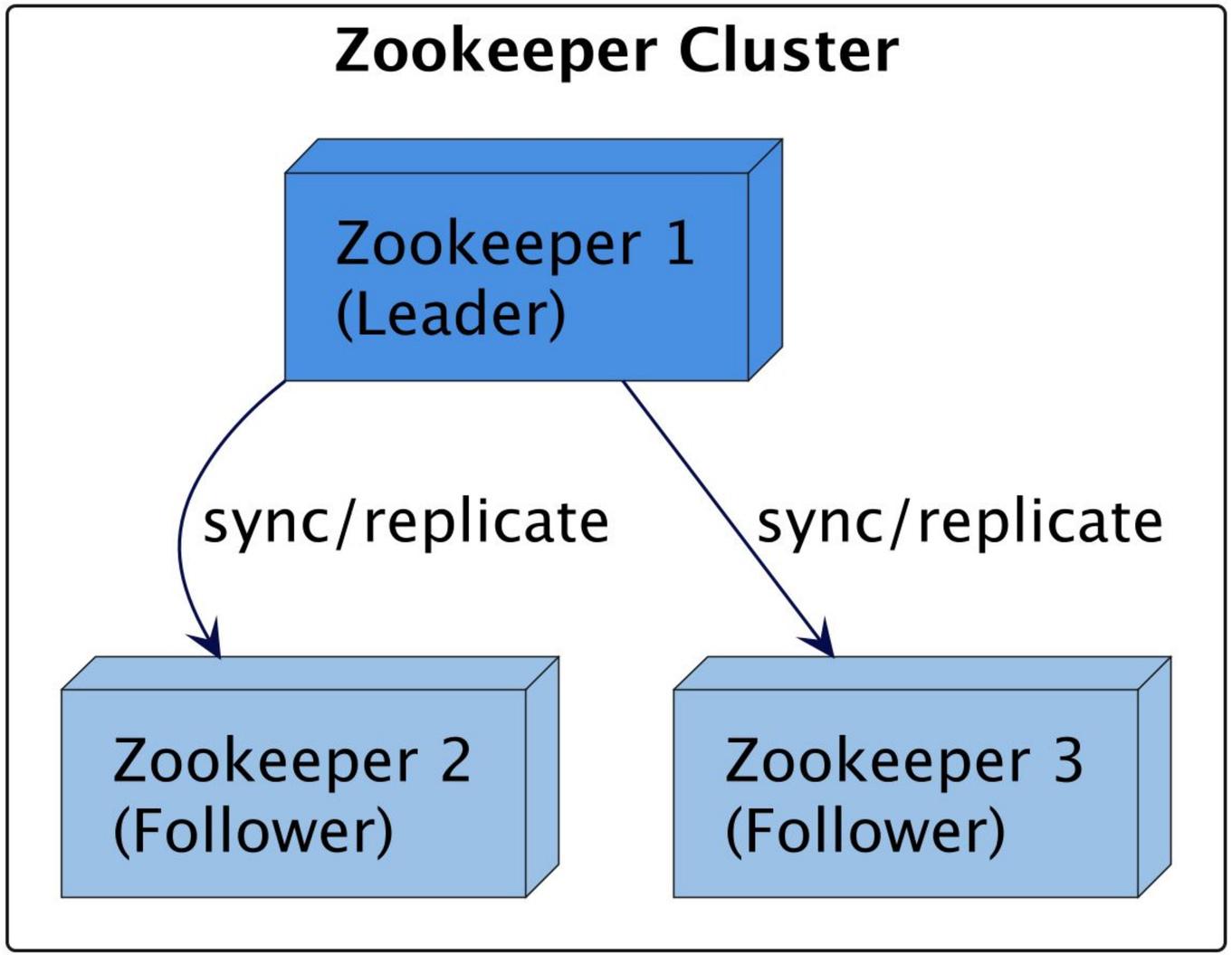
These requirements need consensus

- Managing group membership and metadata requires strong consistency
- Quorum-based consensus algorithms (Paxos, Raft, Zab) do not scale beyond a few nodes (3 to 5)
- Implemented as a separate, small consensus cluster to manage metadata and provide coordination primitives for the larger data cluster

Real world examples

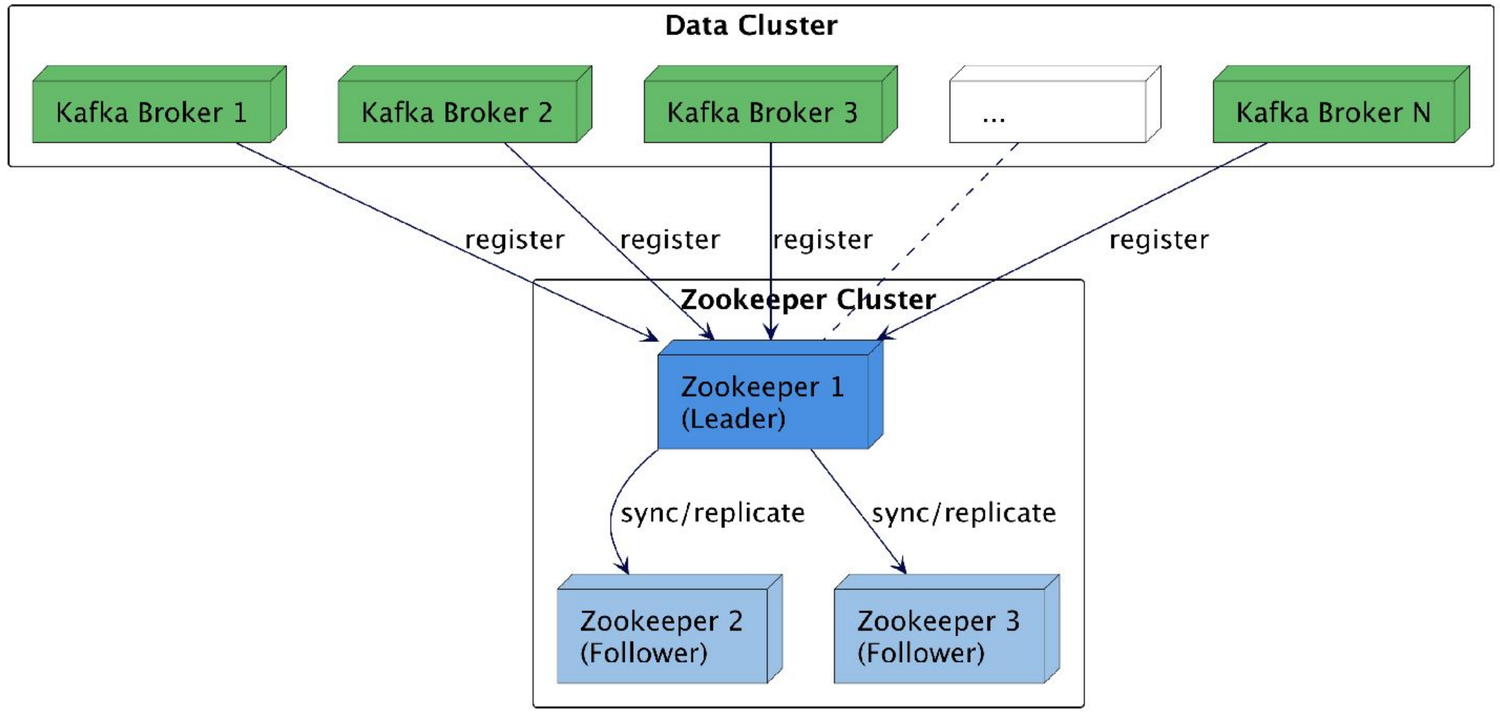
- Apache Kafka: Historically used ZooKeeper; now uses internal KRaft quorum
- Kubernetes: Uses etcd for all cluster state, pod specs, and node status
- HDFS / YARN: Active and Standby NameNodes coordinated via ZooKeeper

Kafka Cluster with Controller and ZooKeeper – Step 1



Kafka Cluster with Controller and ZooKeeper – Step 2

- Zab (Zookeeper Atomic Broadcast) protocol ensures strong consistency across nodes
- Leader handles all writes
- Followers serve reads and replicate data



Kafka today: the core moved inside

- ZooKeeper mode was removed in Kafka 4.0 — KRaft is now the only supported mode
- Dedicated controller nodes form a Raft quorum and hold metadata in an internal __cluster_metadata topic
- The pattern did not change: brokers still depend on a small, strongly consistent core
- What changed is the topology — the Consistent Core now lives inside the cluster rather than beside it
- We use ZooKeeper here as a reference implementation, because the primitives are easier to see in isolation

An Example - Kubernetes

- Worker nodes (Node 1 .. Node N) running Kubelet and hosting Pods
- Control plane backed by etcd running Raft-based consensus (3 to 5 nodes)
- API Server exposes resources; Controller Manager and Scheduler watch etcd state
- Exact same architectural duality as Kafka with ZooKeeper

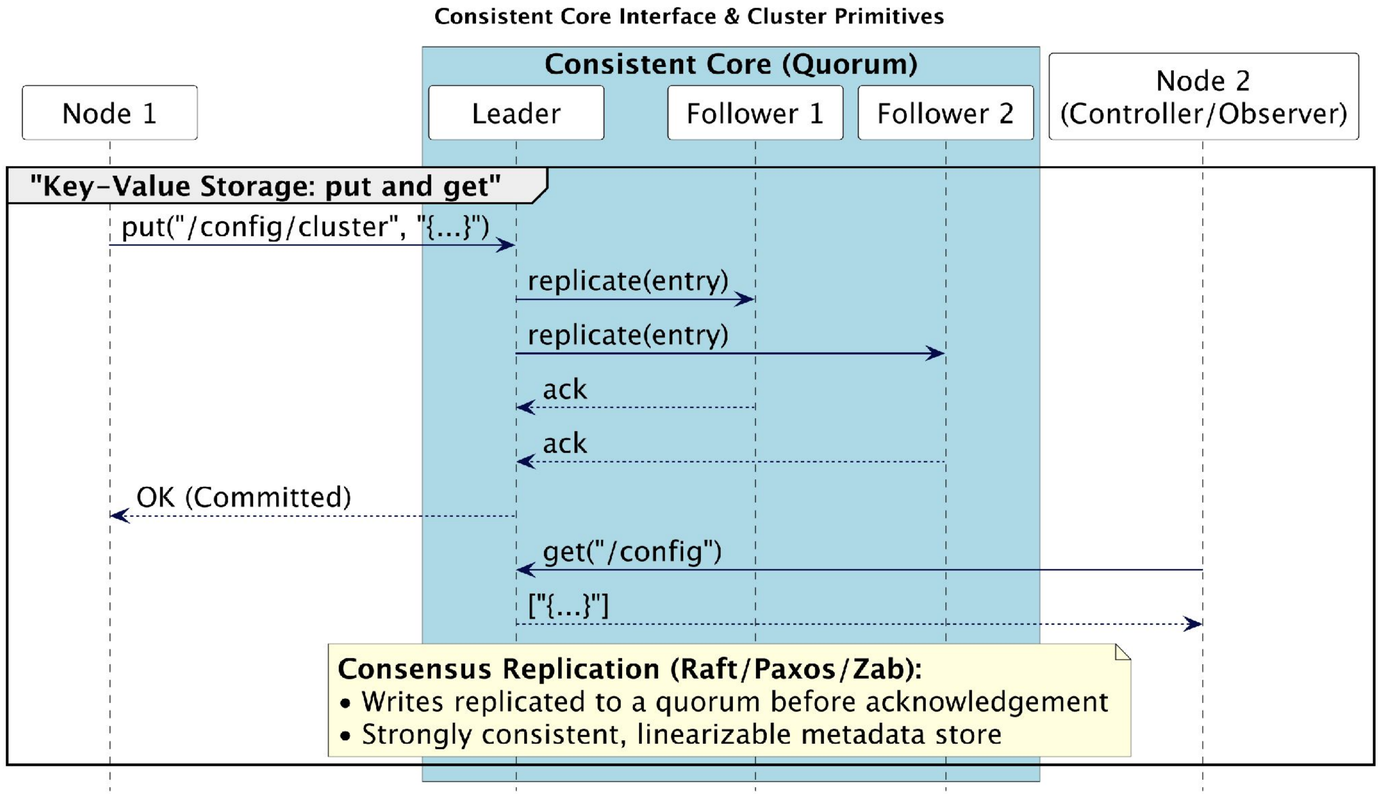
Kubernetes - Registration and lookup

- etcd cluster provides Raft-based consensus (Server 1, 2, 3)
- Kubelets register node metadata (`/registry/nodes/<nodename>`) with IP, capacity, and conditions
- Controllers and Schedulers register interest via ListWatch
- API Server acts as the gatekeeper and abstraction over etcd

What’s common?

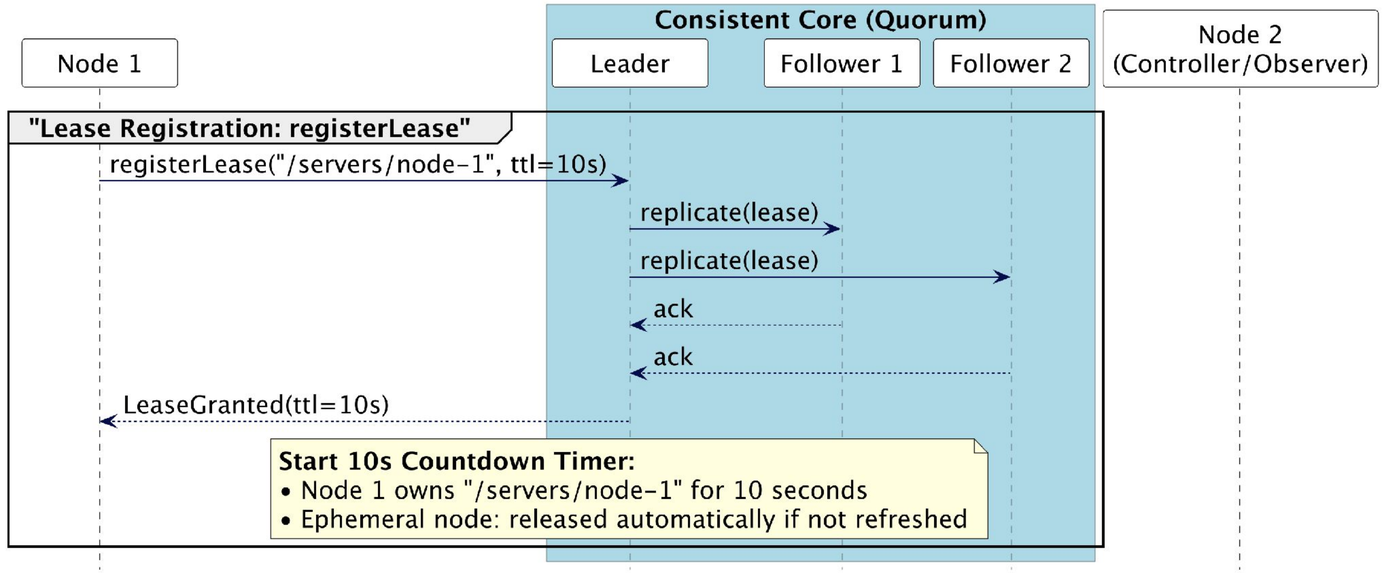
- Group Membership: How many nodes in the cluster? How to map partitions/Pods to nodes?
- Failure Detection: Detect when nodes, partitions, or pods fail
- Information Dissemination: Track and disseminate metadata so appropriate action can be taken

Consistent Core Interface & Cluster Primitives – Step 1



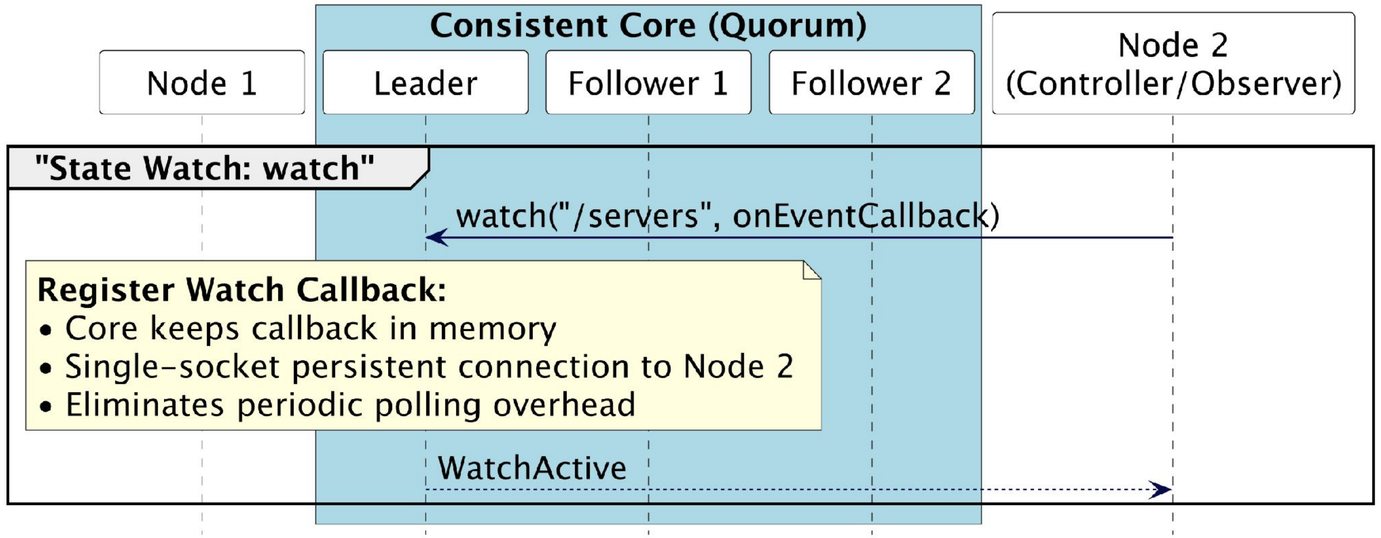
- Key-Value Storage (put / get): Linearizable metadata replication via consensus
- Lease (registerLease / refreshLease): Time-bound liveness and automatic failure detection
- State Watch (watch): Asynchronous change notifications eliminate polling

Consistent Core Interface & Cluster Primitives - Step 2



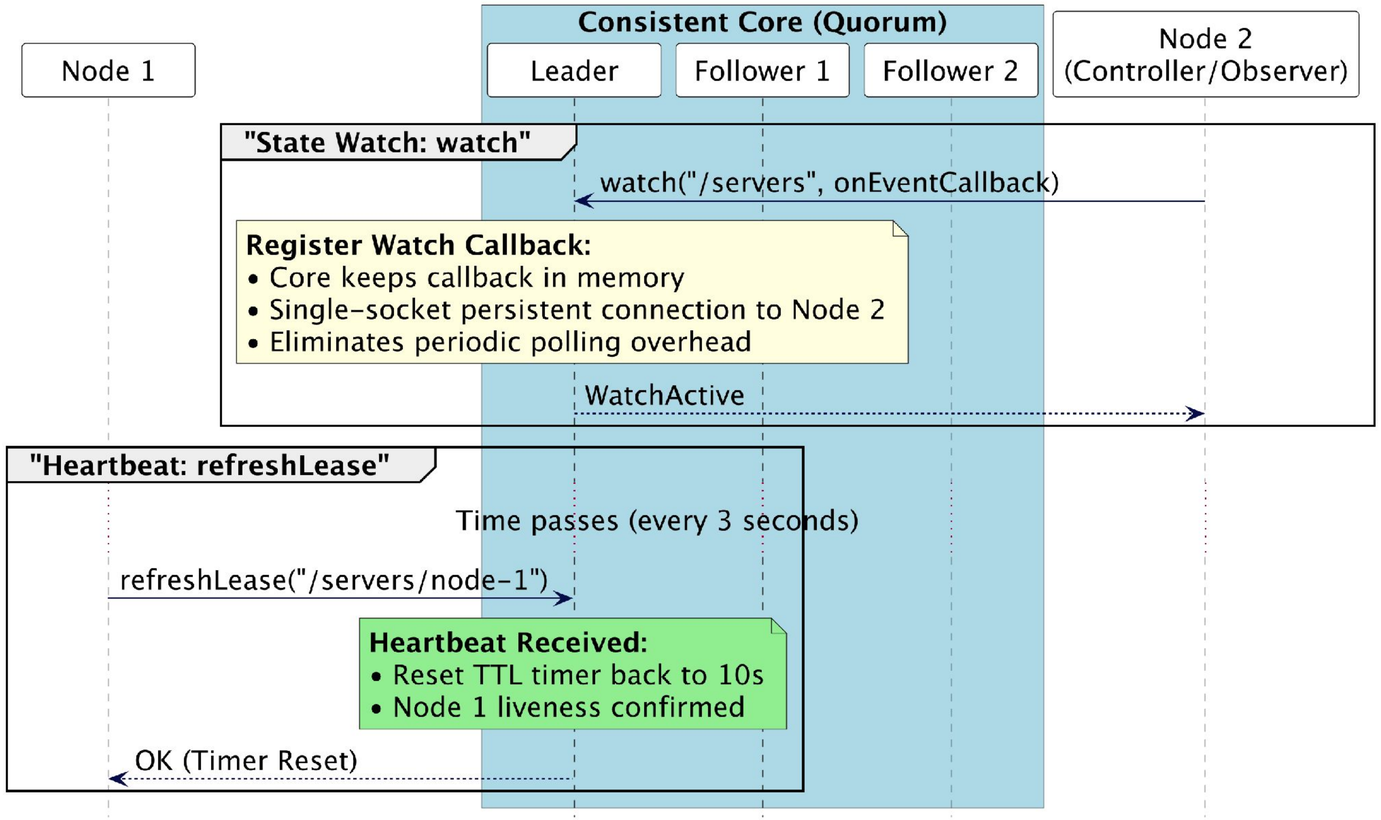
- Key-Value Storage (put / get): Linearizable metadata replication via consensus
- Lease (registerLease / refreshLease): Time-bound liveness and automatic failure detection
- State Watch (watch): Asynchronous change notifications eliminate polling

Consistent Core Interface & Cluster Primitives – Step 3



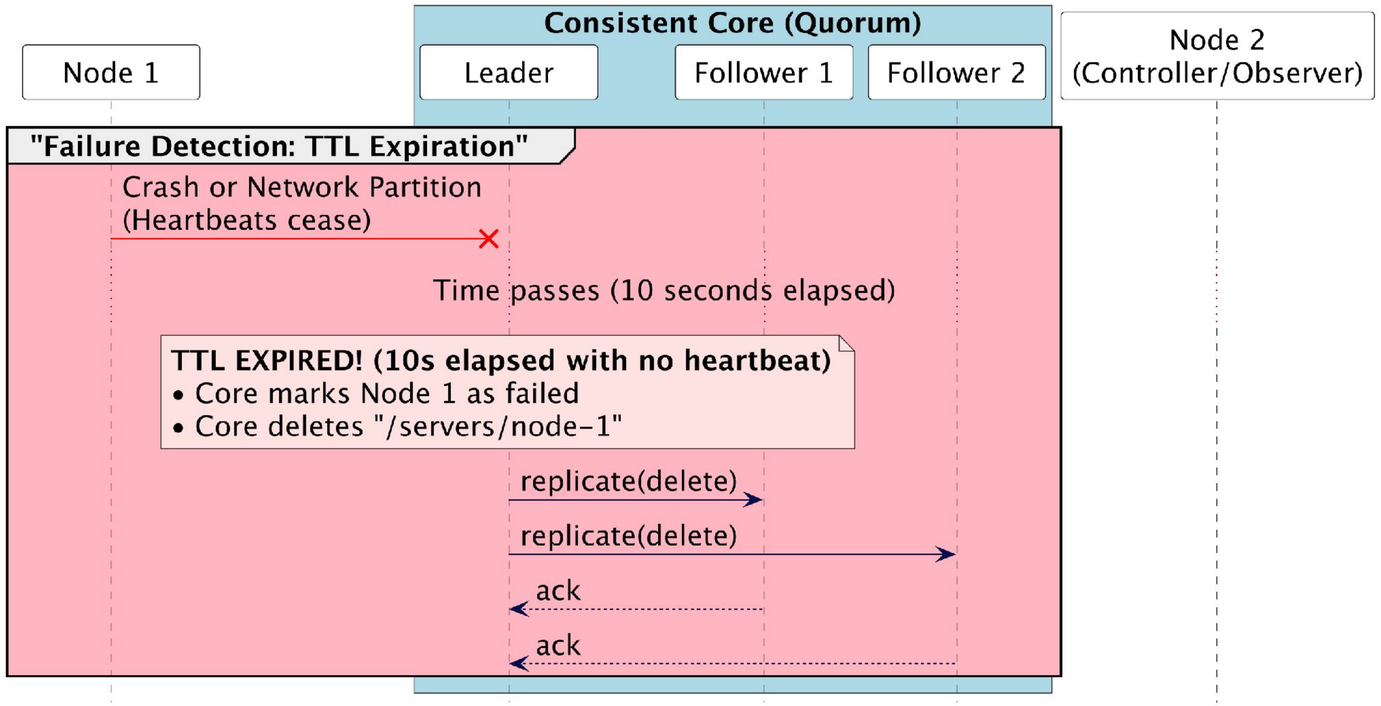
- Key-Value Storage (put / get): Linearizable metadata replication via consensus
- Lease (registerLease / refreshLease): Time-bound liveness and automatic failure detection
- State Watch (watch): Asynchronous change notifications eliminate polling

Consistent Core Interface & Cluster Primitives - Step 4



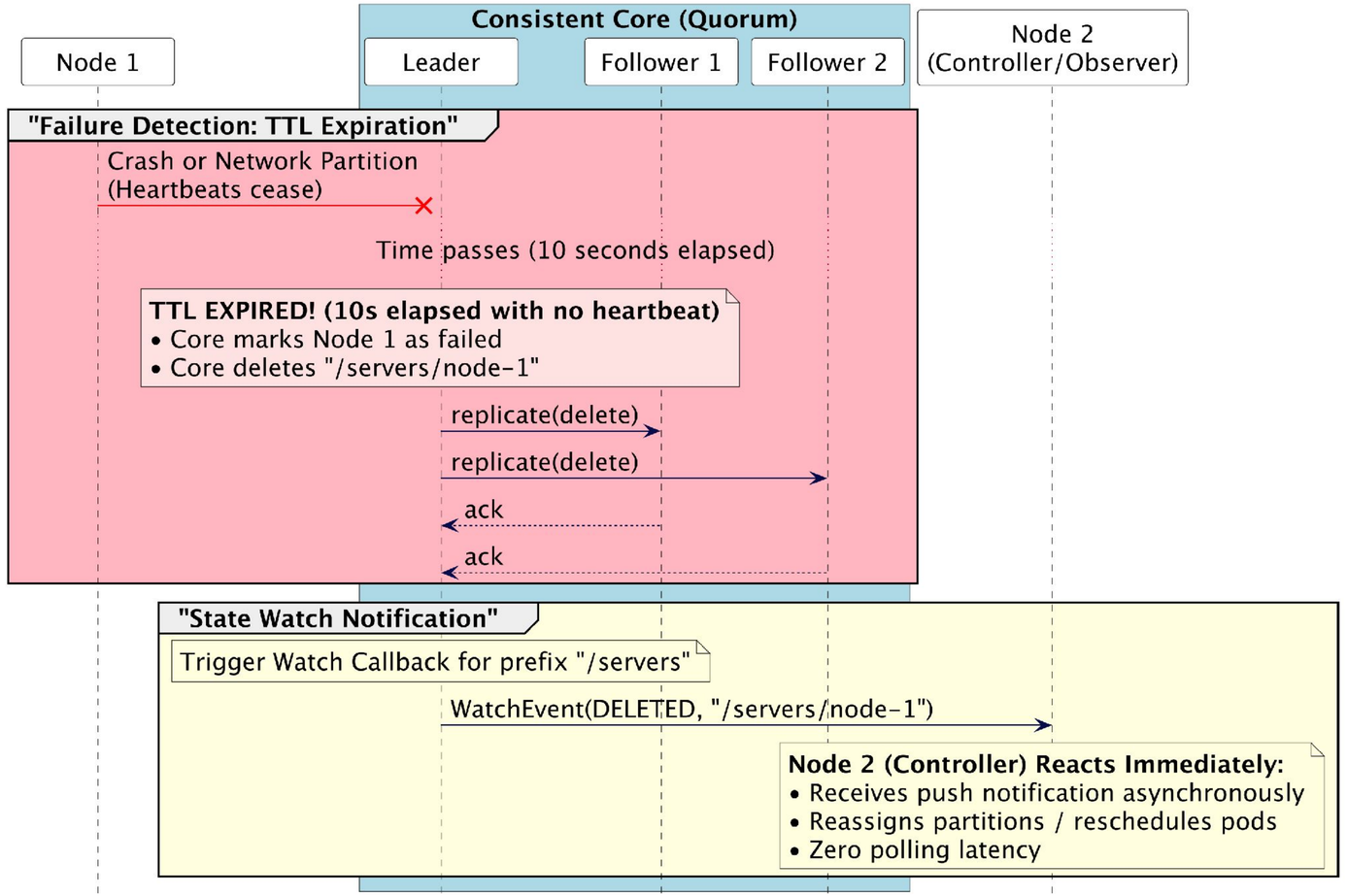
- Key-Value Storage (put / get): Linearizable metadata replication via consensus
- Lease (registerLease / refreshLease): Time-bound liveness and automatic failure detection
- State Watch (watch): Asynchronous change notifications eliminate polling

Consistent Core Interface & Cluster Primitives - Step 5



- Key-Value Storage (put / get): Linearizable metadata replication via consensus
- Lease (registerLease / refreshLease): Time-bound liveness and automatic failure detection
- State Watch (watch): Asynchronous change notifications eliminate polling

Consistent Core Interface & Cluster Primitives - Step 6



- Key-Value Storage (put / get): Linearizable metadata replication via consensus
- Lease (registerLease / refreshLease): Time-bound liveness and automatic failure detection
- State Watch (watch): Asynchronous change notifications eliminate polling

An expired lease is not enough

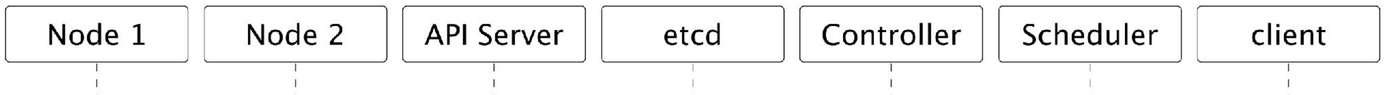
- A node can hold a valid lease, pause longer than the TTL (GC pause, VM suspend, host stall), and resume believing it is still the owner
- Meanwhile the core expired that lease and granted it to another node
- Nothing went wrong in the core — this is not a consensus failure
- Fix: The core returns a monotonically increasing fencing token with every grant
- The resource, not the client, rejects any write carrying a token older than the highest it has already seen

Assignment 1: Node registration with Zookeeper

- Task 1: Register Broker information (id, host, port) in ZooKeeper by creating an ephemeral node
- Task 2: Register a callback to detect broker node changes dynamically

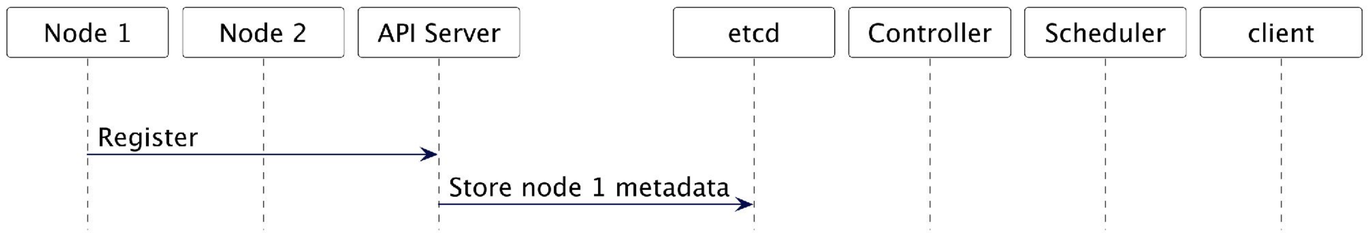
Kubernetes Kubelet Registration – Step 1

- Kubelet registers with API Server
- API Server persists node metadata in etcd
- Key-value pair stored under `/registry/nodes/<node>`

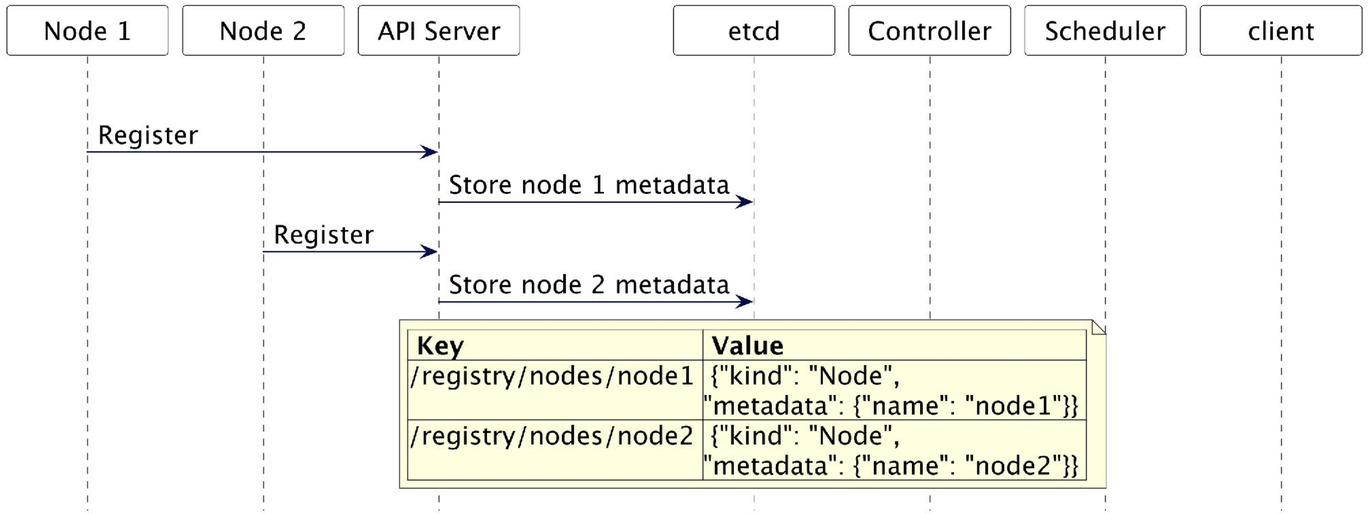


Kubernetes Kubelet Registration – Step 2

- Kubelet registers with API Server
- API Server persists node metadata in etcd
- Key-value pair stored under `/registry/nodes/<node>`



Kubernetes Kubelet Registration – Step 3



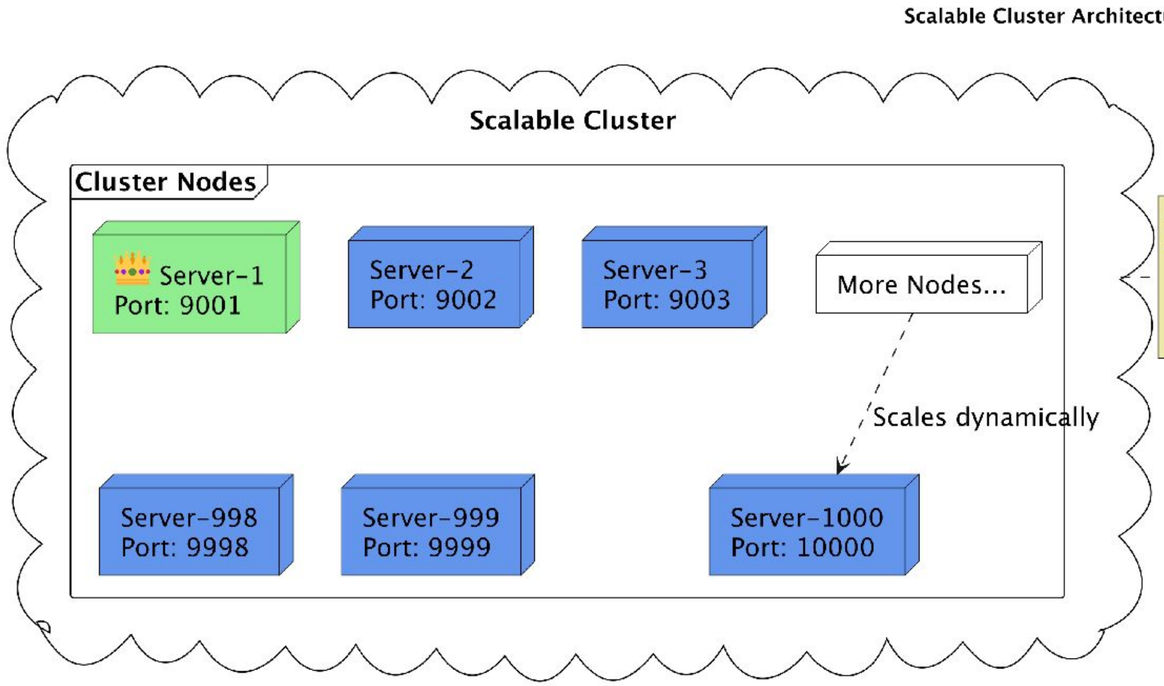
- Kubelet registers with API Server
- API Server persists node metadata in etcd
- Key-value pair stored under `/registry/nodes/<node>`

Assignment 2: Node registration with etcd

- Task 1: Register nodes with etcd

Module 2 · Cluster Coordination & Workload Placement

A specific server needs to coordinate the cluster activity

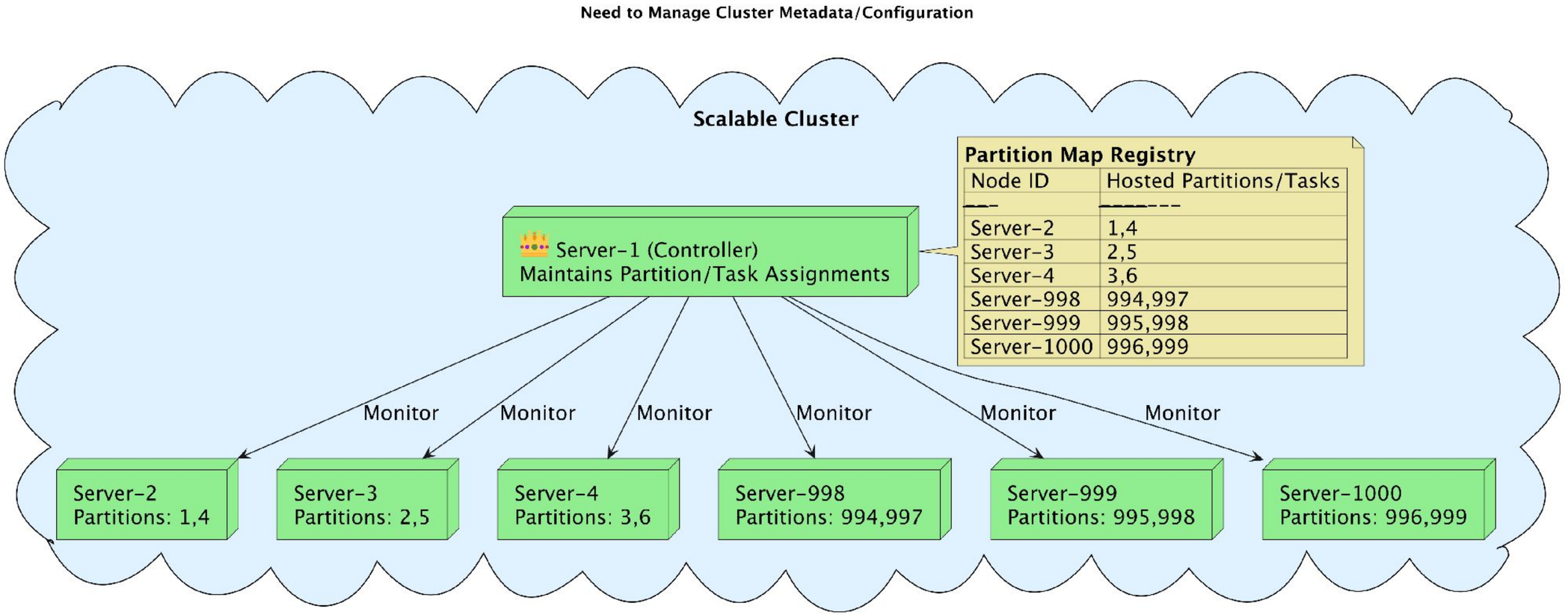


Cluster Requirements:
- Cluster operations need to be coordinated.
  - Manage group membership and failure detection.
  - Nodes join and leave the cluster dynamically.
  - One node typically acts as the **controller** to coordinate cluster activities.

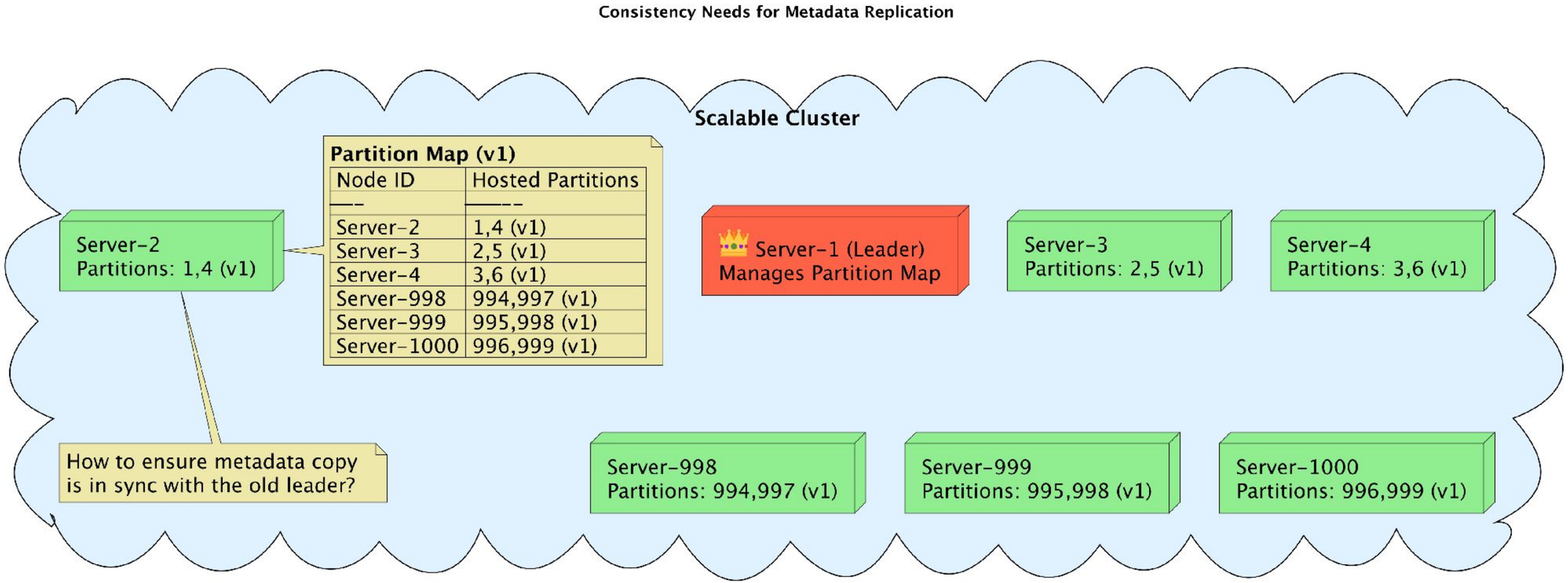
Needs automatic re-election in case of failure

- Nodes join and leave dynamically in large clusters
- If the controller crashes or becomes partitioned, the cluster cannot freeze
- Automatic re-election is required to maintain cluster operations
- Followers detect controller failure and initiate election

Need to manage cluster metadata/configuration



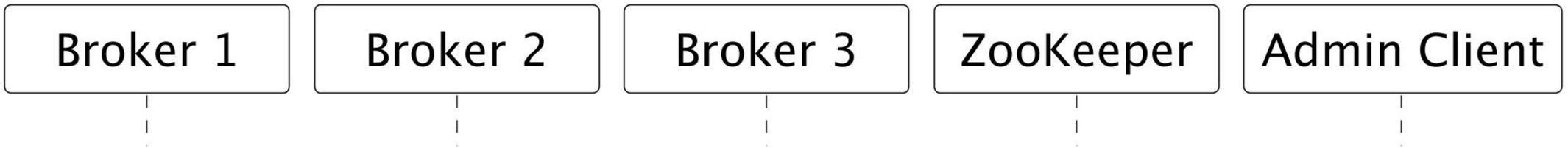
Controller notifies nodes about configuration changes



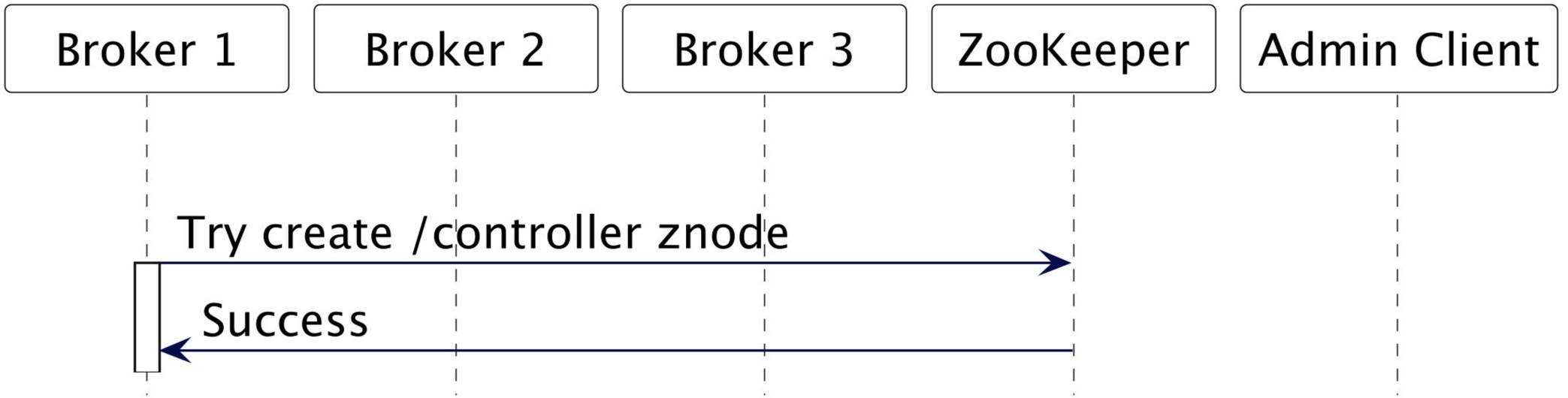
New controllers must inherit consistent metadata

- When a controller fails, a new controller is elected
- The new controller must take over with the exact current metadata state
- How to ensure the metadata copy is consistent and in sync with the previous leader?
- Requires a strongly consistent, fault-tolerant store

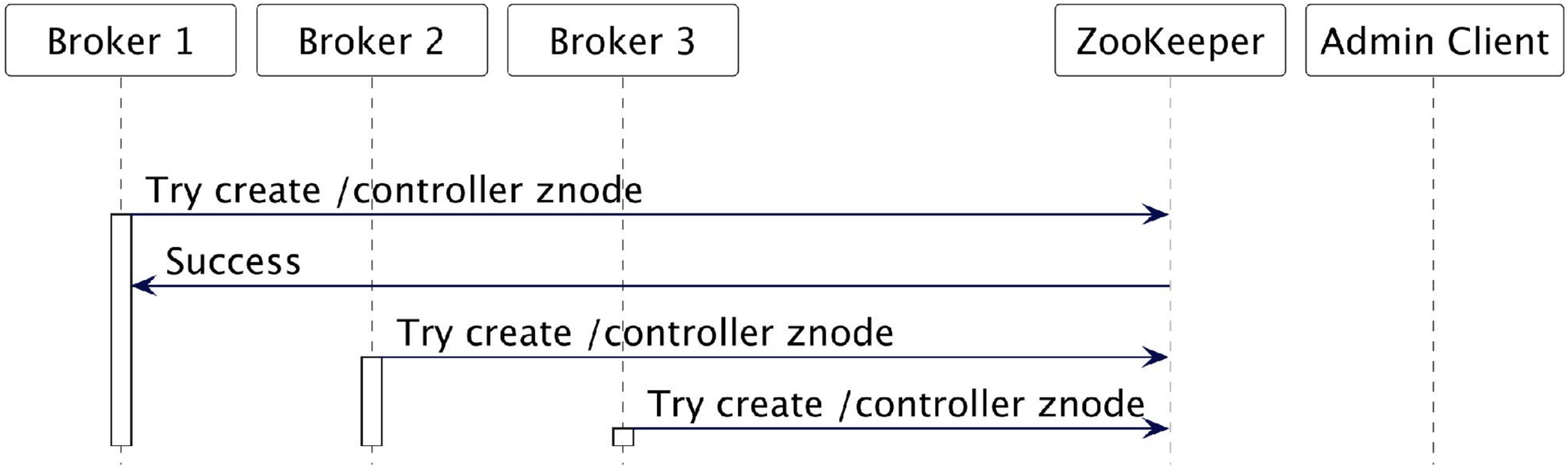
One of the brokers is chosen as the Controller – Step 1



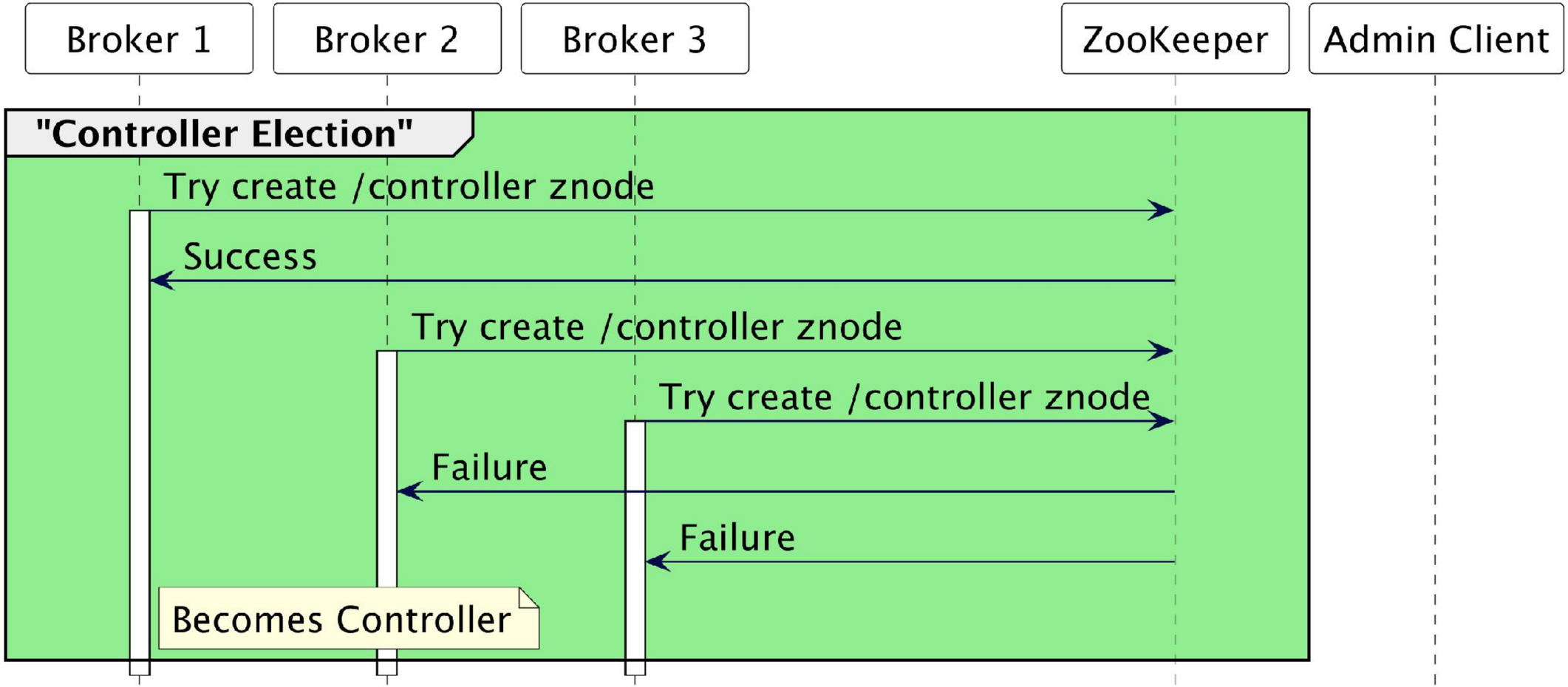
One of the brokers is chosen as the Controller – Step 2



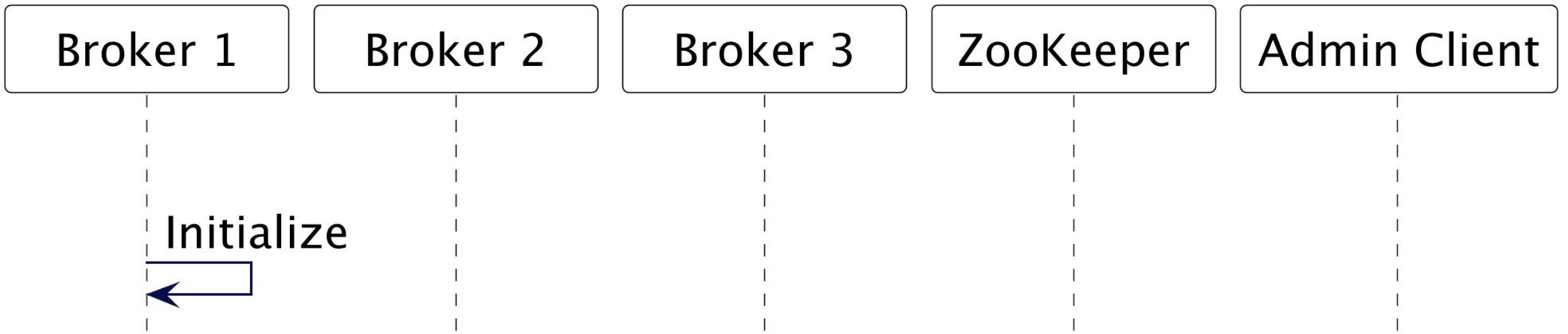
One of the brokers is chosen as the Controller – Step 3



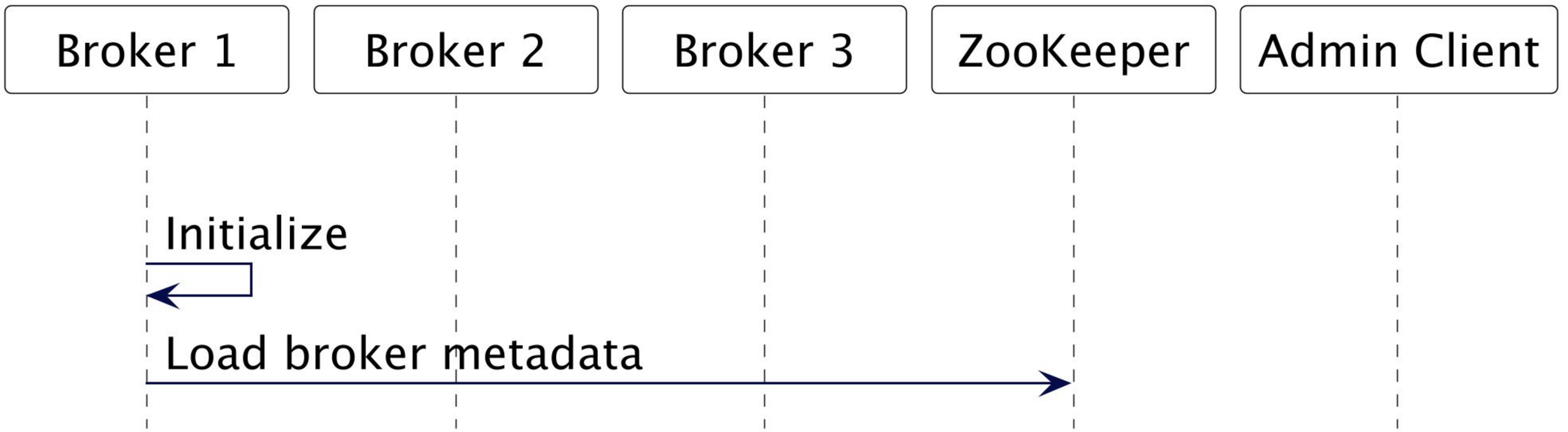
One of the brokers is chosen as the Controller - Step 4



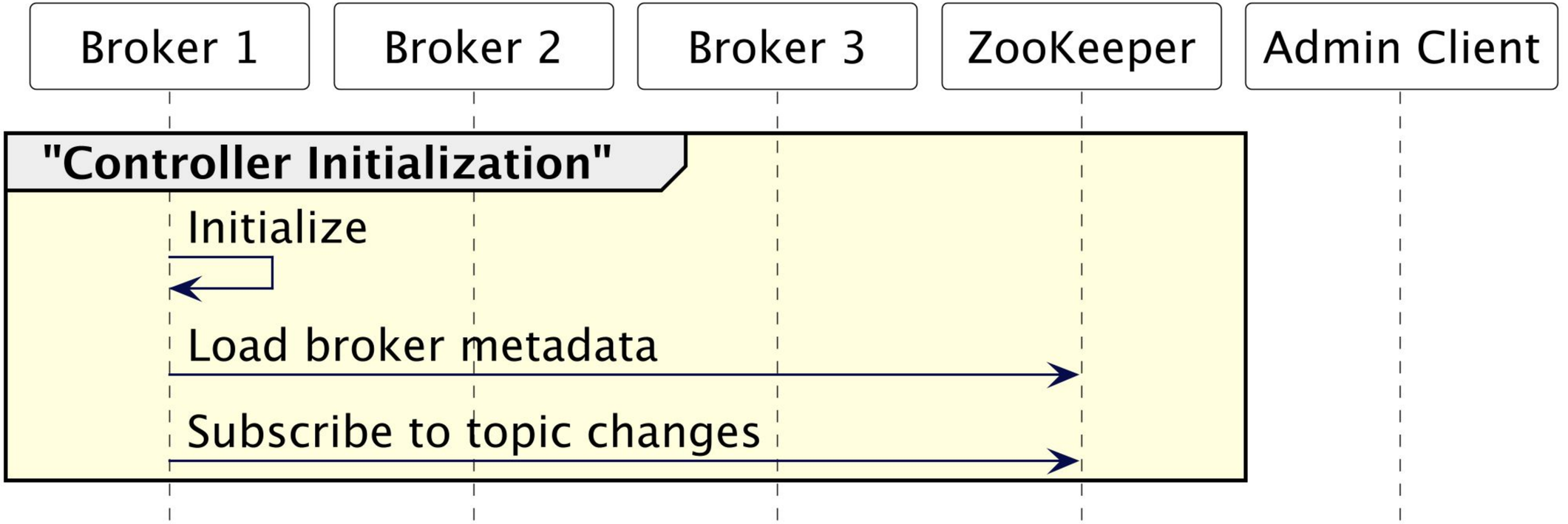
One of the brokers is chosen as the Controller – Step 5



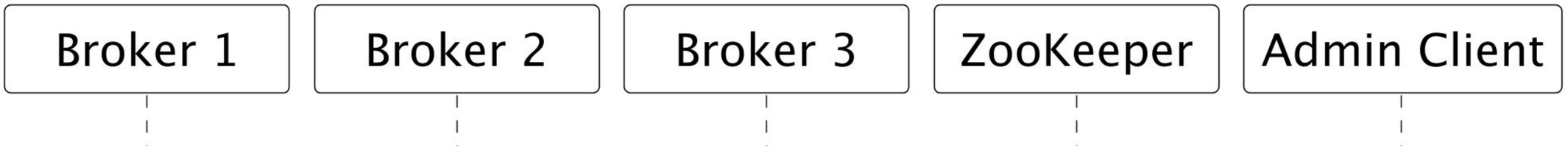
One of the brokers is chosen as the Controller – Step 6



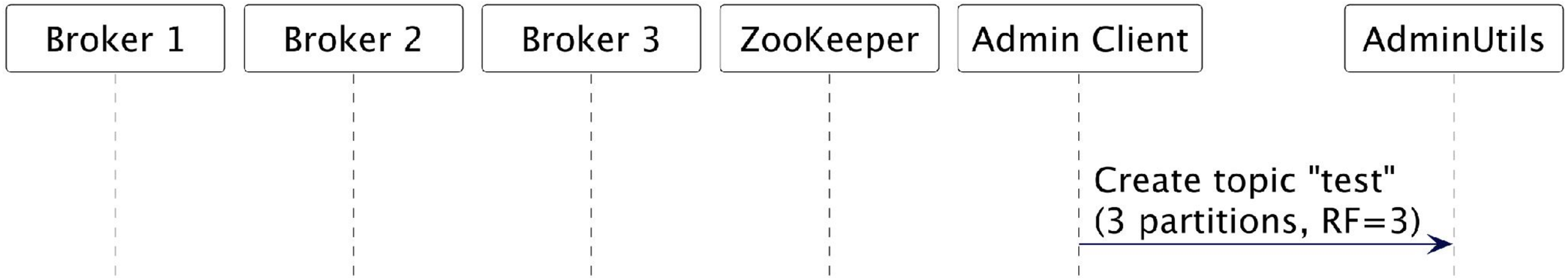
One of the brokers is chosen as the Controller – Step 7



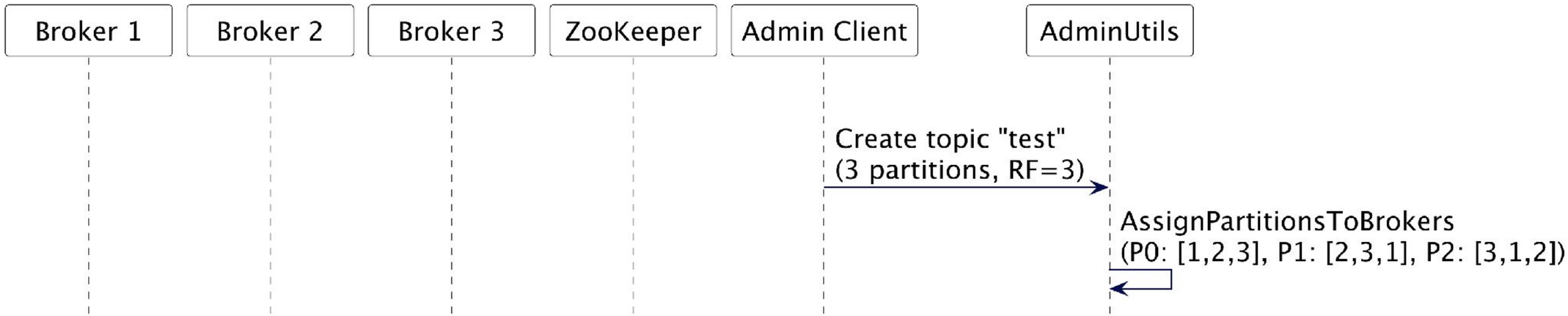
Topic Creation uses fixed number of partitions – Step 1



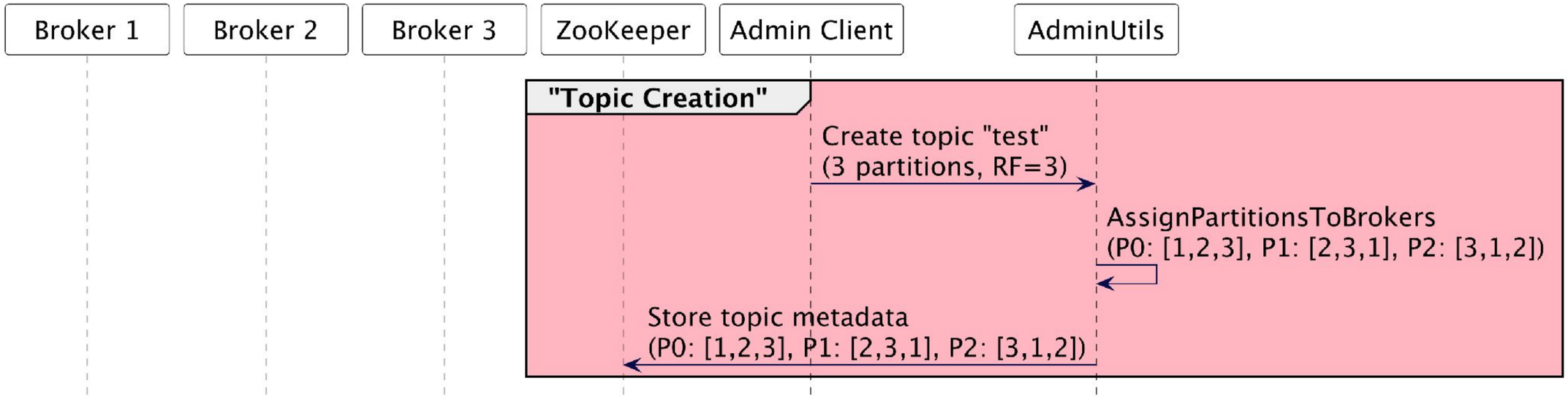
Topic Creation uses fixed number of partitions – Step 2



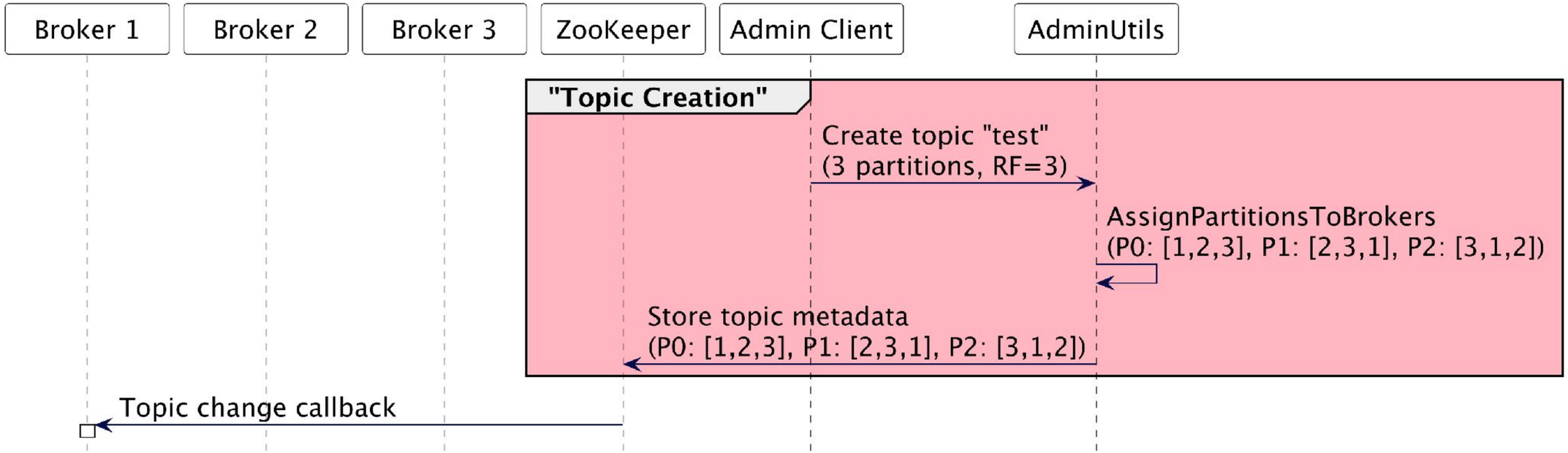
Topic Creation uses fixed number of partitions - Step 3



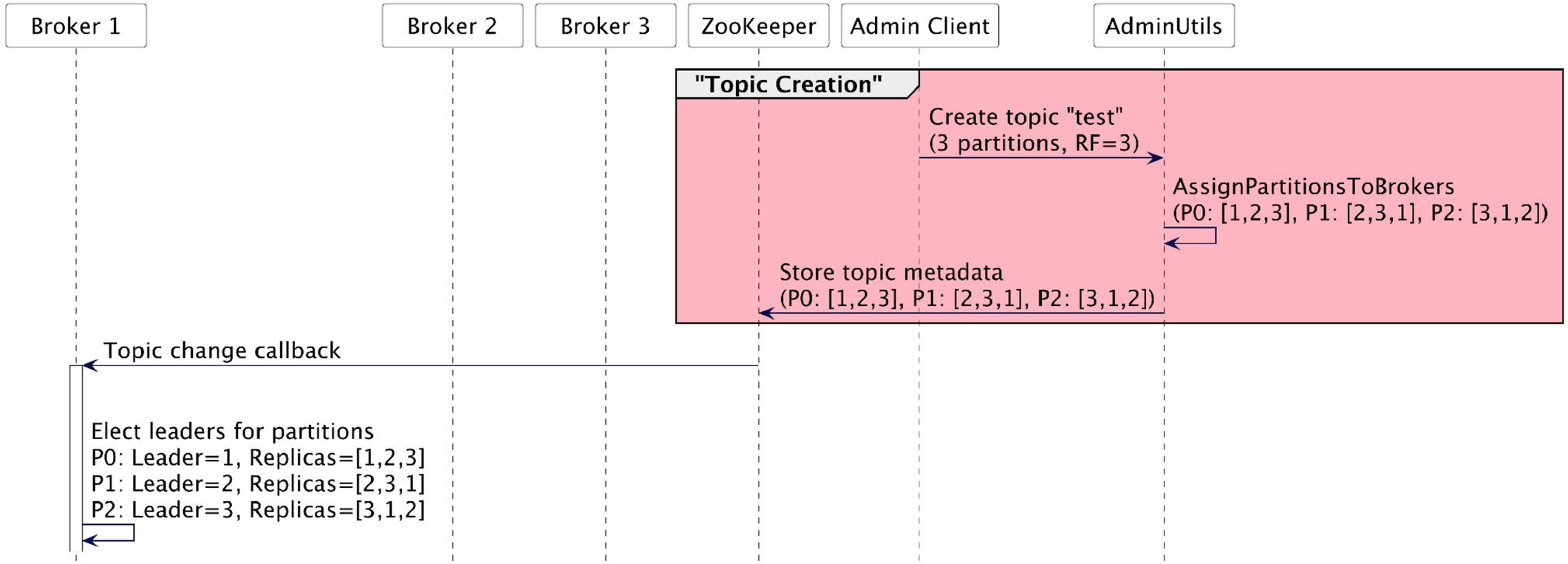
Topic Creation uses fixed number of partitions – Step 4



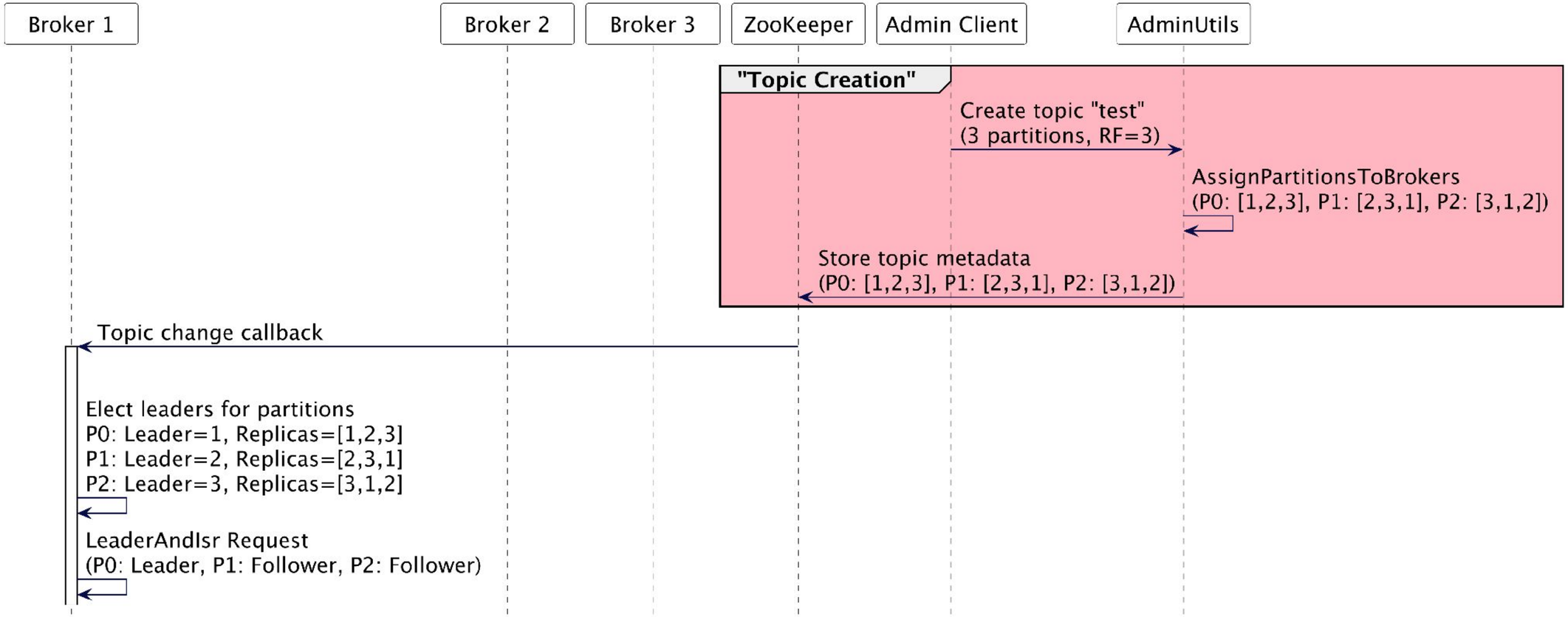
Topic Creation uses fixed number of partitions – Step 5



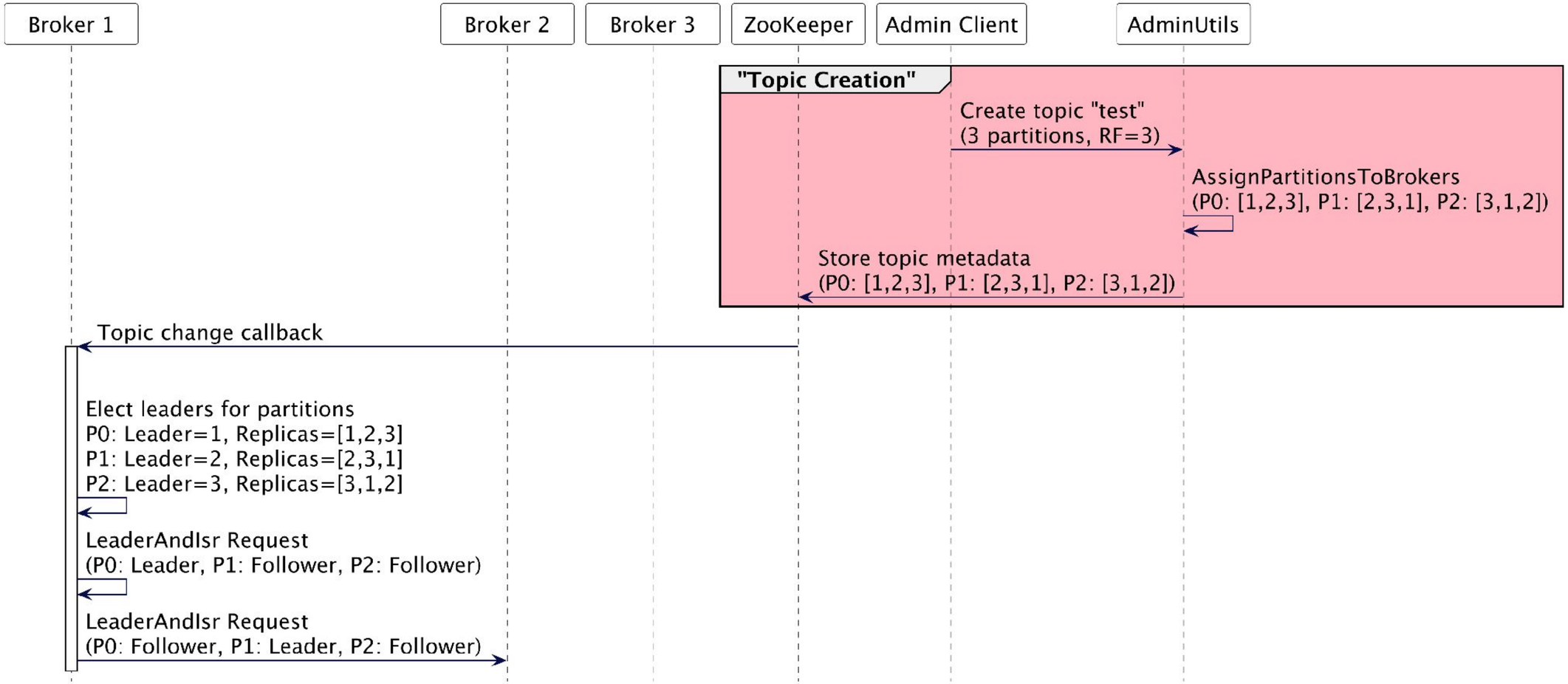
Topic Creation uses fixed number of partitions – Step 6



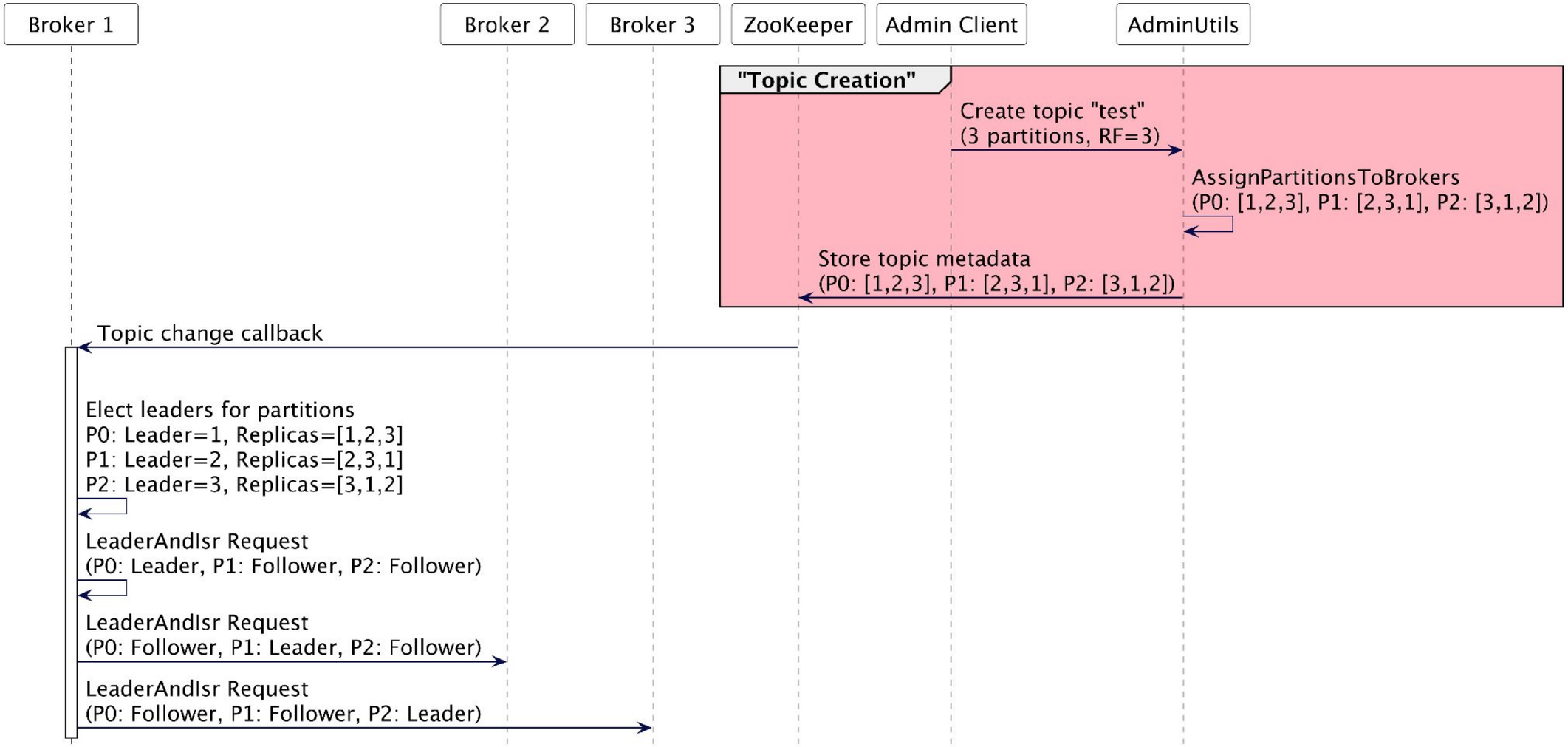
Topic Creation uses fixed number of partitions – Step 7



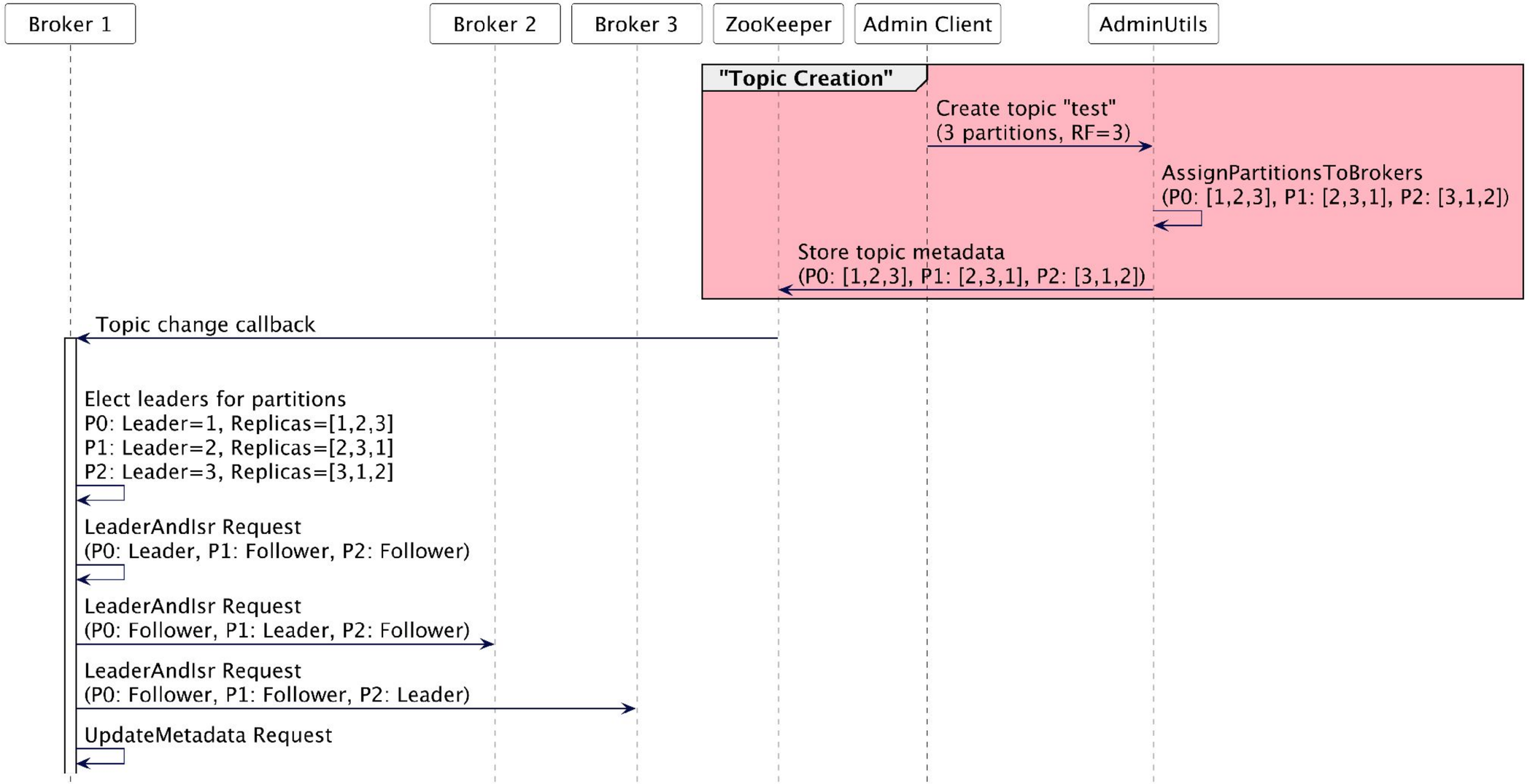
Topic Creation uses fixed number of partitions – Step 8



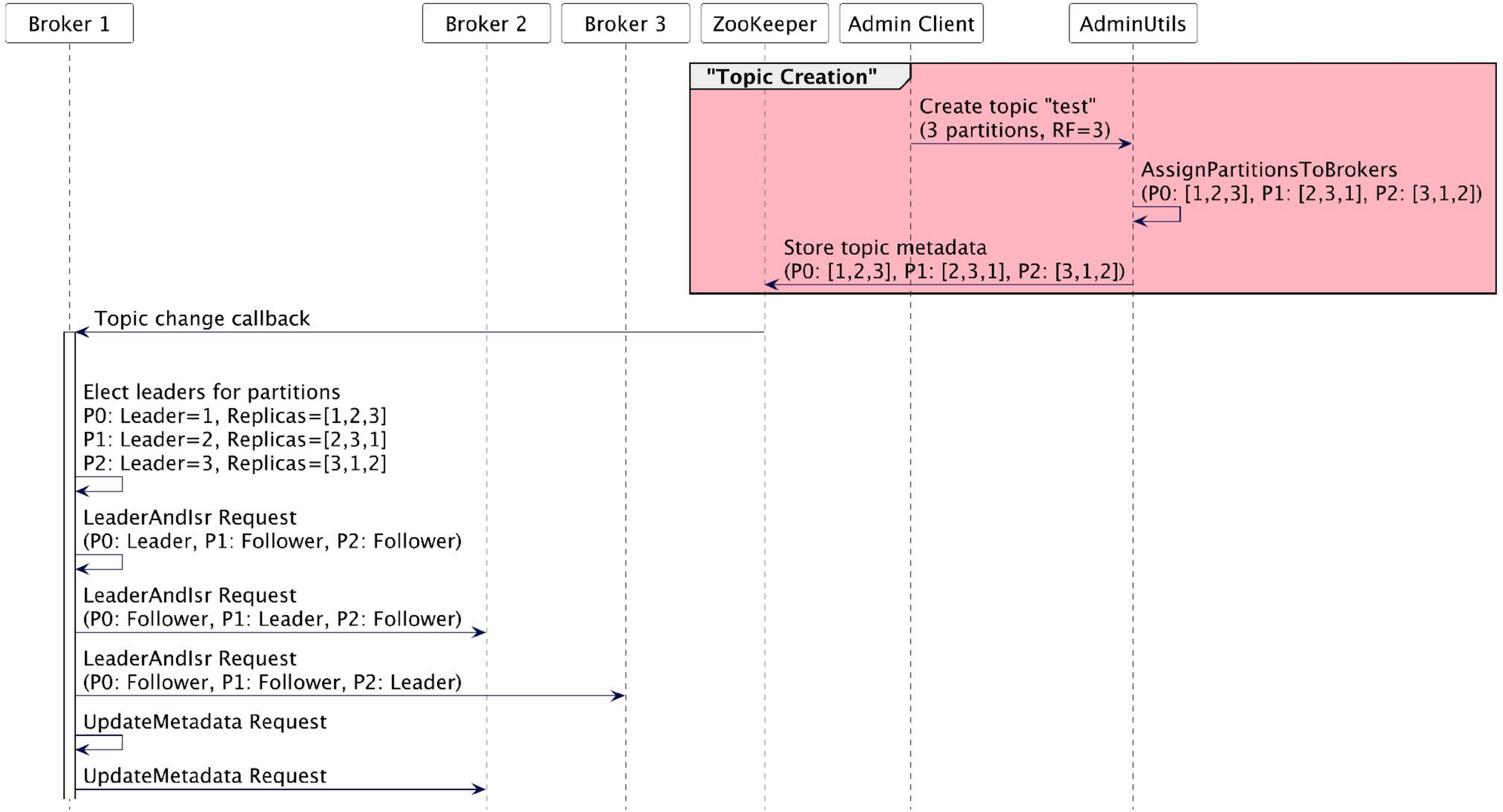
Topic Creation uses fixed number of partitions - Step 9



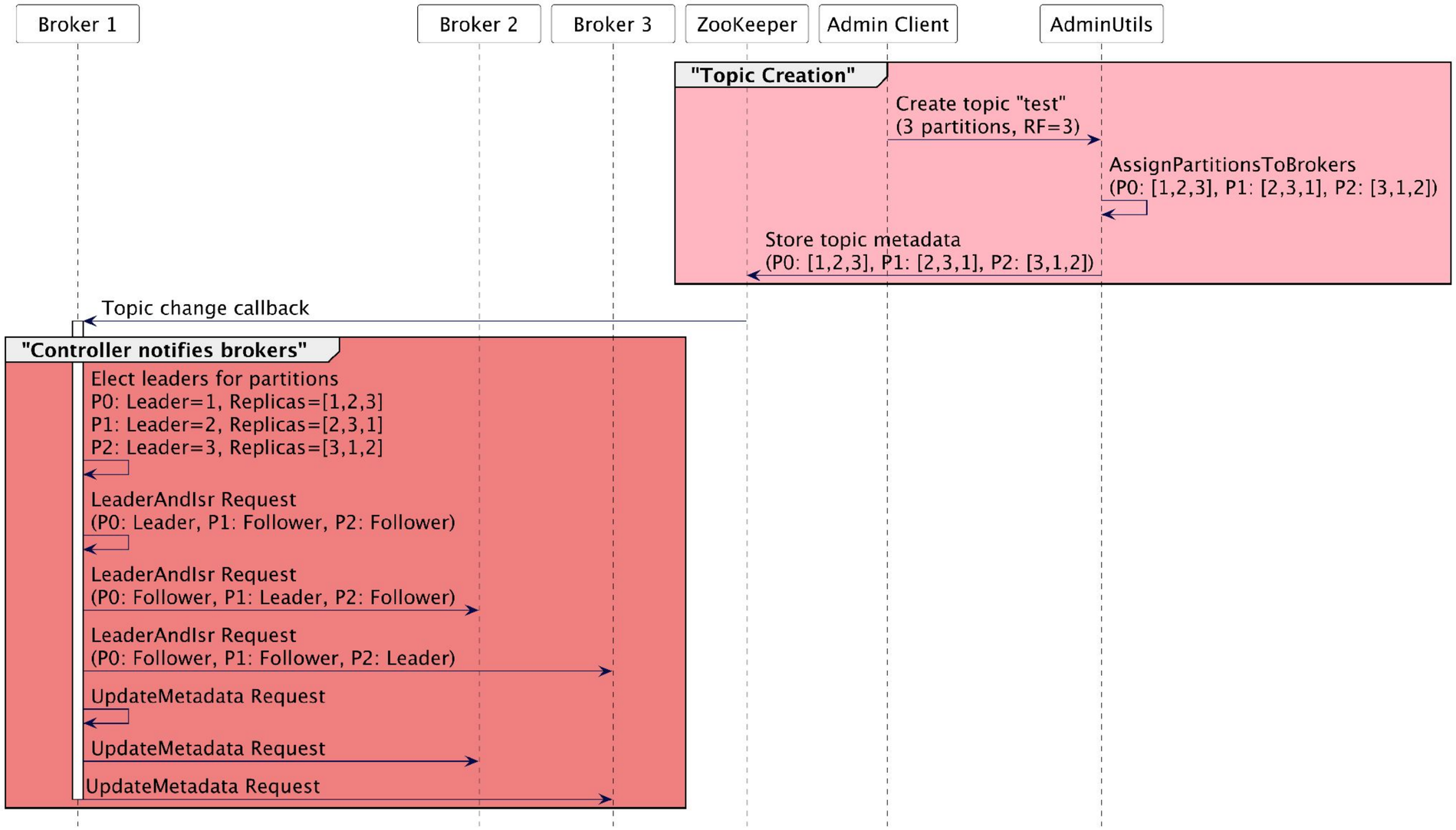
Topic Creation uses fixed number of partitions - Step 10



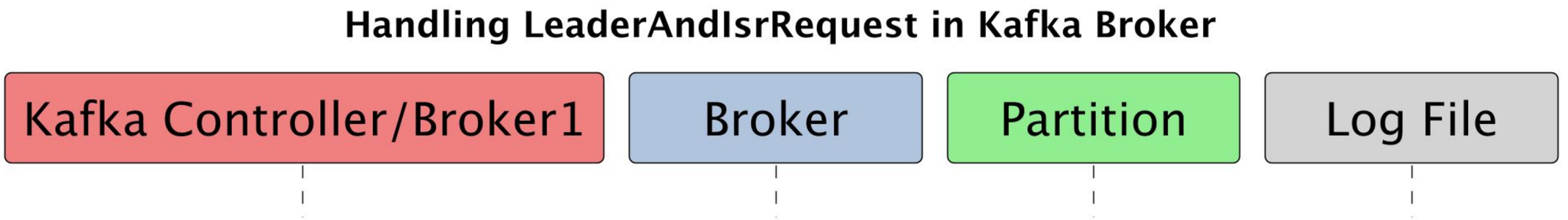
Topic Creation uses fixed number of partitions - Step 11



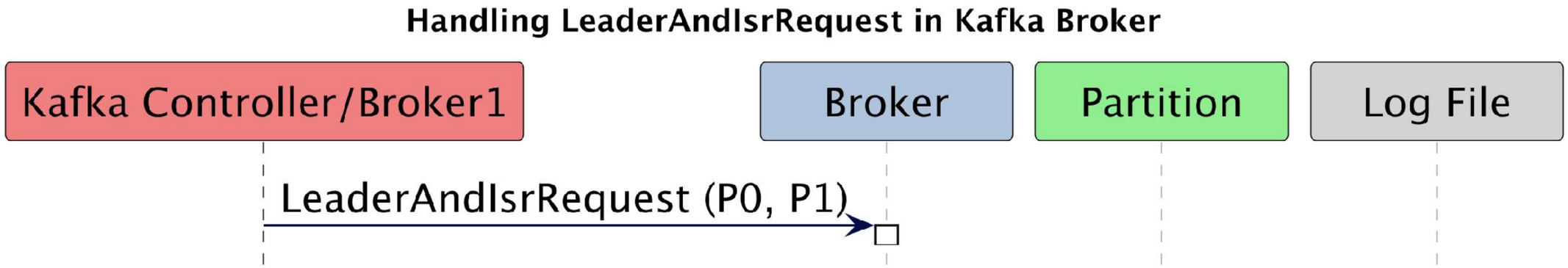
Topic Creation uses fixed number of partitions - Step 12



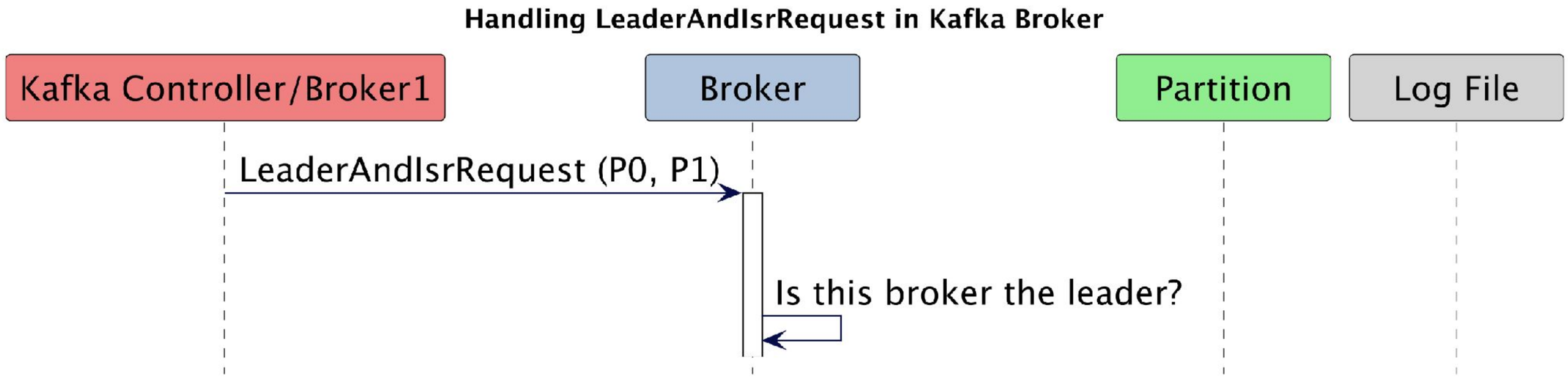
Brokers handle LeaderAndIsrRequest – Step 1



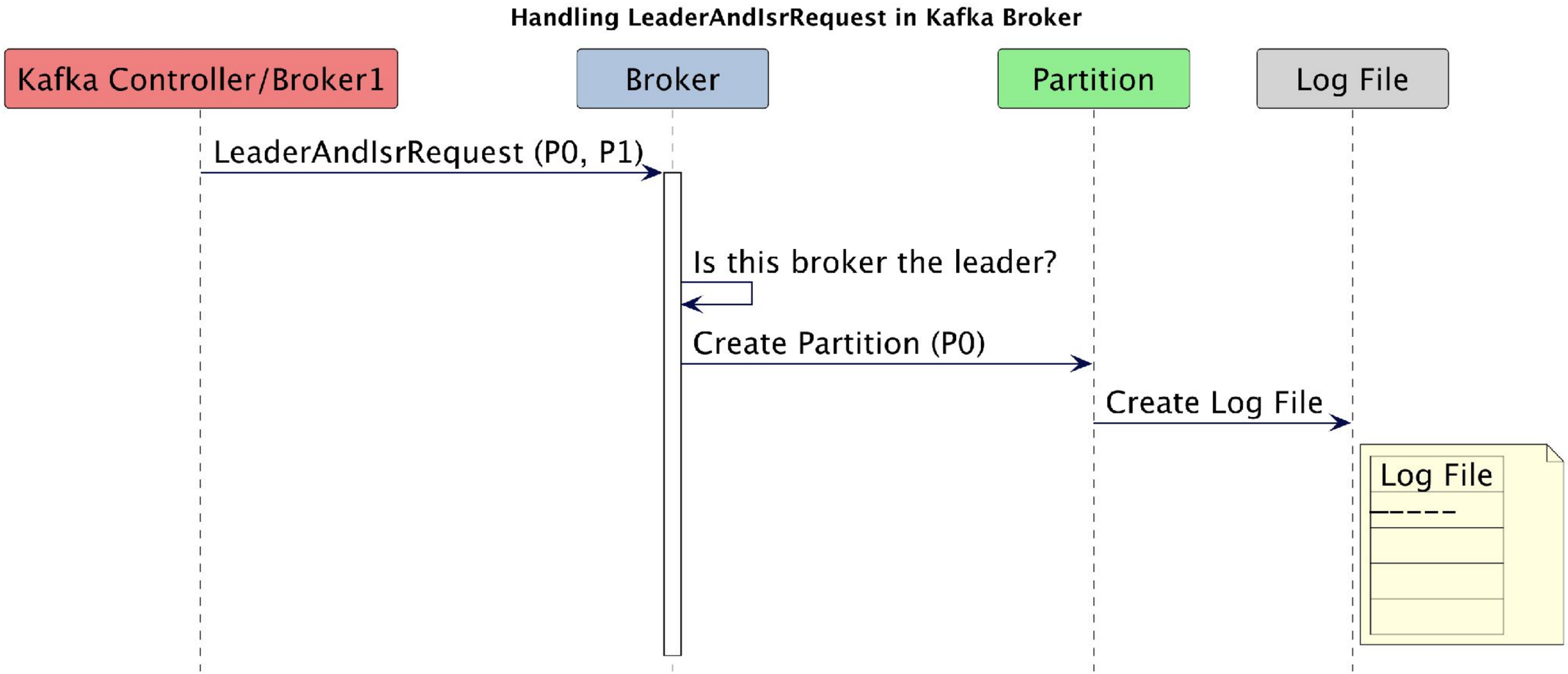
Brokers handle LeaderAndIsrRequest – Step 2



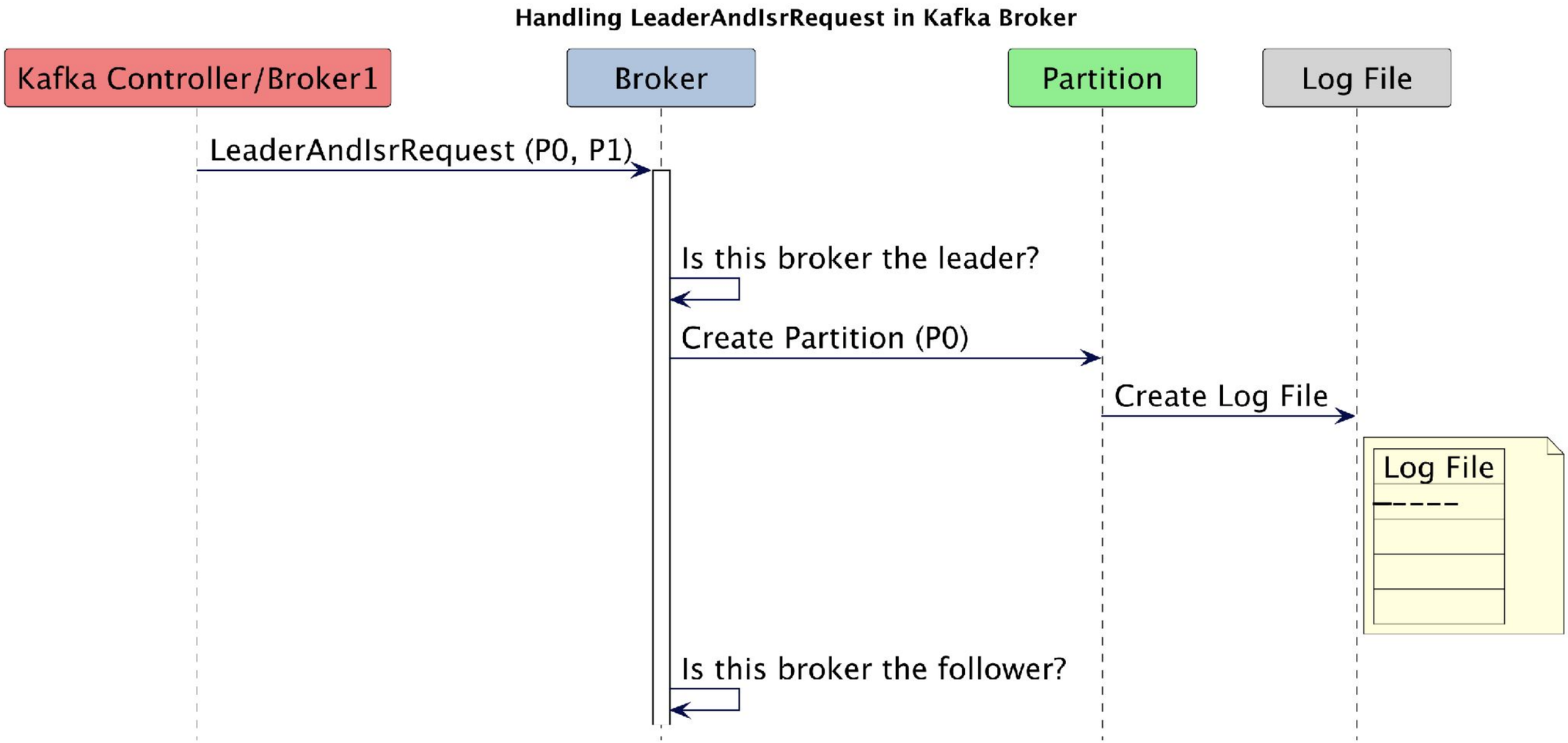
Brokers handle LeaderAndIsrRequest – Step 3



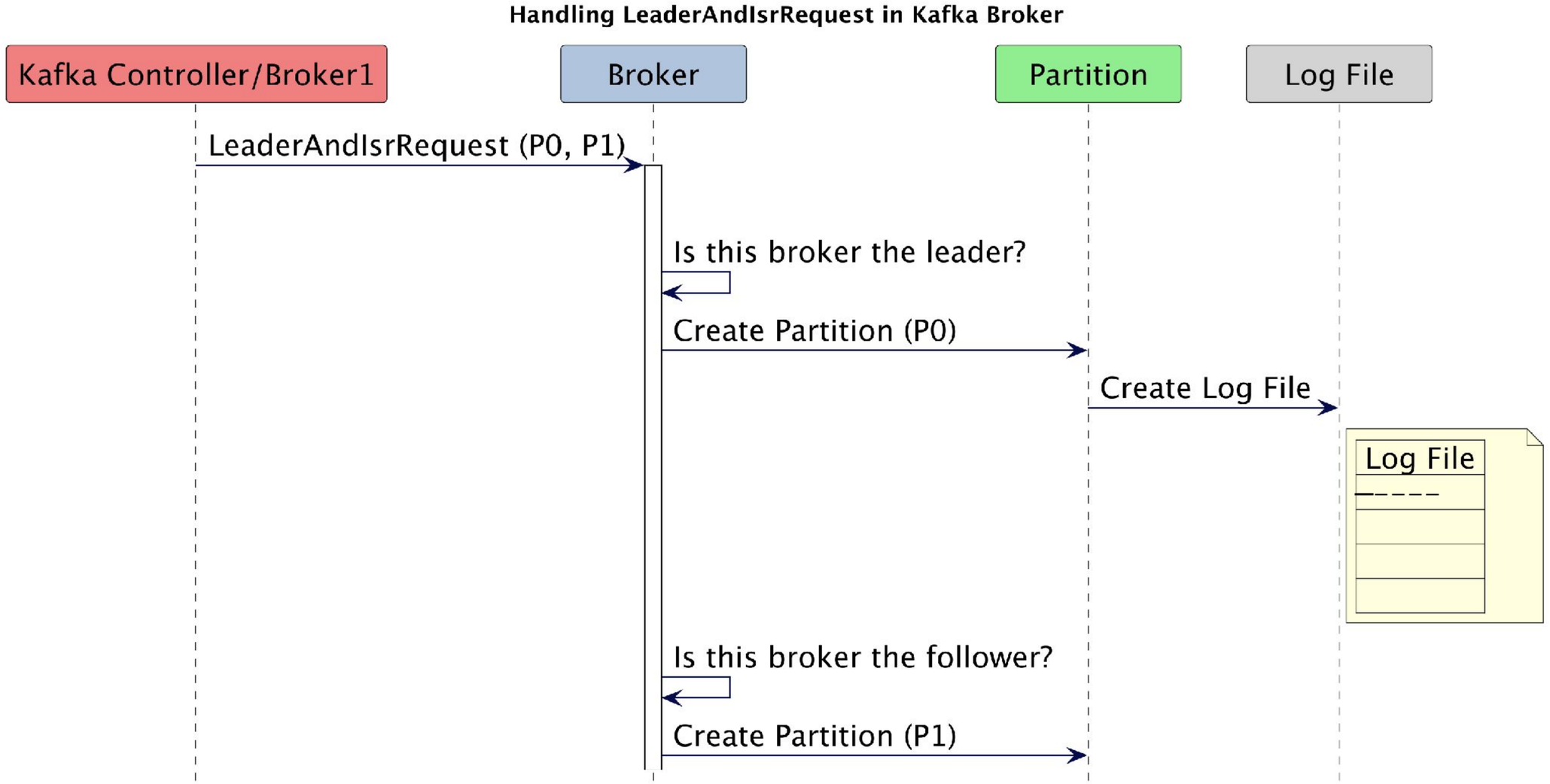
Brokers handle LeaderAndIsrRequest – Step 4



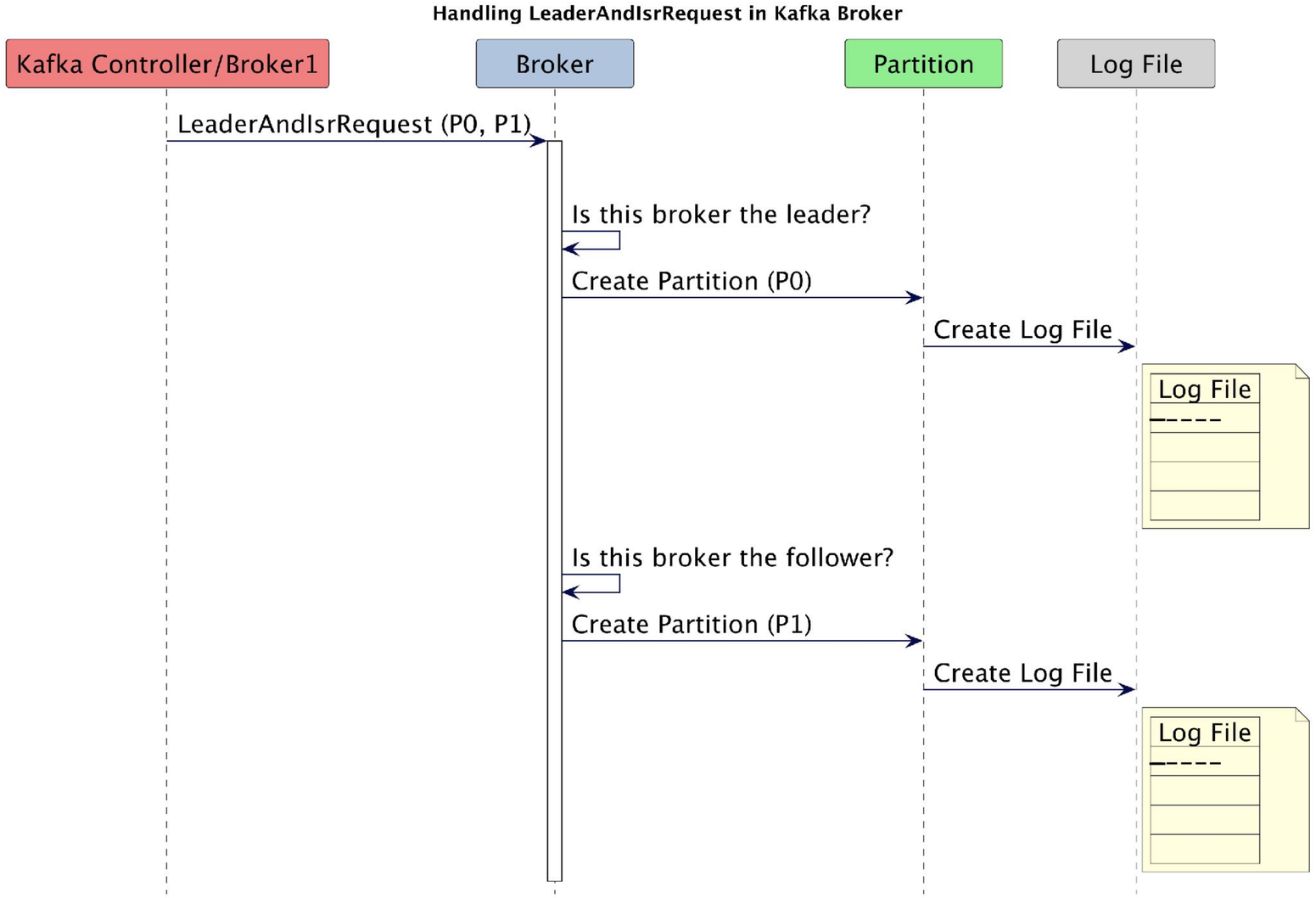
Brokers handle LeaderAndIsrRequest – Step 5



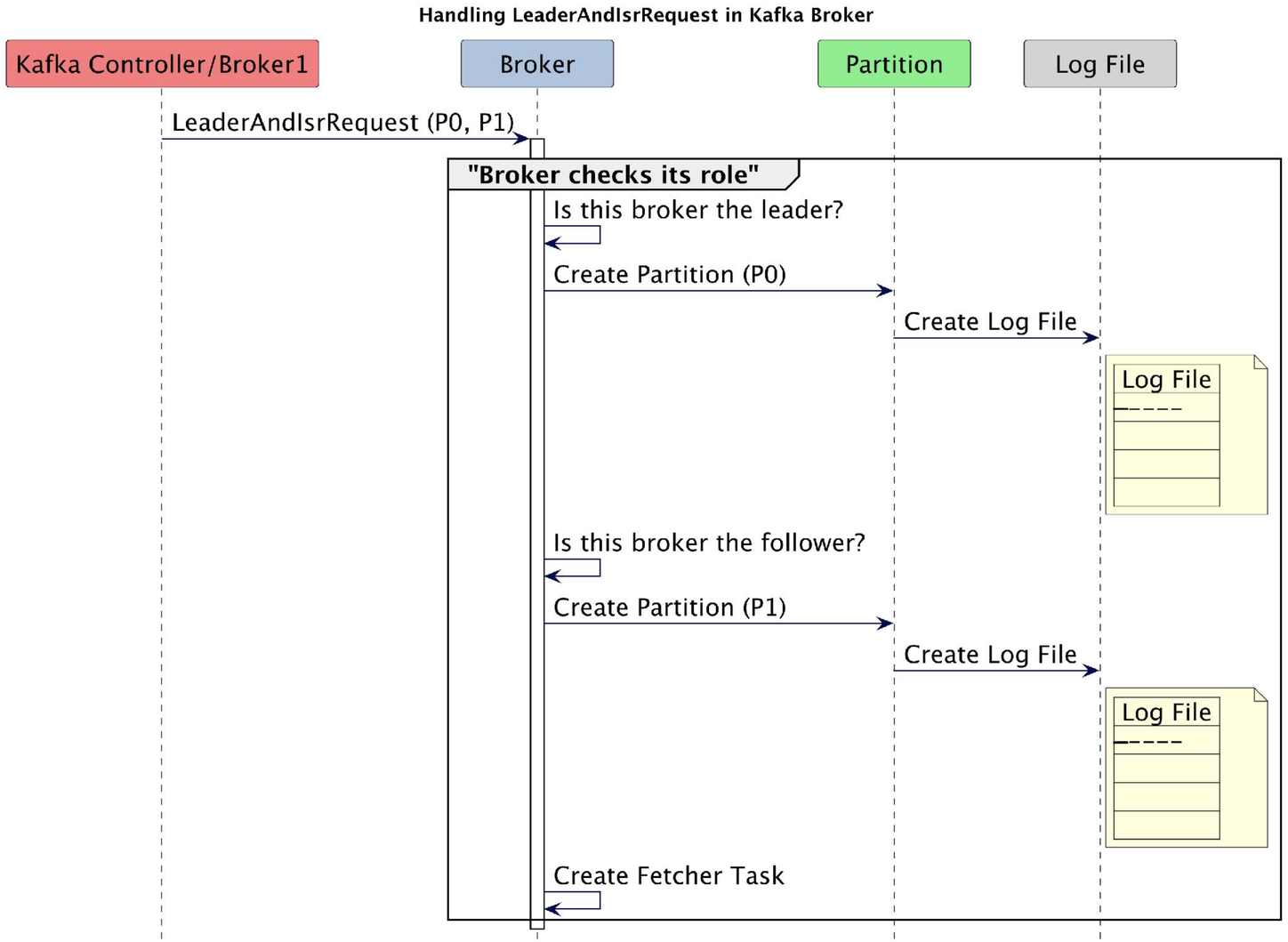
Brokers handle LeaderAndIsrRequest – Step 6



Brokers handle LeaderAndIsrRequest – Step 7



Brokers handle LeaderAndIsrRequest – Step 8



Assignment: Partition Change Detection

- Register a callback in the broker to detect topic partition changes from ZooKeeper / Consistent Core
- Parse updated partition replica assignments from metadata event

Assignment: Leader & Follower Election

- The callback for topic partition changes should elect leader and followers for partitions
- Update local state and log files accordingly

Assignments: Controller Requests & Epoch Fencing

- Send leader/follower decisions to specific brokers: LeaderAndIsr request (Test using stub Network Handler)
- Send UpdateMetadata request to all live brokers (Test using stub Network Handler)
- Stamp both requests with the current controllerEpoch, and reject stale epochs in the receiver

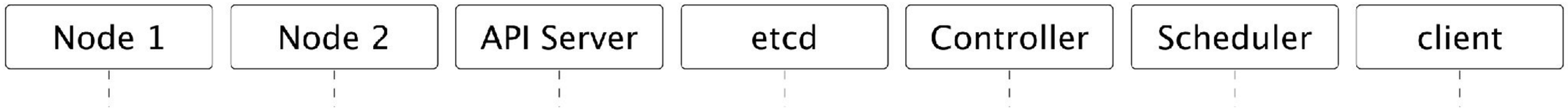
Two controllers at once

- A controller partitioned from the core keeps believing it is still the controller
- A new controller is elected; the old one returns and issues its own LeaderAndIsr and UpdateMetadata requests
- Brokers now receive conflicting partition assignments from two sources
- Fix: every election increments an epoch — in Kafka, the controllerEpoch — carried on every request
- Brokers record the highest epoch seen and reject anything stamped with an older one

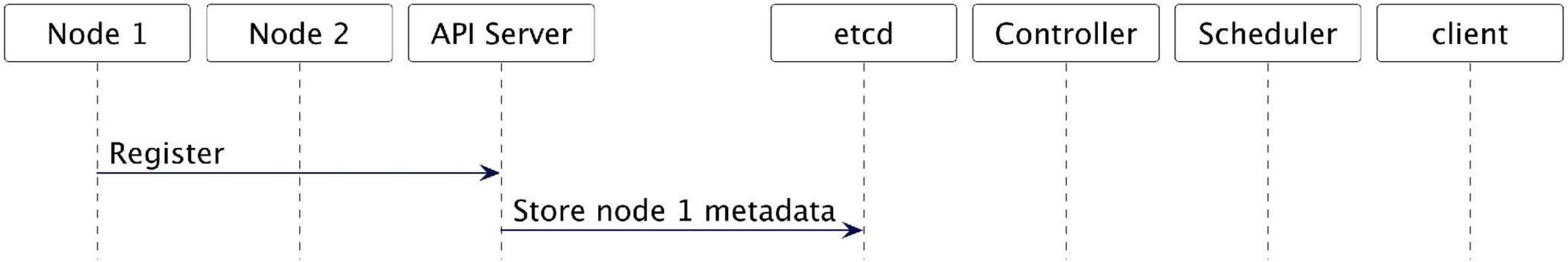
Kubernetes Controllers

- Controllers implement control loops monitoring actual vs desired state
- Each controller is responsible for one resource type
- Continuously reconcile desired state specified in specs
- Handle reconciliation and error recovery automatically
- Level-triggered (not edge-triggered) for reliability

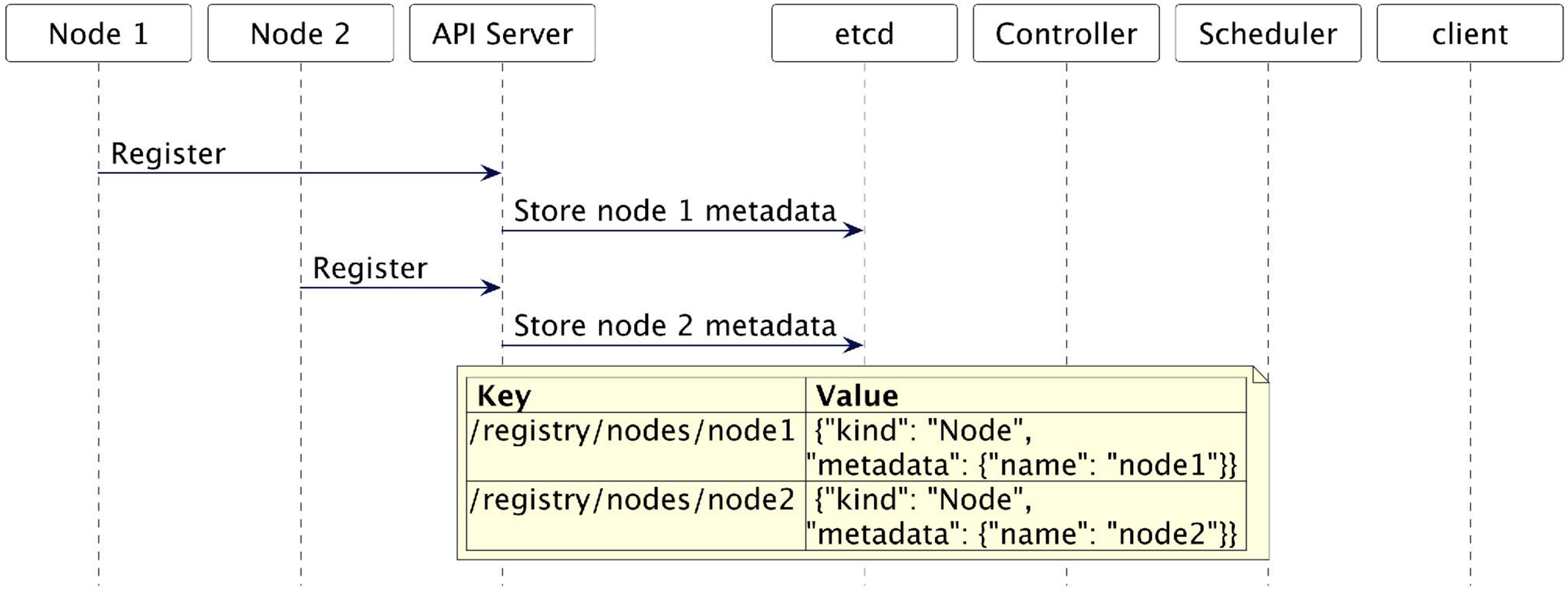
Controller Flow - Step 1



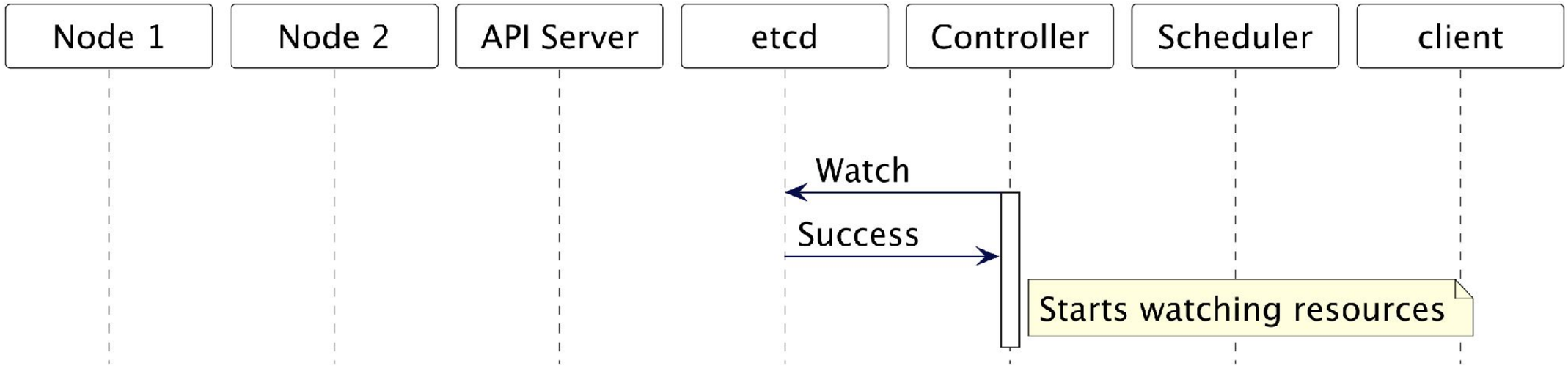
Controller Flow - Step 2



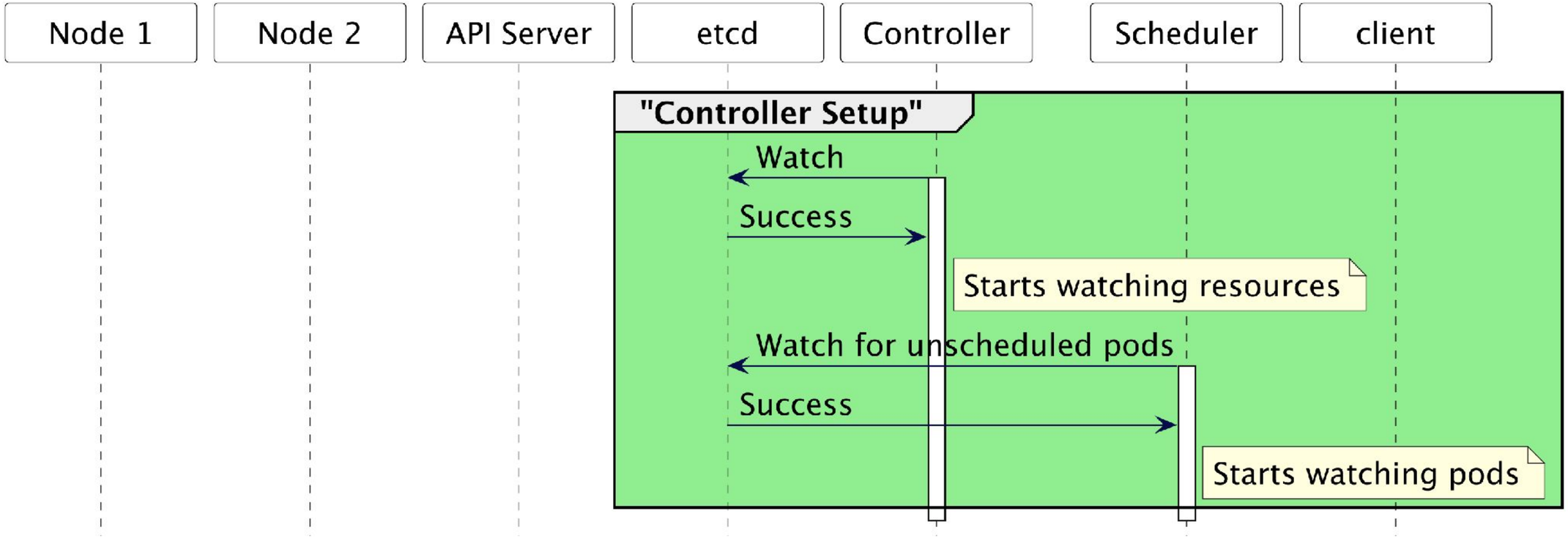
Controller Flow – Step 3



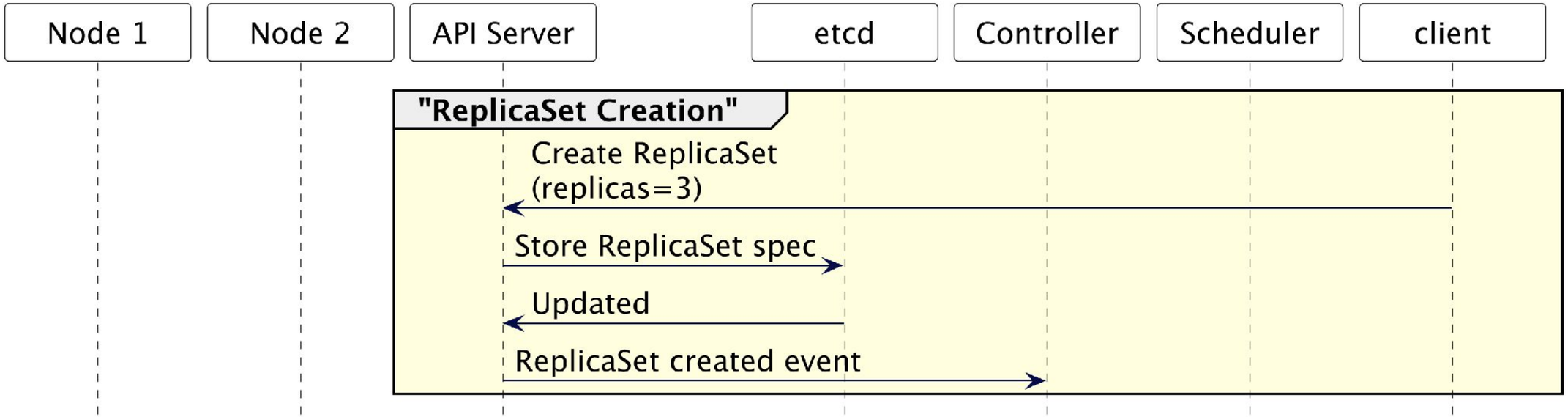
Controller Flow - Step 4



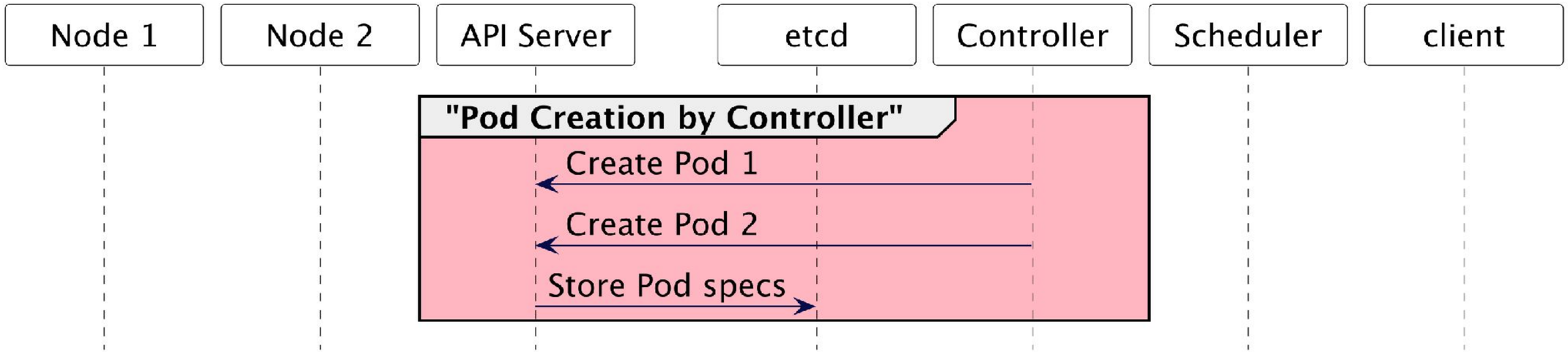
Controller Flow - Step 5



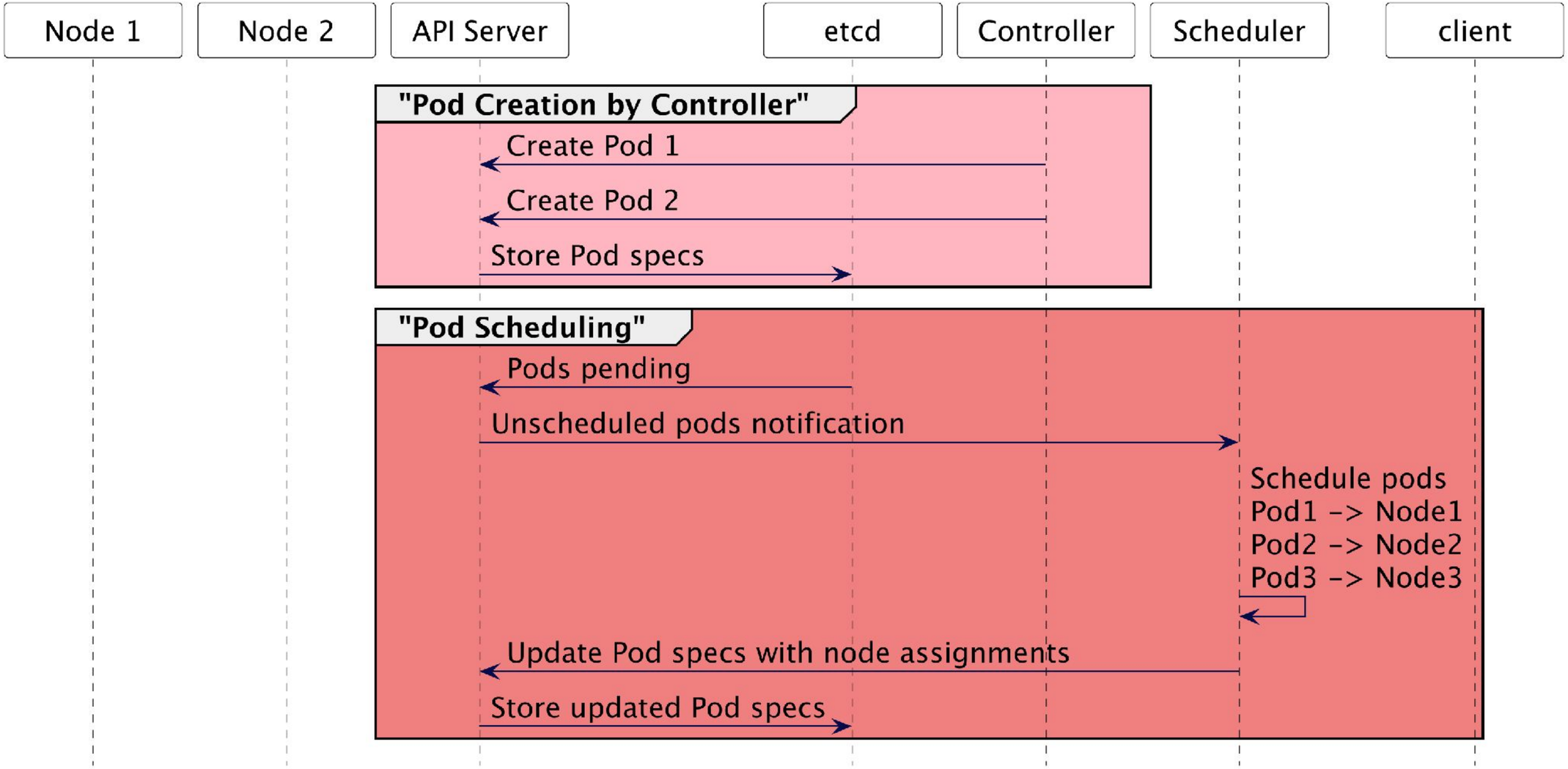
Controller Flow – Step 6



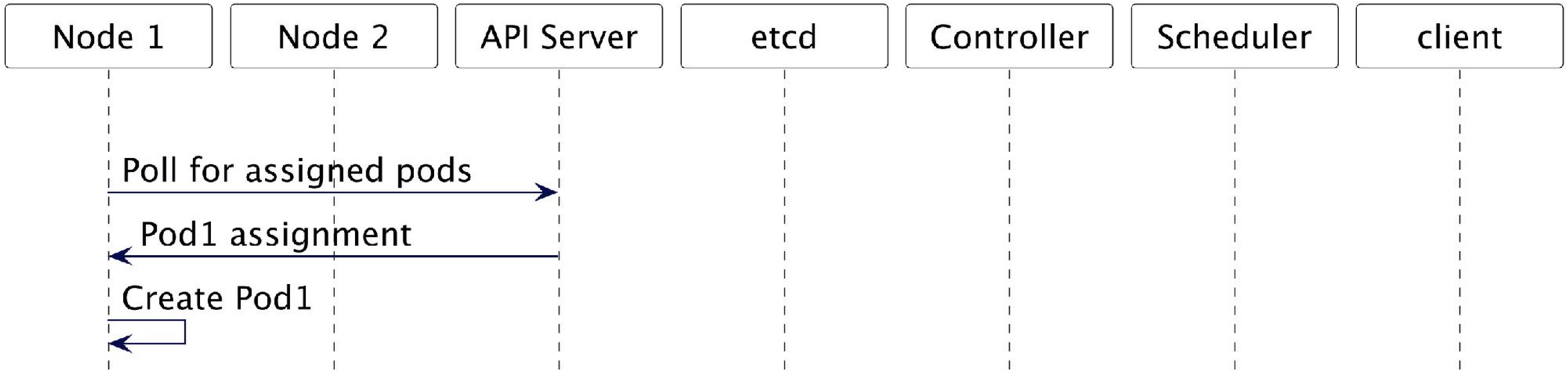
Controller Flow – Step 7



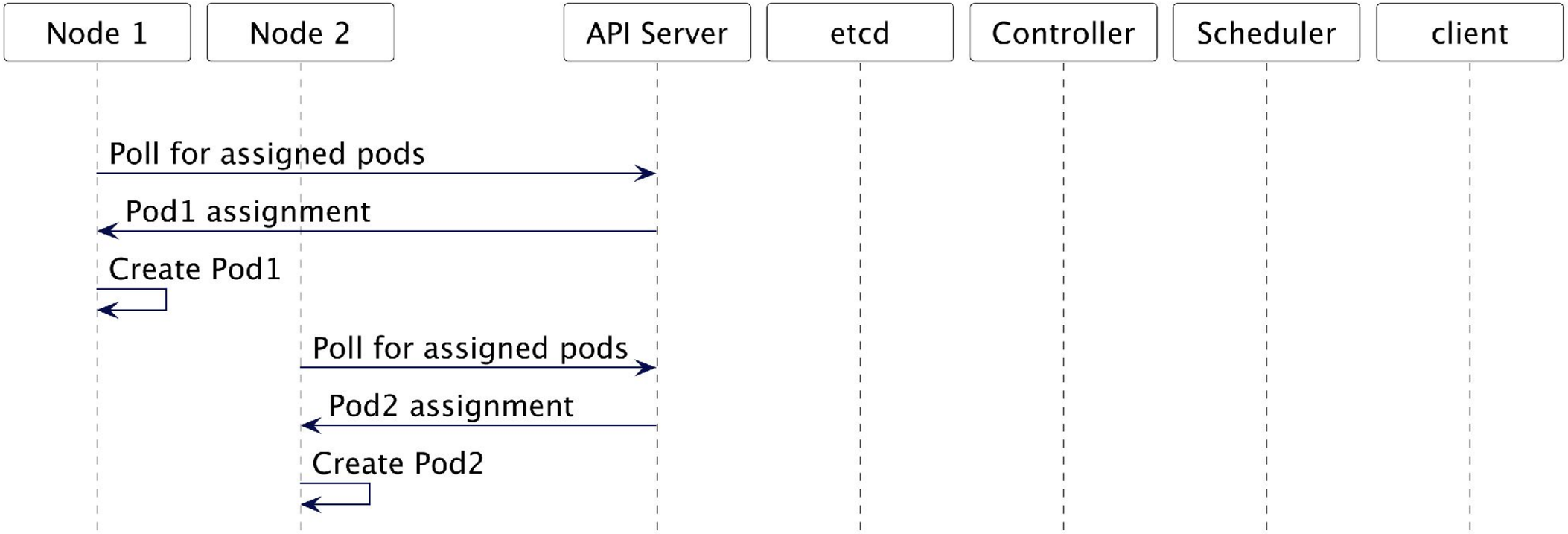
Controller Flow - Step 8



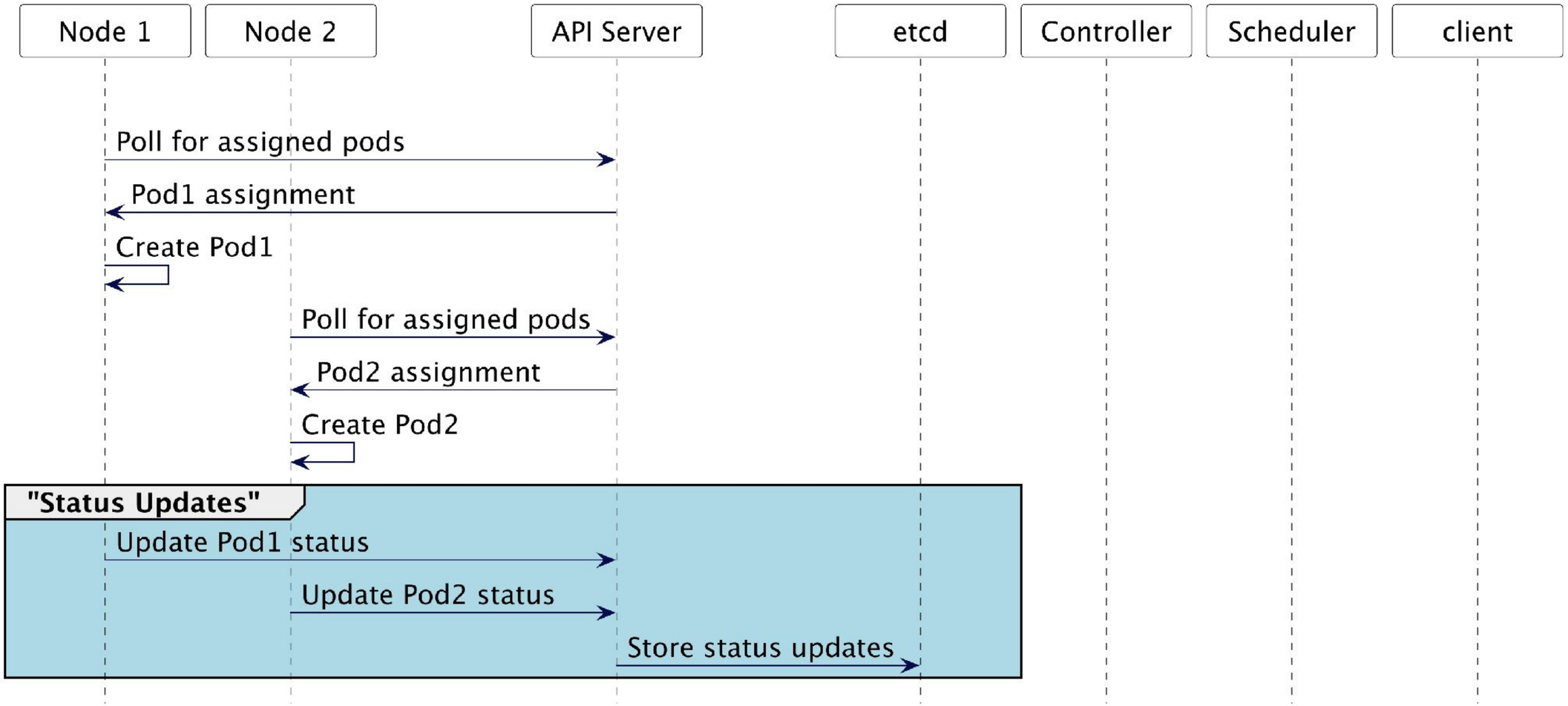
Controller Flow – Step 9



Controller Flow – Step 10



Controller Flow – Step 11

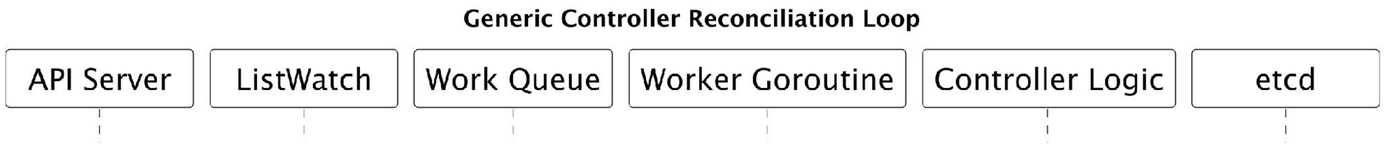


Controller Implementation Pattern

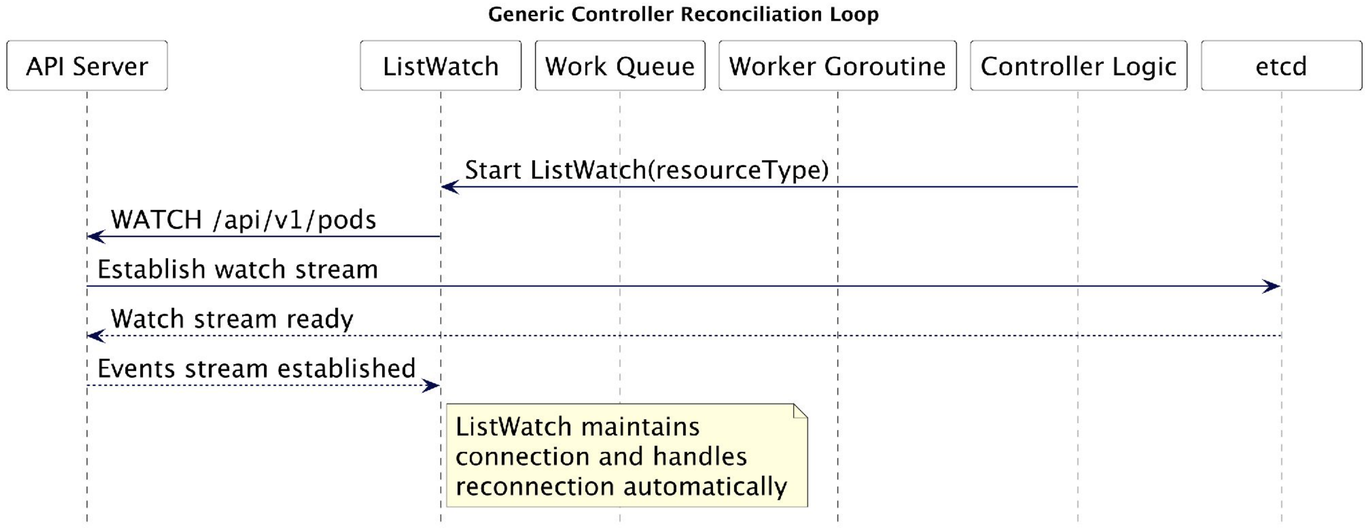
- Watch relevant resources using ListWatch pattern
- Work queue processes events asynchronously
- Reconcile function implements business logic
- Compare current state vs desired state
- Take actions to converge to desired state
- Update status to reflect current reality

Generic Controller Reconciliation Loop – Step 1

- ListWatch detects changes and adds items to work queue
- Worker goroutines process queue items by calling Reconcile()
- Reconcile() compares current vs desired state and takes actions
- Status updates reflect the new current state

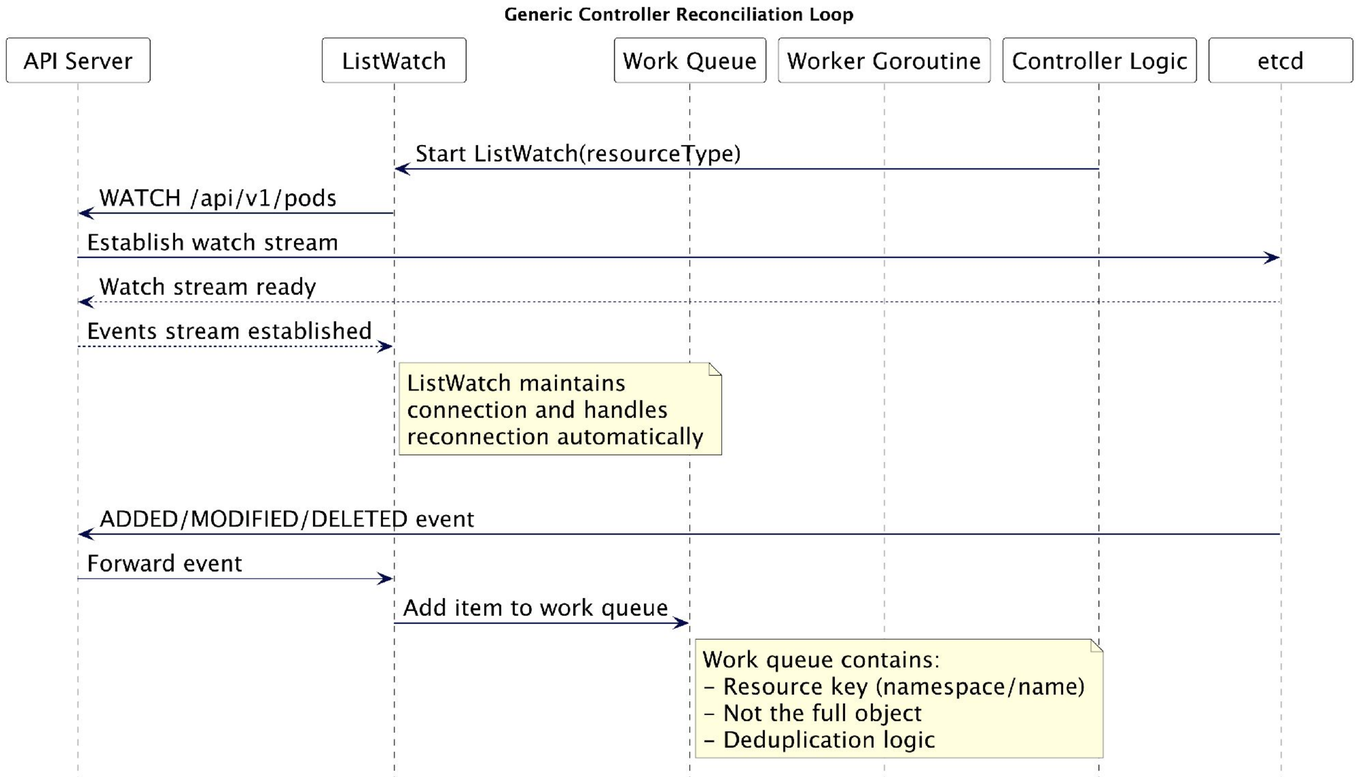


Generic Controller Reconciliation Loop – Step 2



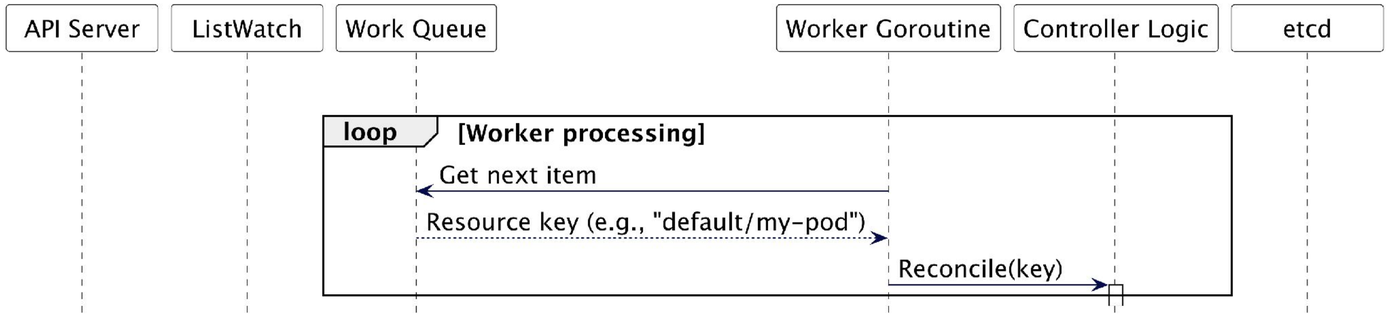
- ListWatch detects changes and adds items to work queue
- Worker goroutines process queue items by calling Reconcile()
- Reconcile() compares current vs desired state and takes actions
- Status updates reflect the new current state

Generic Controller Reconciliation Loop – Step 3



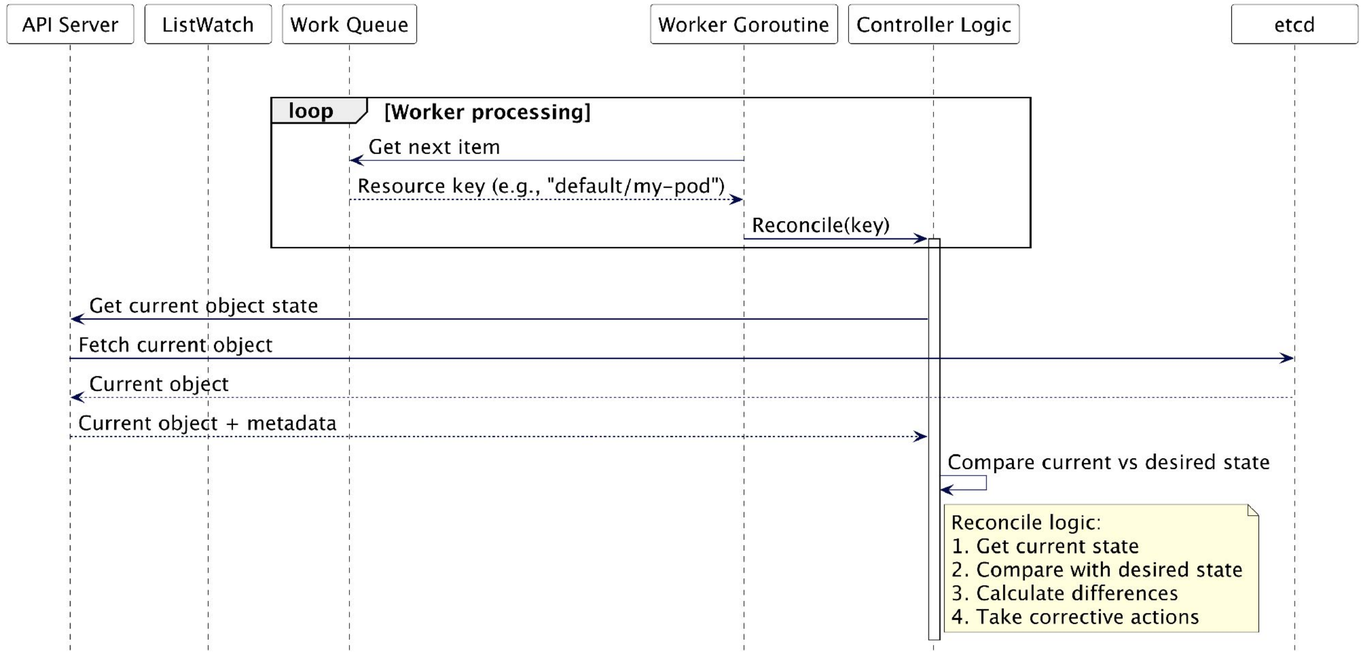
- ListWatch detects changes and adds items to work queue
- Worker goroutines process queue items by calling Reconcile()
- Reconcile() compares current vs desired state and takes actions
- Status updates reflect the new current state

Generic Controller Reconciliation Loop – Step 4



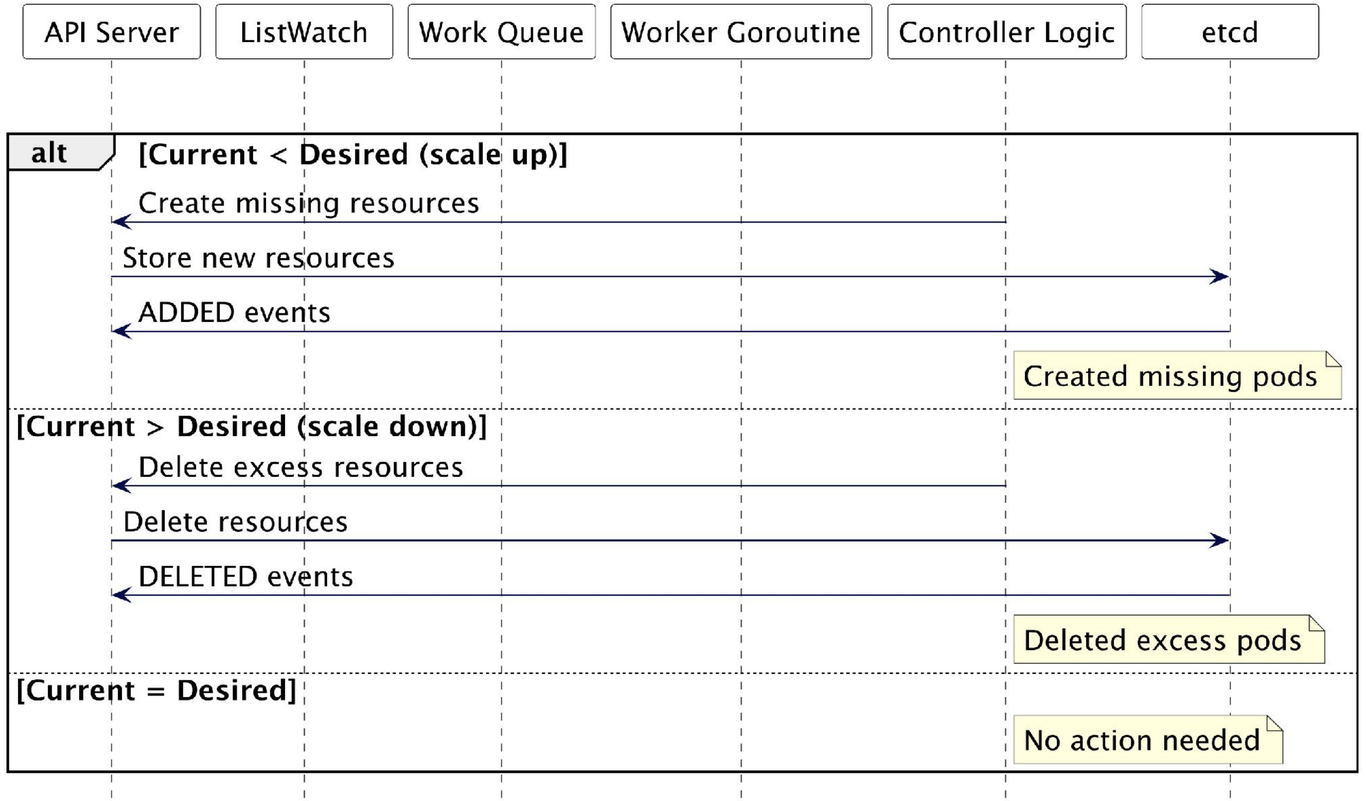
- ListWatch detects changes and adds items to work queue
- Worker goroutines process queue items by calling Reconcile()
- Reconcile() compares current vs desired state and takes actions
- Status updates reflect the new current state

Generic Controller Reconciliation Loop – Step 5



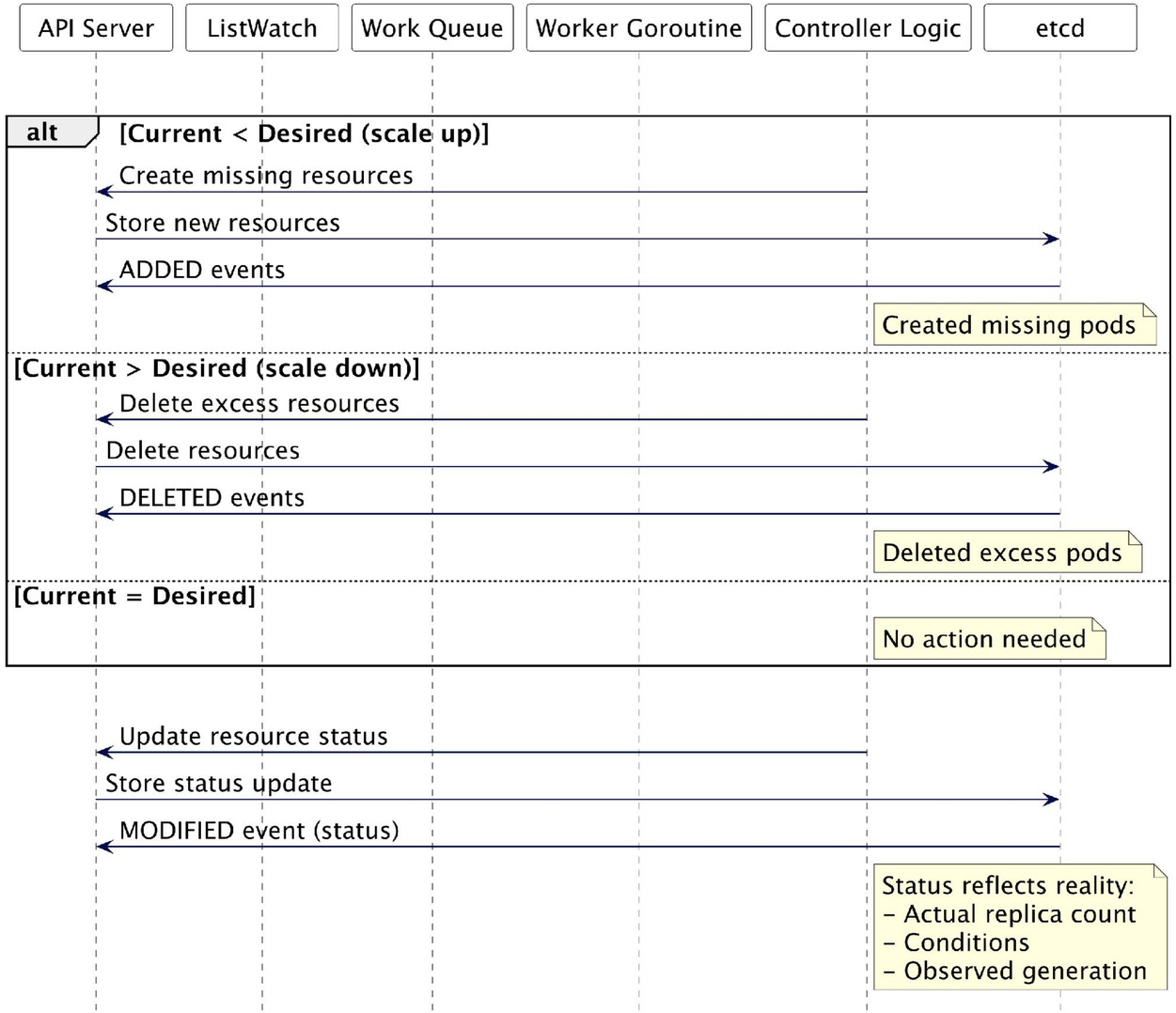
- ListWatch detects changes and adds items to work queue
- Worker goroutines process queue items by calling Reconcile()
- Reconcile() compares current vs desired state and takes actions
- Status updates reflect the new current state

Generic Controller Reconciliation Loop – Step 6



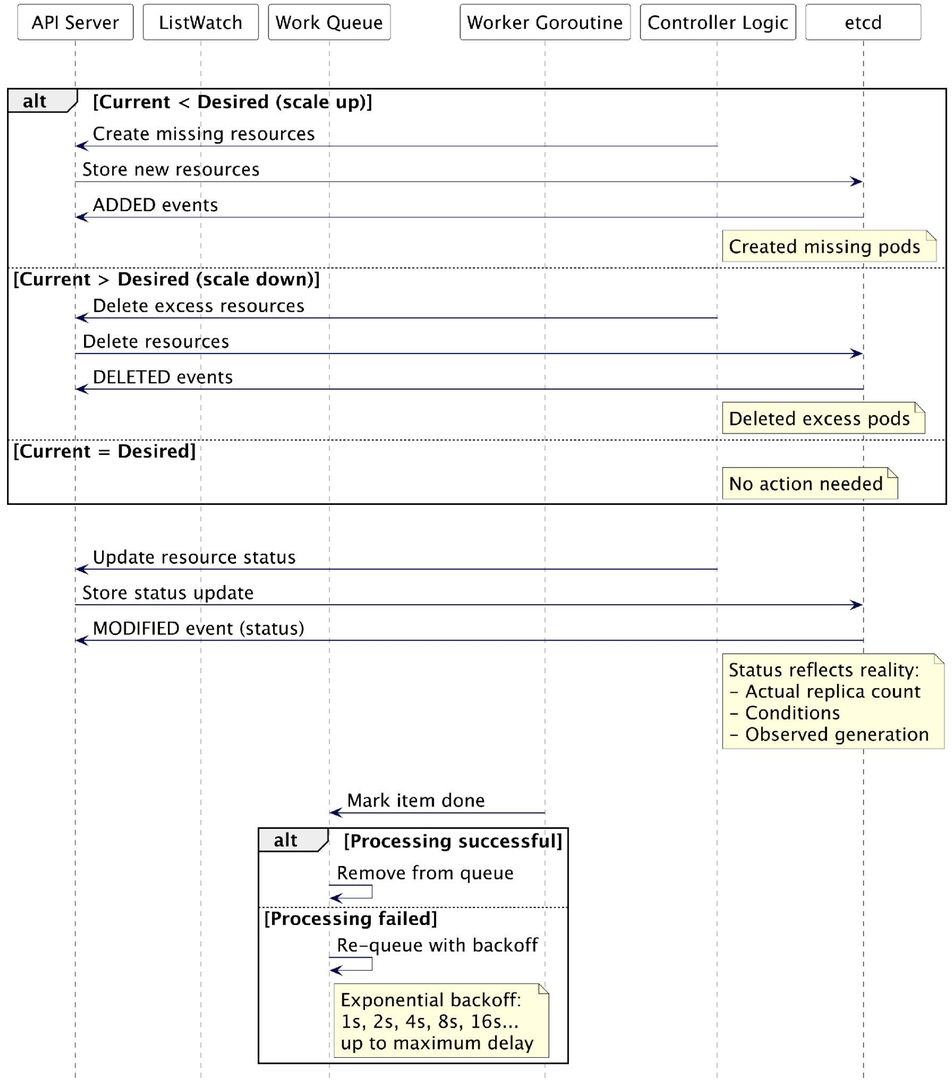
- ListWatch detects changes and adds items to work queue
- Worker goroutines process queue items by calling Reconcile()
- Reconcile() compares current vs desired state and takes actions
- Status updates reflect the new current state

Generic Controller Reconciliation Loop – Step 7



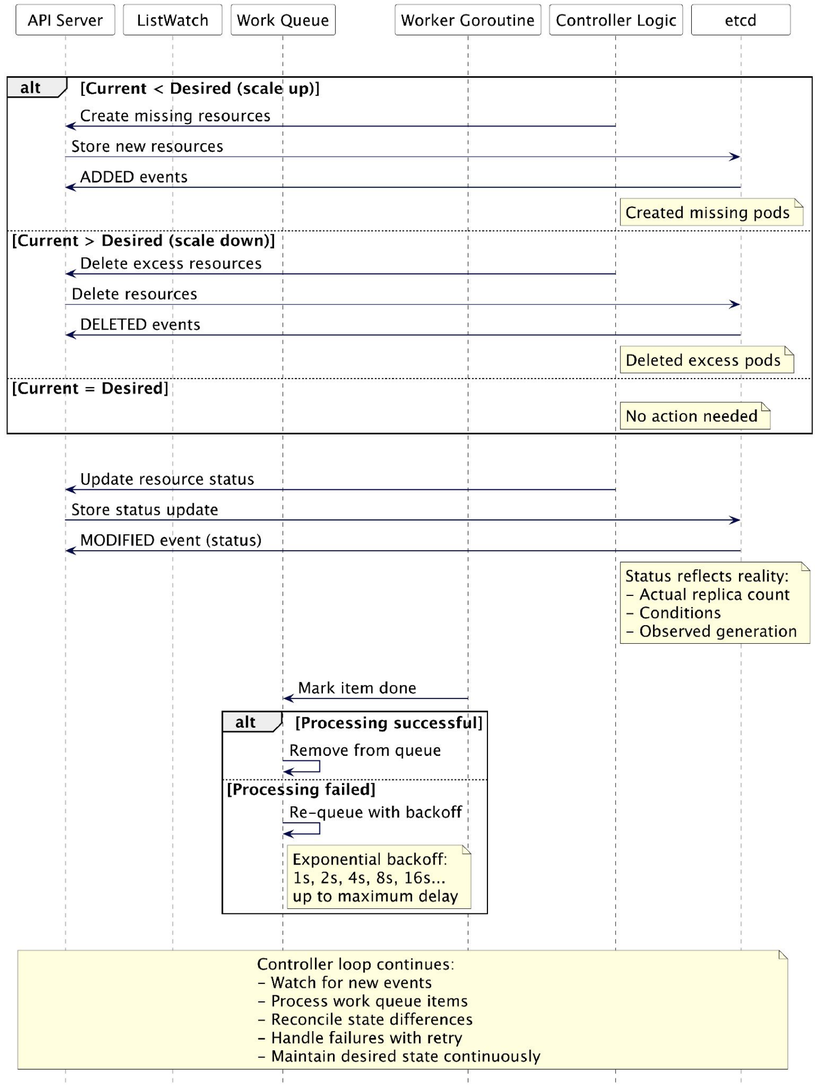
- ListWatch detects changes and adds items to work queue
- Worker goroutines process queue items by calling Reconcile()
- Reconcile() compares current vs desired state and takes actions
- Status updates reflect the new current state

Generic Controller Reconciliation Loop – Step 8



- ListWatch detects changes and adds items to work queue
- Worker goroutines process queue items by calling Reconcile()
- Reconcile() compares current vs desired state and takes actions
- Status updates reflect the new current state

Generic Controller Reconciliation Loop – Step 9



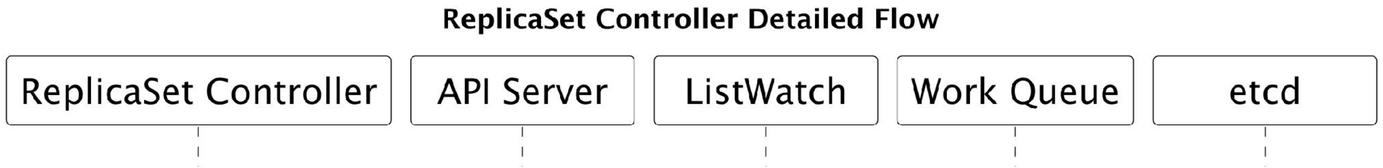
- ListWatch detects changes and adds items to work queue
- Worker goroutines process queue items by calling Reconcile()
- Reconcile() compares current vs desired state and takes actions
- Status updates reflect the new current state

ReplicaSet Controller Example

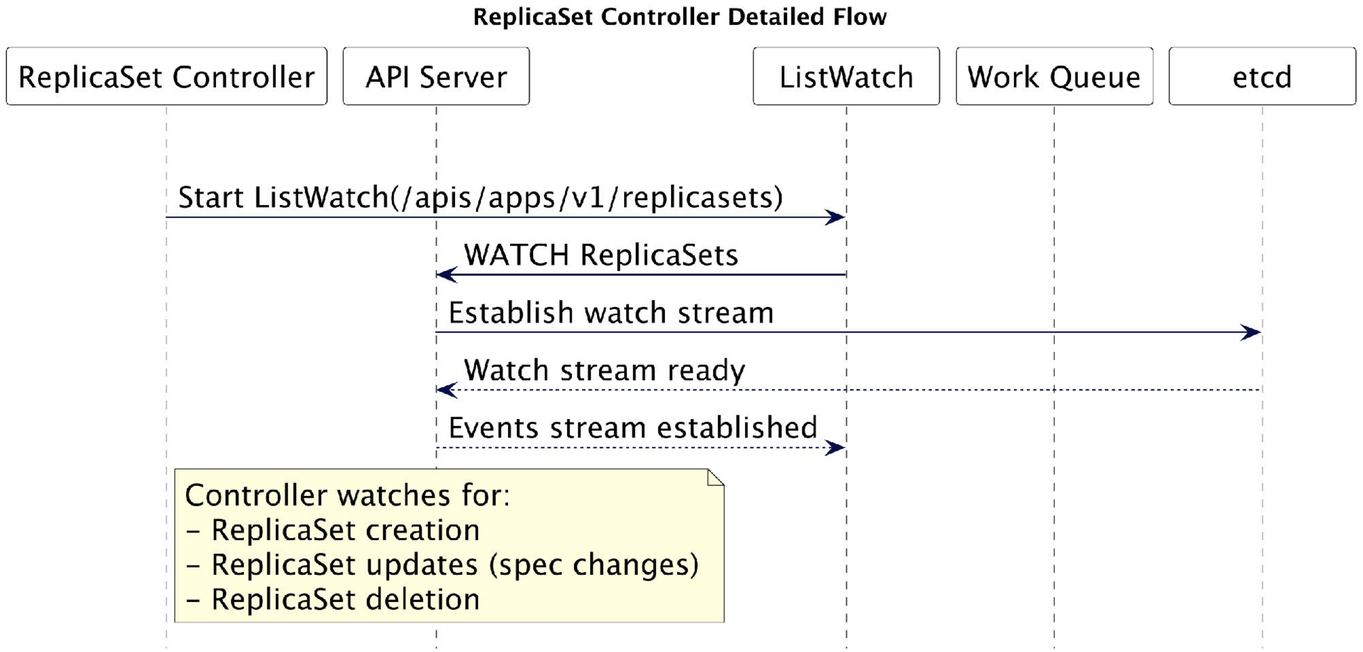
- Watches ReplicaSet objects for changes in replica count
- Lists all pods and filters those owned by ReplicaSet
- Creates pods when current count < desired count
- Deletes pods when current count > desired count
- Updates ReplicaSet status with actual replica counts

ReplicaSet Controller Detailed Flow – Step 1

- Controller watches ReplicaSet changes via ListWatch
- For each ReplicaSet, lists pods and filters by ownership
- Creates missing pods or deletes excess pods as needed
- Updates ReplicaSet status with current replica counts

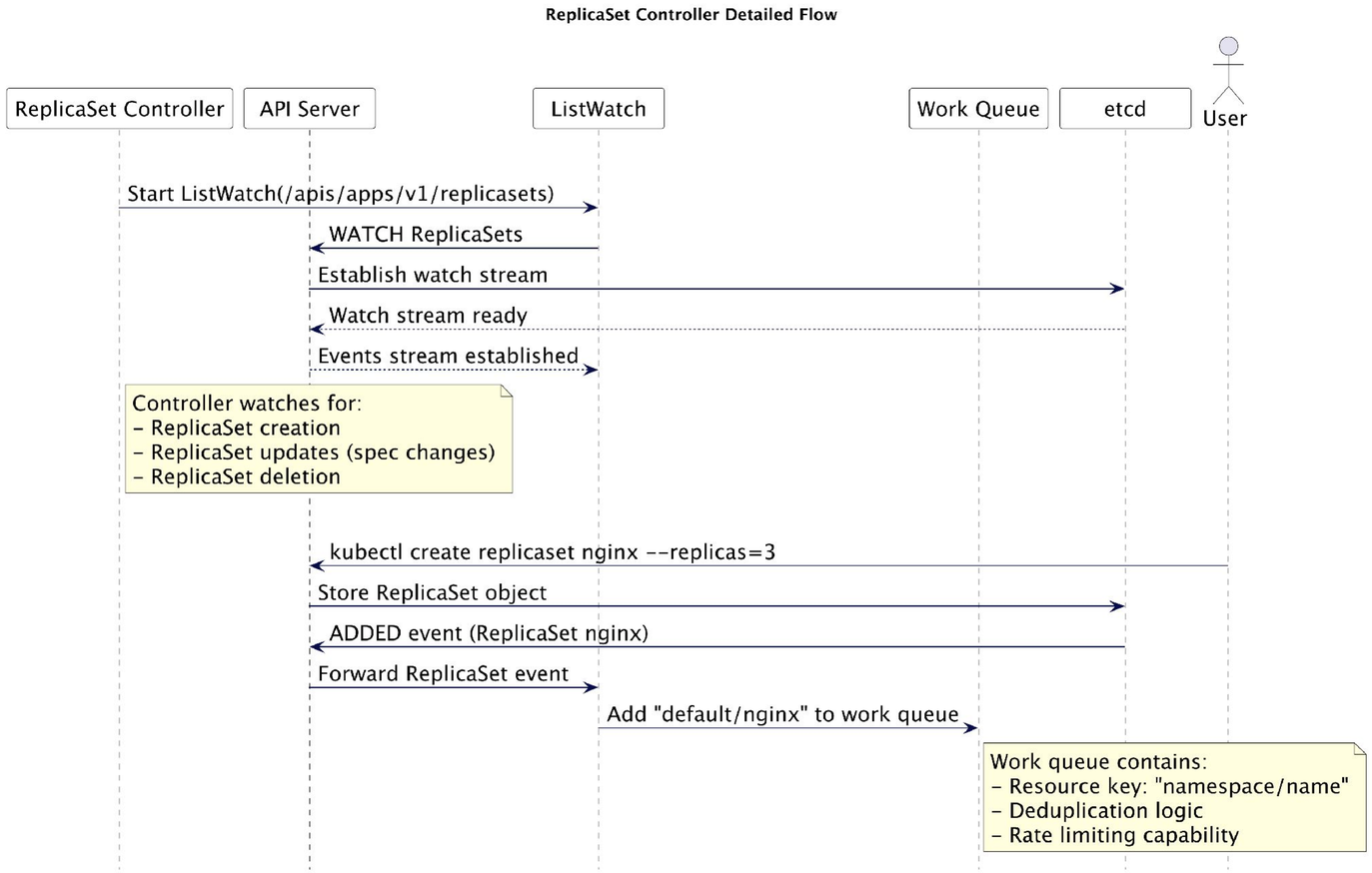


ReplicaSet Controller Detailed Flow – Step 2



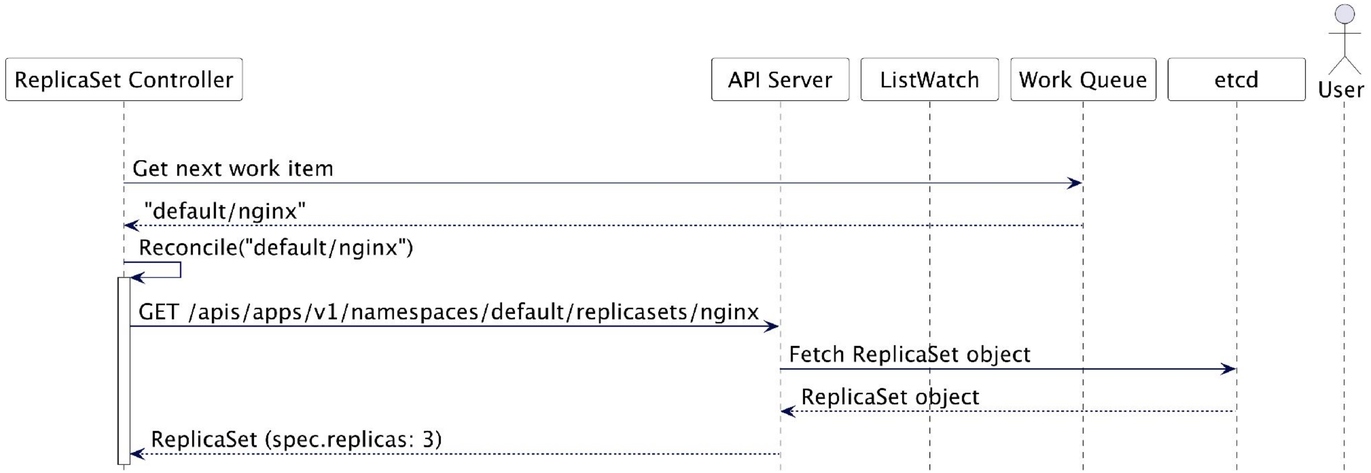
- Controller watches ReplicaSet changes via ListWatch
- For each ReplicaSet, lists pods and filters by ownership
- Creates missing pods or deletes excess pods as needed
- Updates ReplicaSet status with current replica counts

ReplicaSet Controller Detailed Flow - Step 3



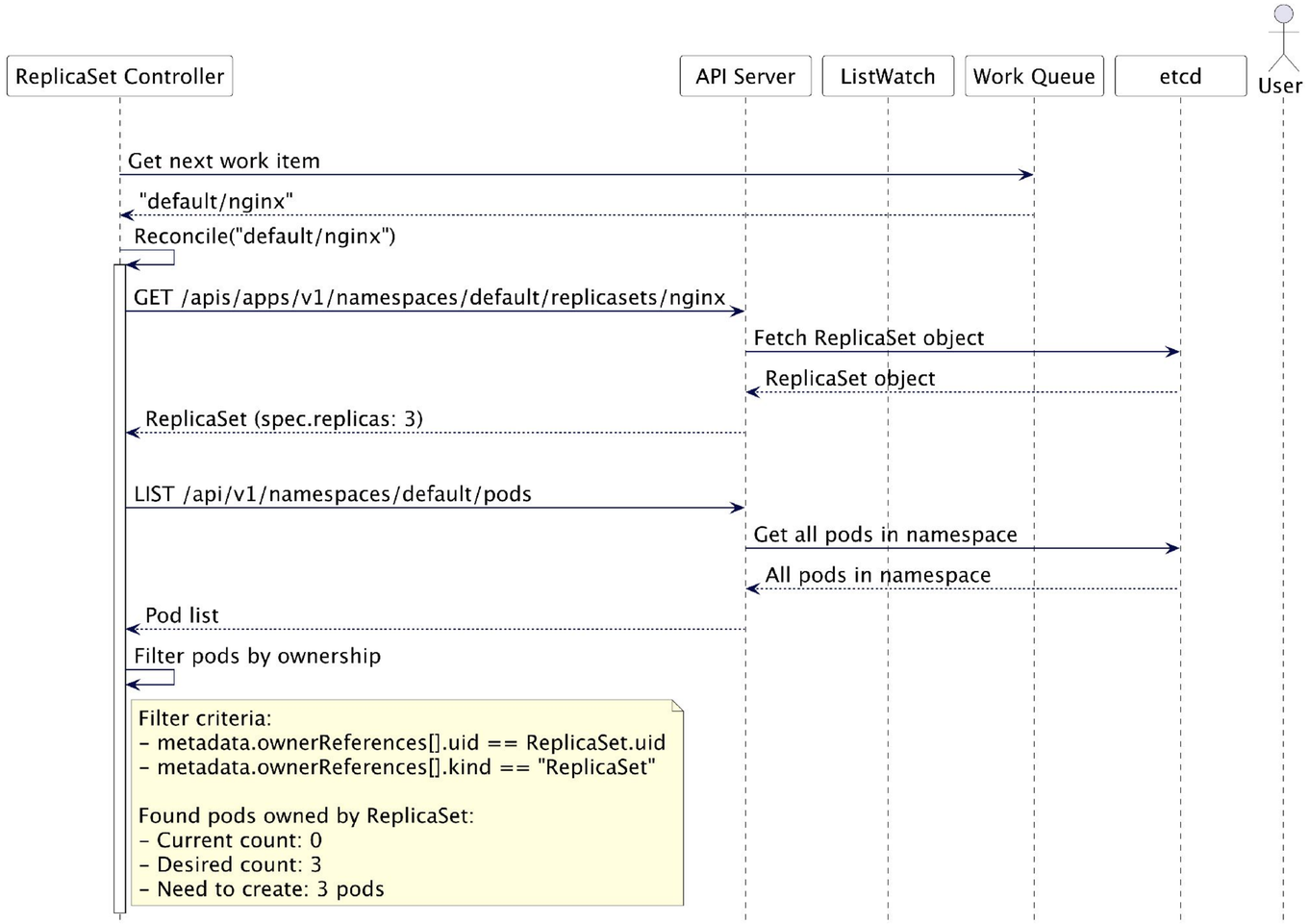
- Controller watches ReplicaSet changes via ListWatch
- For each ReplicaSet, lists pods and filters by ownership
- Creates missing pods or deletes excess pods as needed
- Updates ReplicaSet status with current replica counts

ReplicaSet Controller Detailed Flow – Step 4



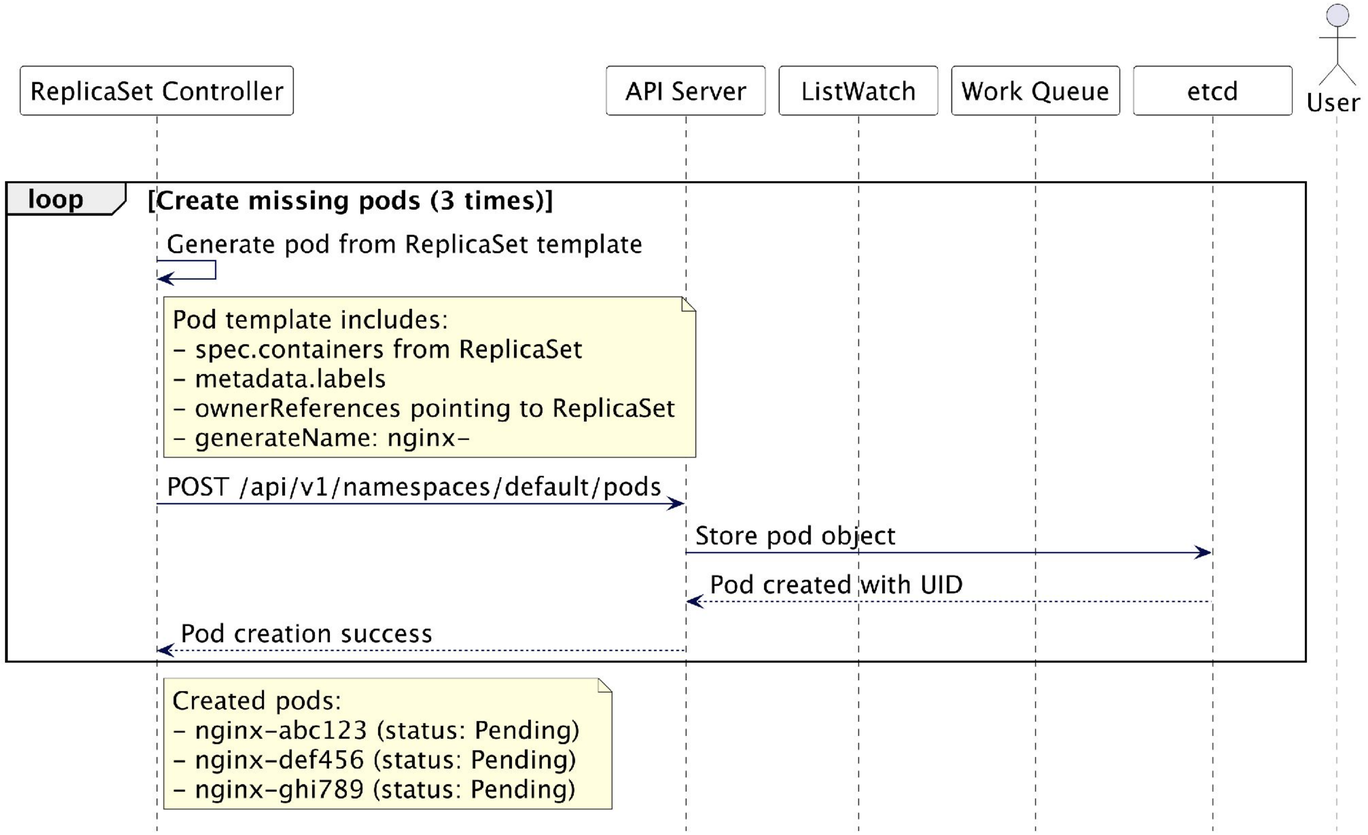
- Controller watches ReplicaSet changes via ListWatch
- For each ReplicaSet, lists pods and filters by ownership
- Creates missing pods or deletes excess pods as needed
- Updates ReplicaSet status with current replica counts

ReplicaSet Controller Detailed Flow - Step 5



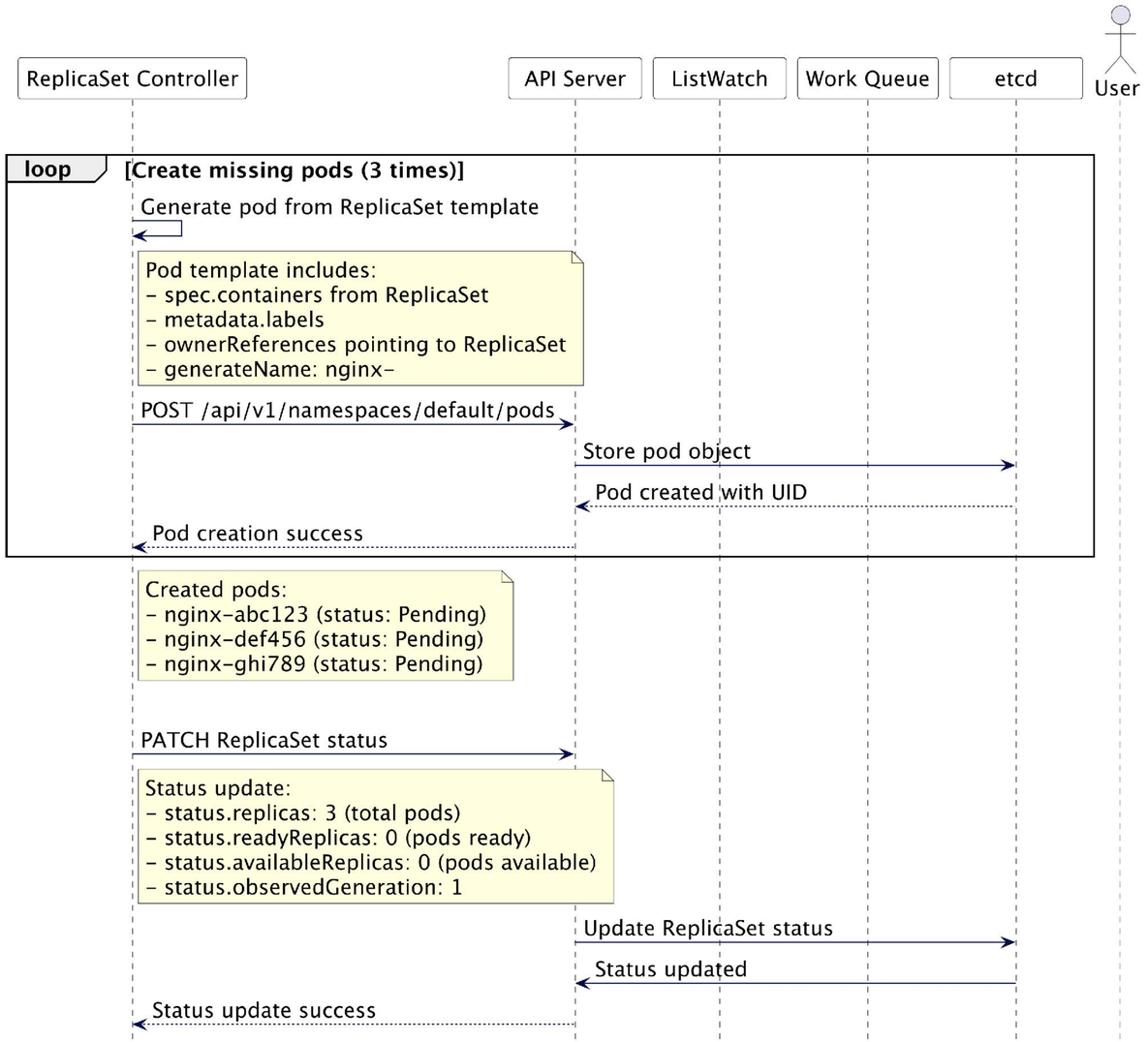
- Controller watches ReplicaSet changes via ListWatch
- For each ReplicaSet, lists pods and filters by ownership
- Creates missing pods or deletes excess pods as needed
- Updates ReplicaSet status with current replica counts

ReplicaSet Controller Detailed Flow – Step 6



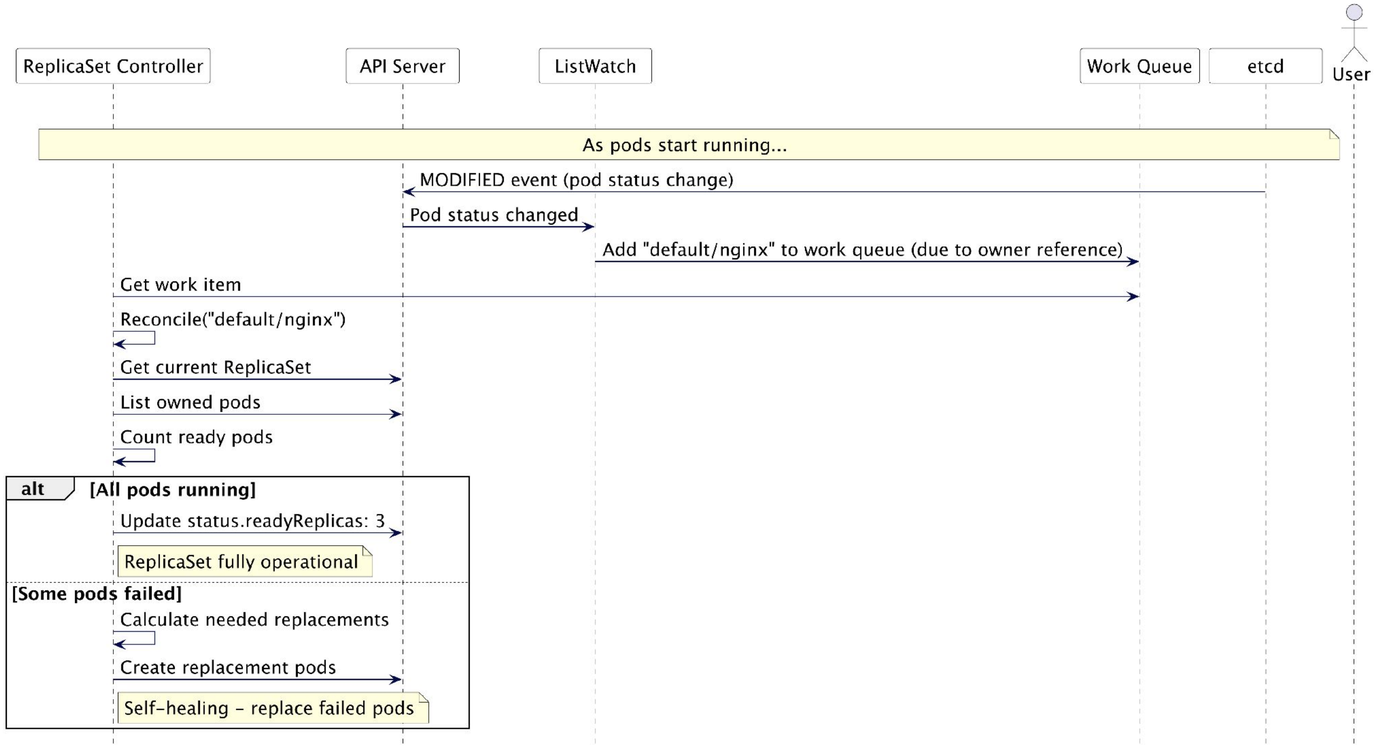
- Controller watches ReplicaSet changes via ListWatch
- For each ReplicaSet, lists pods and filters by ownership
- Creates missing pods or deletes excess pods as needed
- Updates ReplicaSet status with current replica counts

ReplicaSet Controller Detailed Flow – Step 7



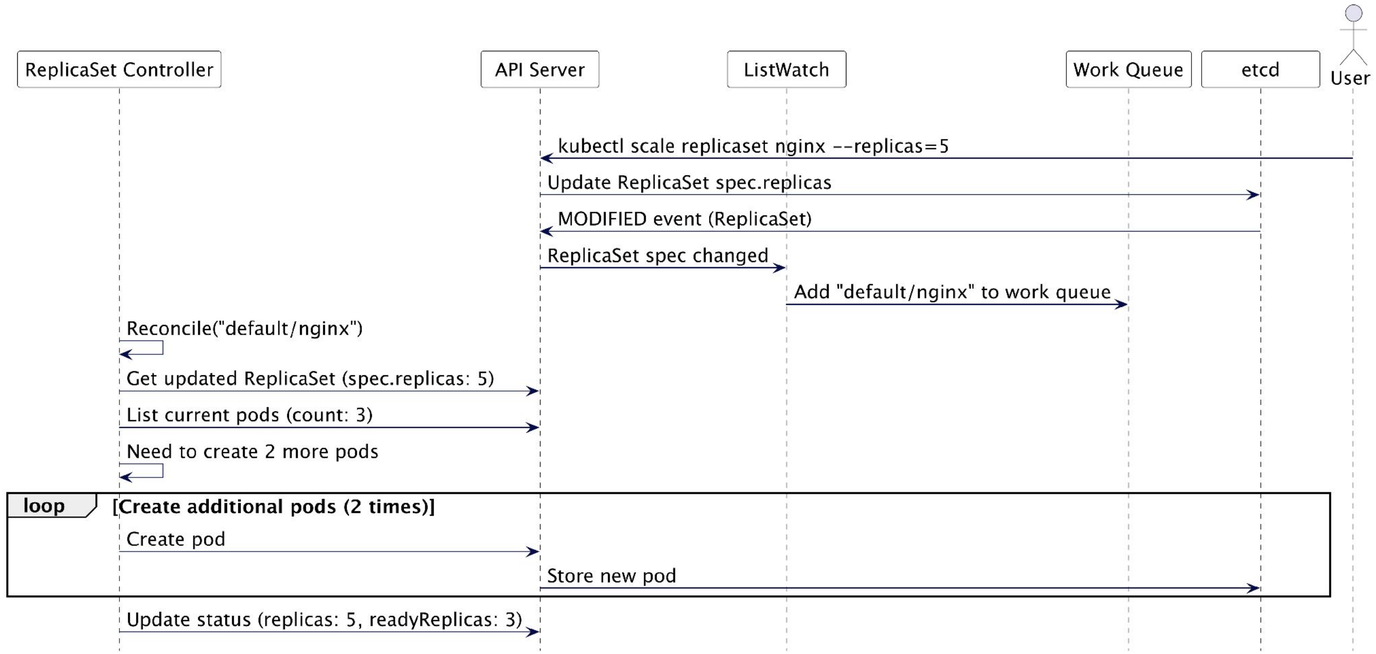
- Controller watches ReplicaSet changes via ListWatch
- For each ReplicaSet, lists pods and filters by ownership
- Creates missing pods or deletes excess pods as needed
- Updates ReplicaSet status with current replica counts

ReplicaSet Controller Detailed Flow – Step 8



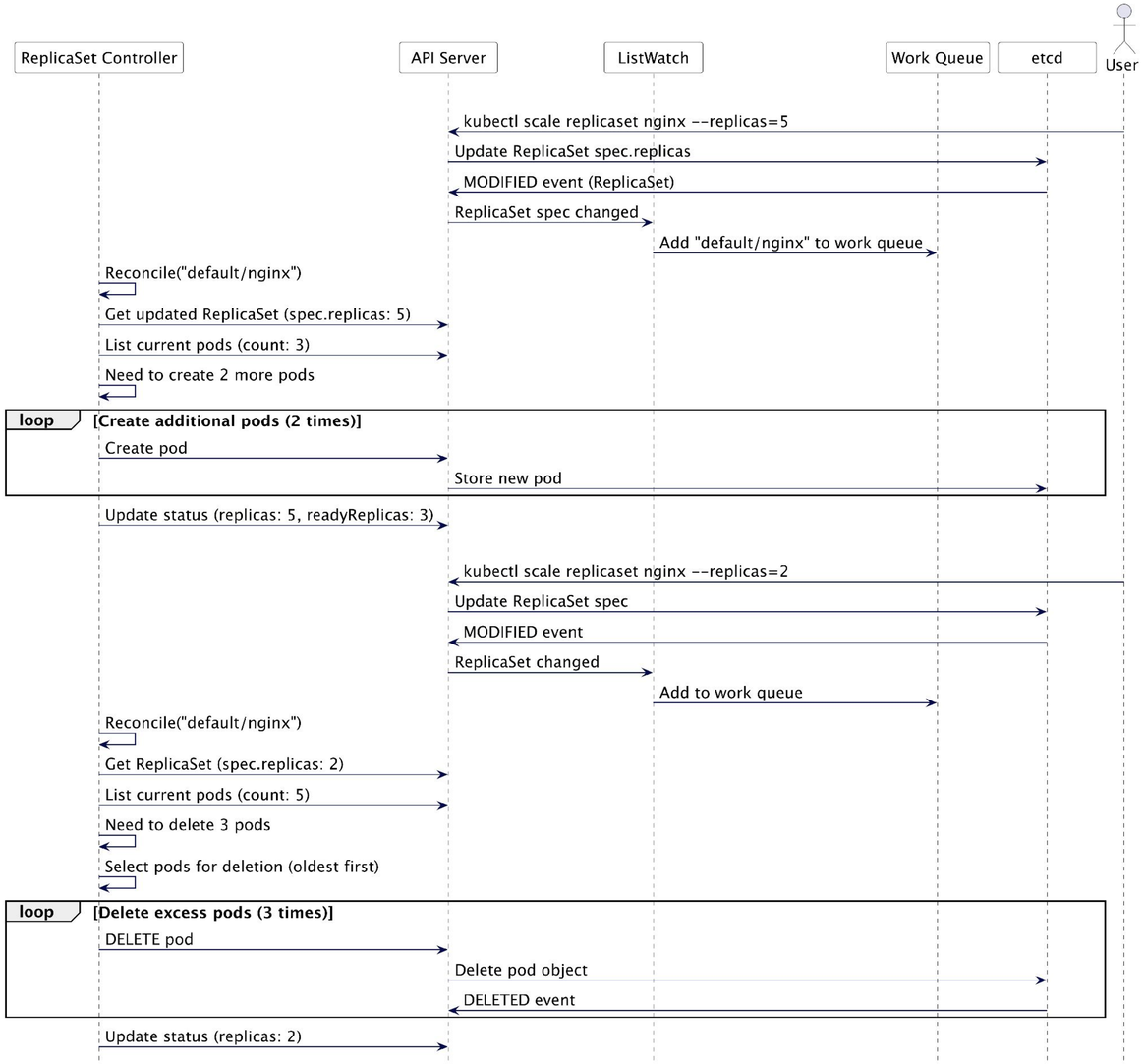
- Controller watches ReplicaSet changes via ListWatch
- For each ReplicaSet, lists pods and filters by ownership
- Creates missing pods or deletes excess pods as needed
- Updates ReplicaSet status with current replica counts

ReplicaSet Controller Detailed Flow – Step 9



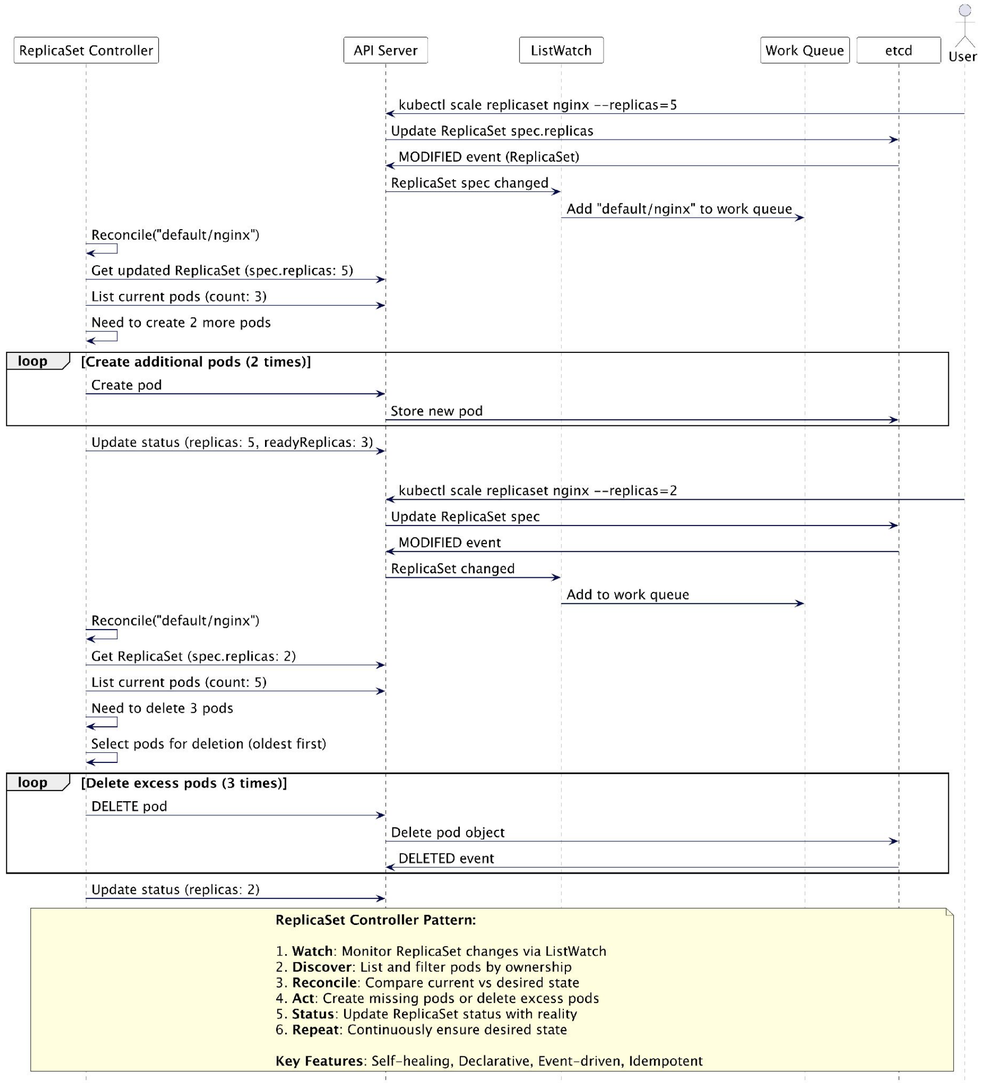
- Controller watches ReplicaSet changes via ListWatch
- For each ReplicaSet, lists pods and filters by ownership
- Creates missing pods or deletes excess pods as needed
- Updates ReplicaSet status with current replica counts

ReplicaSet Controller Detailed Flow - Step 10



- Controller watches ReplicaSet changes via ListWatch
- For each ReplicaSet, lists pods and filters by ownership
- Creates missing pods or deletes excess pods as needed
- Updates ReplicaSet status with current replica counts

ReplicaSet Controller Detailed Flow - Step 11

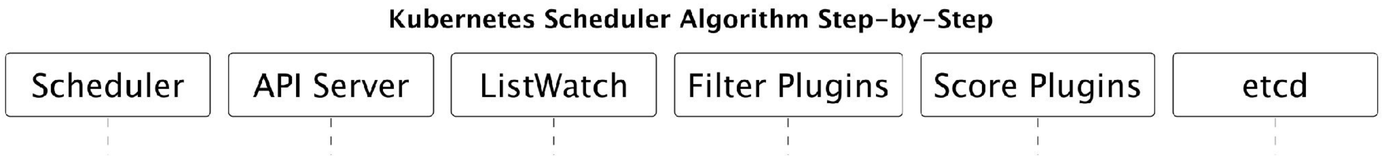


- Controller watches ReplicaSet changes via ListWatch
- For each ReplicaSet, lists pods and filters by ownership
- Creates missing pods or deletes excess pods as needed
- Updates ReplicaSet status with current replica counts

Kubernetes Scheduler Overview

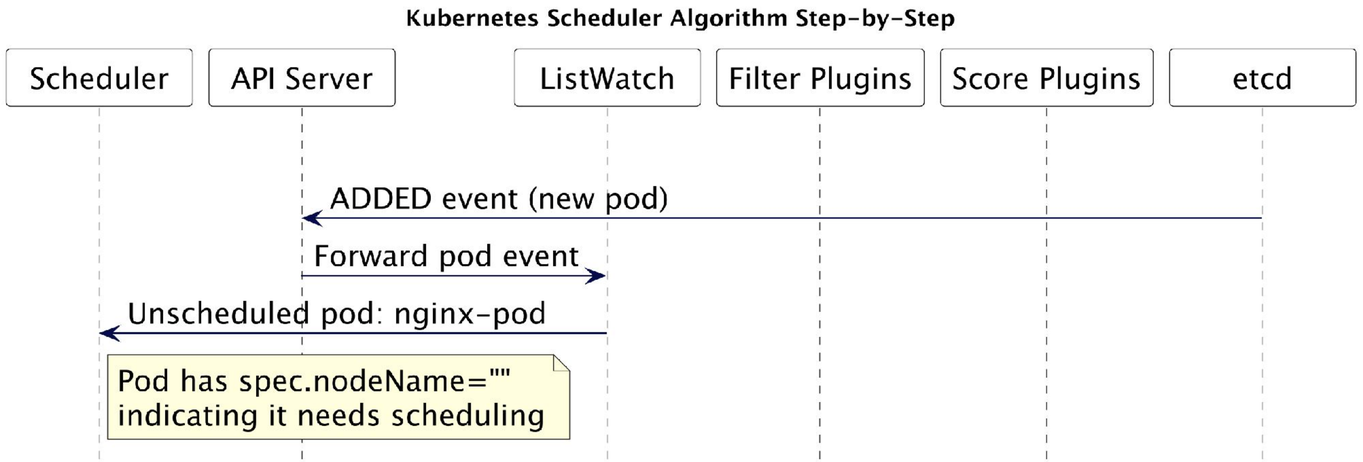
- Responsible for pod placement decisions across the cluster
- Two-phase process: Filtering (predicates) and Scoring (priorities)
- Considers resource requirements, affinity rules, and constraints
- Pluggable architecture allows custom scheduling algorithms
- Watches for unscheduled pods and assigns them to nodes

Scheduling Algorithm Step-by-Step - Step 1



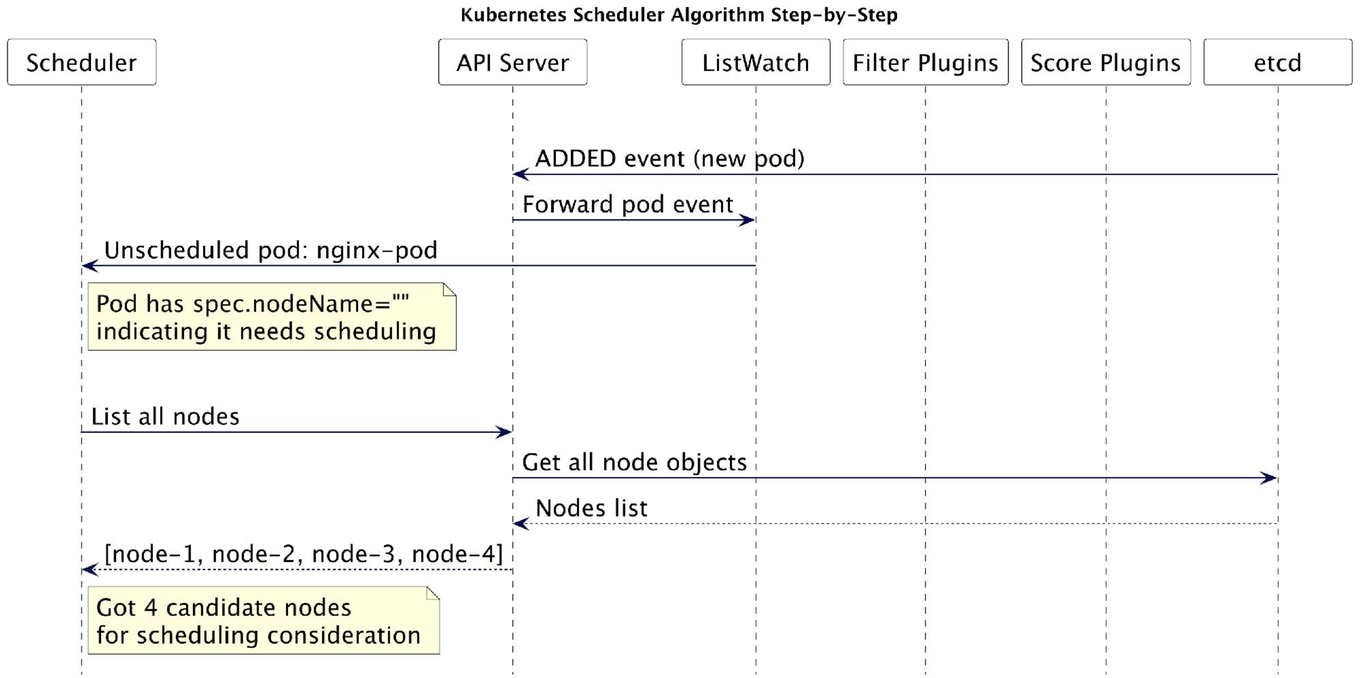
- Scheduler watches for unscheduled pods via ListWatch
- For each pod, gets list of all available nodes
- Filtering phase eliminates nodes that don't meet requirements
- Scoring phase ranks remaining nodes and selects the best
- Binding operation assigns pod to chosen node

Scheduling Algorithm Step-by-Step - Step 2



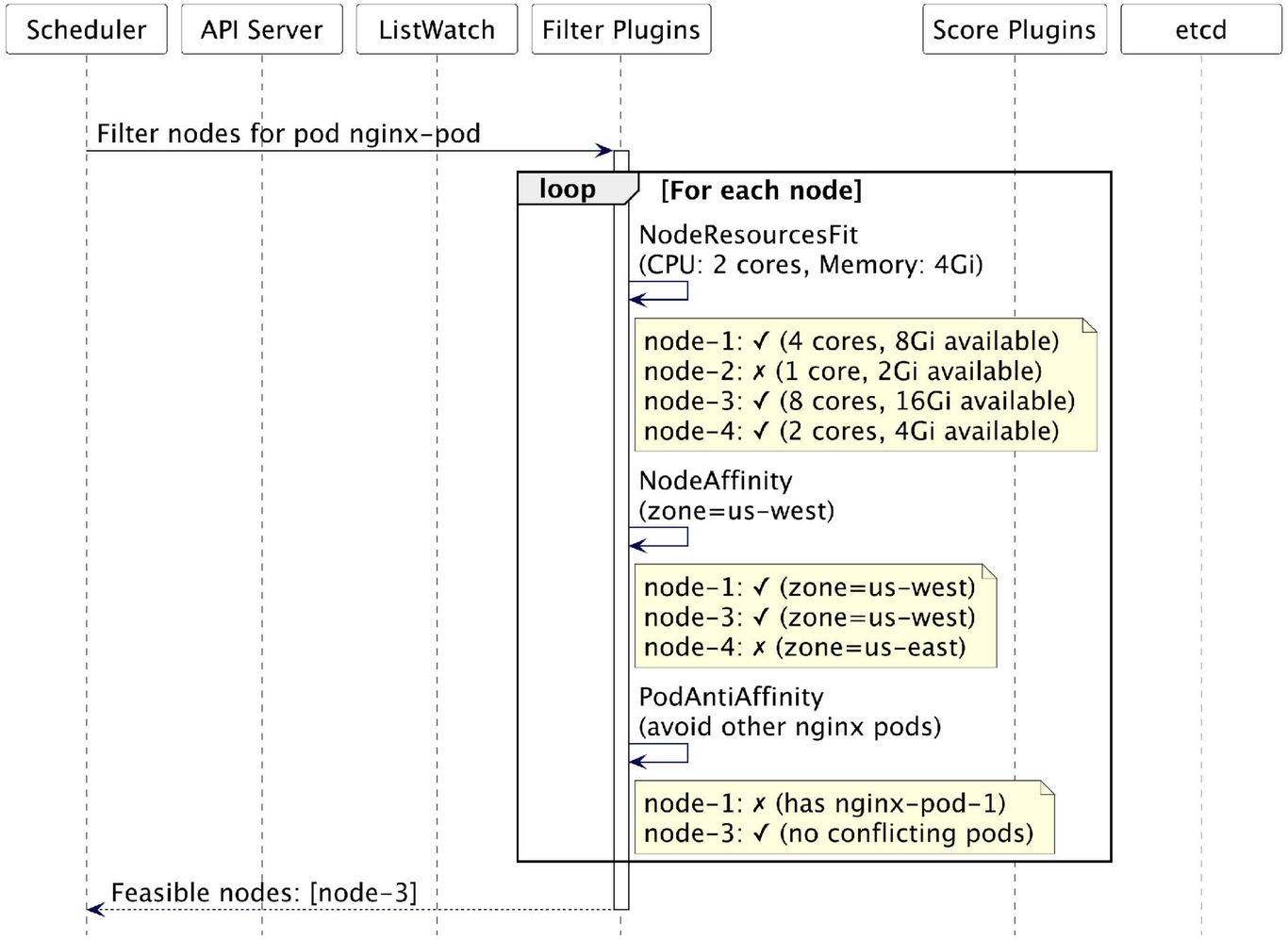
- Scheduler watches for unscheduled pods via ListWatch
- For each pod, gets list of all available nodes
- Filtering phase eliminates nodes that don't meet requirements
- Scoring phase ranks remaining nodes and selects the best
- Binding operation assigns pod to chosen node

Scheduling Algorithm Step-by-Step - Step 3



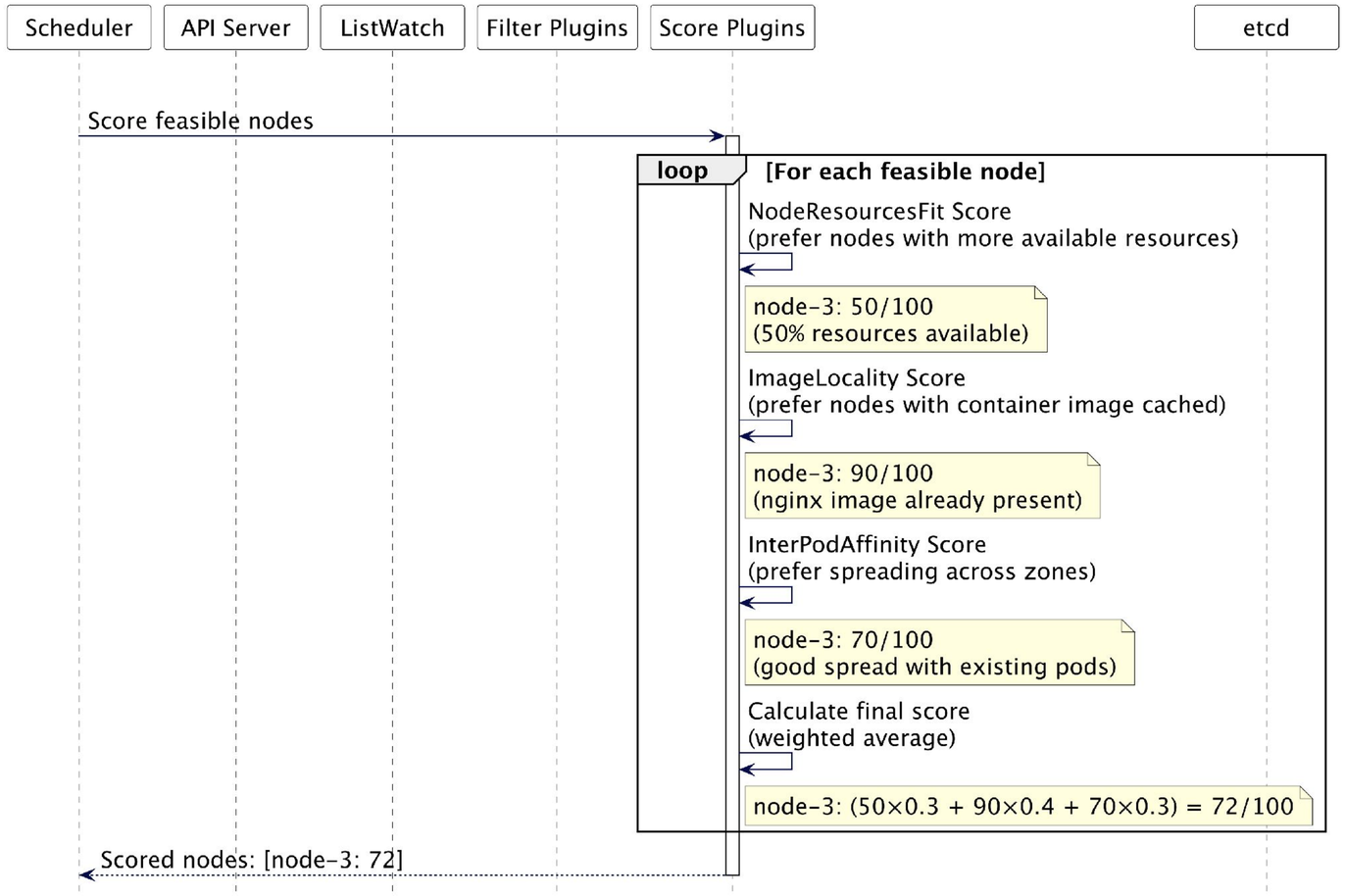
- Scheduler watches for unscheduled pods via ListWatch
- For each pod, gets list of all available nodes
- Filtering phase eliminates nodes that don't meet requirements
- Scoring phase ranks remaining nodes and selects the best
- Binding operation assigns pod to chosen node

Scheduling Algorithm Step-by-Step - Step 4



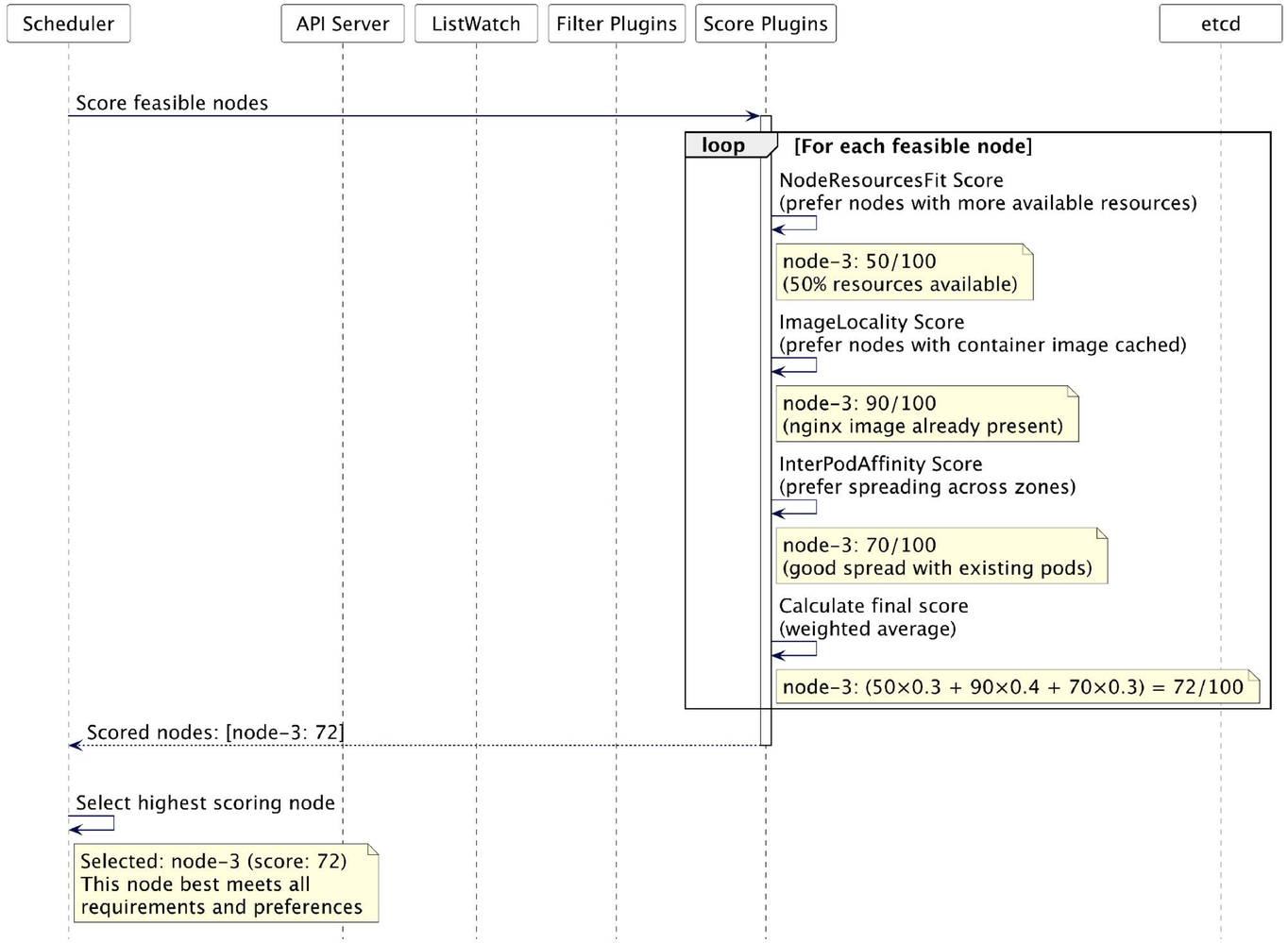
- Scheduler watches for unscheduled pods via ListWatch
- For each pod, gets list of all available nodes
- Filtering phase eliminates nodes that don't meet requirements
- Scoring phase ranks remaining nodes and selects the best
- Binding operation assigns pod to chosen node

Scheduling Algorithm Step-by-Step - Step 5



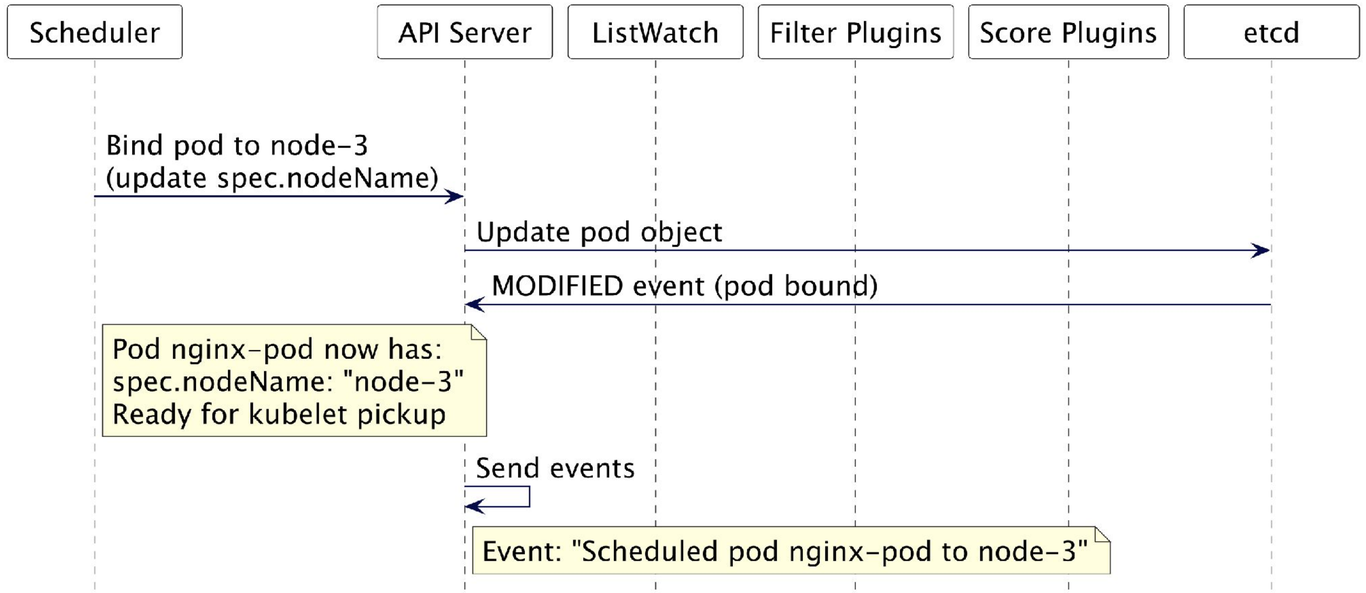
- Scheduler watches for unscheduled pods via ListWatch
- For each pod, gets list of all available nodes
- Filtering phase eliminates nodes that don't meet requirements
- Scoring phase ranks remaining nodes and selects the best
- Binding operation assigns pod to chosen node

Scheduling Algorithm Step-by-Step - Step 6



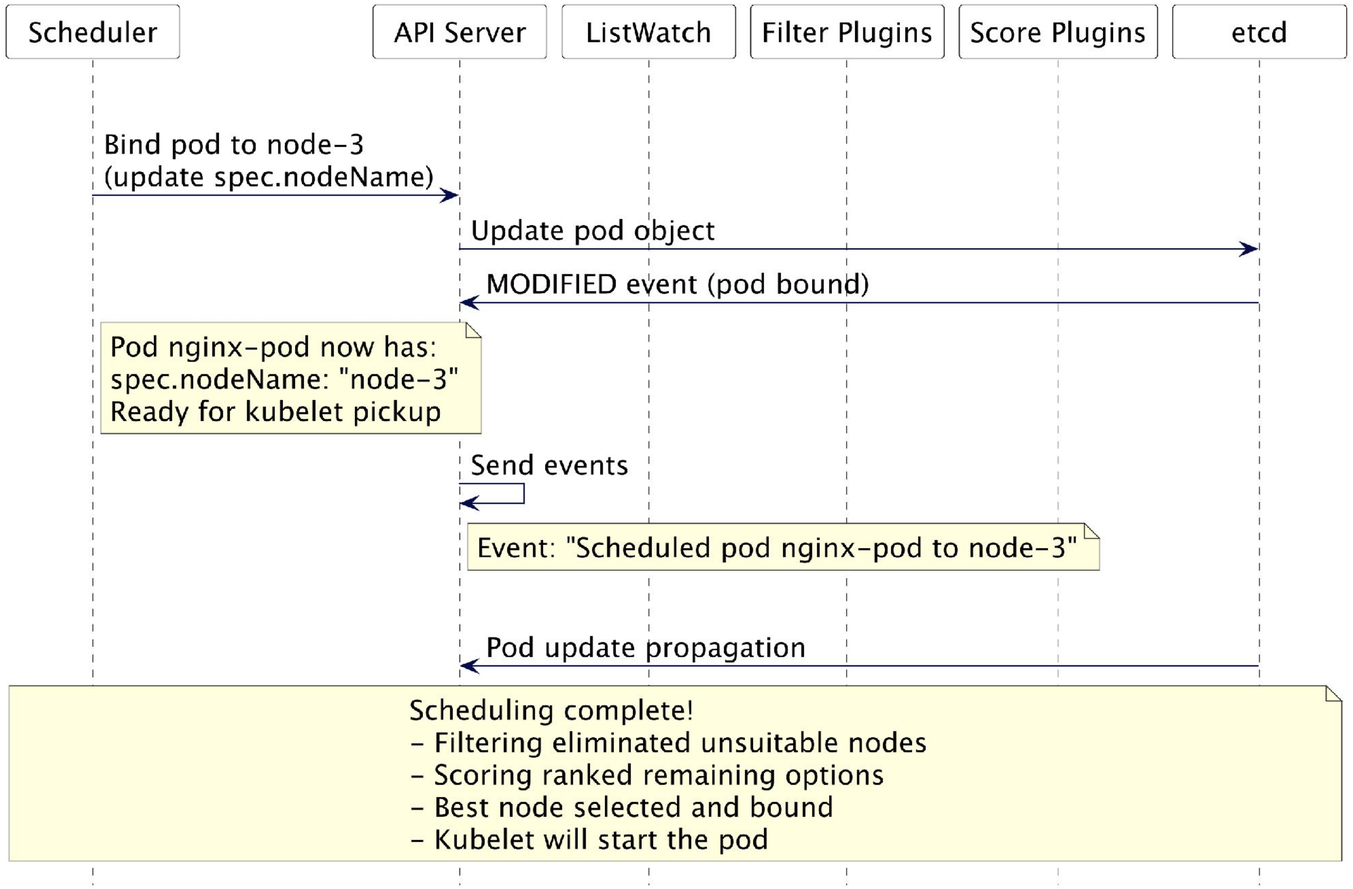
- Scheduler watches for unscheduled pods via ListWatch
- For each pod, gets list of all available nodes
- Filtering phase eliminates nodes that don't meet requirements
- Scoring phase ranks remaining nodes and selects the best
- Binding operation assigns pod to chosen node

Scheduling Algorithm Step-by-Step - Step 7



- Scheduler watches for unscheduled pods via ListWatch
- For each pod, gets list of all available nodes
- Filtering phase eliminates nodes that don't meet requirements
- Scoring phase ranks remaining nodes and selects the best
- Binding operation assigns pod to chosen node

Scheduling Algorithm Step-by-Step - Step 8



- Scheduler watches for unscheduled pods via ListWatch
- For each pod, gets list of all available nodes
- Filtering phase eliminates nodes that don't meet requirements
- Scoring phase ranks remaining nodes and selects the best
- Binding operation assigns pod to chosen node

Advanced Scheduling Features

- Node Affinity: Specify preferred or required node characteristics
- Pod Affinity/Anti-affinity: Co-locate or separate pods
- Taints and Tolerations: Node-level pod admission control
- Resource Requests/Limits: Ensure adequate resources
- Custom Schedulers: Implement domain-specific logic

Module 3 · Partitioning Work & Data

Partitioning Data vs. Scheduling Workloads

- Kubernetes Schedulers place dynamic, interchangeable Pods on Nodes based on resource capacity and constraints
- Distributed Data Systems (Kafka, Spark, Distributed DBs) partition stateful data across nodes via keys or row-groups
- In Kafka and Parquet/Spark, partitions are fundamental units of data storage, ordering, and I/O parallelism

Mapping keys to nodes – the requirements

- Uniform distribution: Data and load should spread evenly across nodes
- Deterministic routing: Any client must know exactly which node to query for a key
- High throughput and low latency lookup

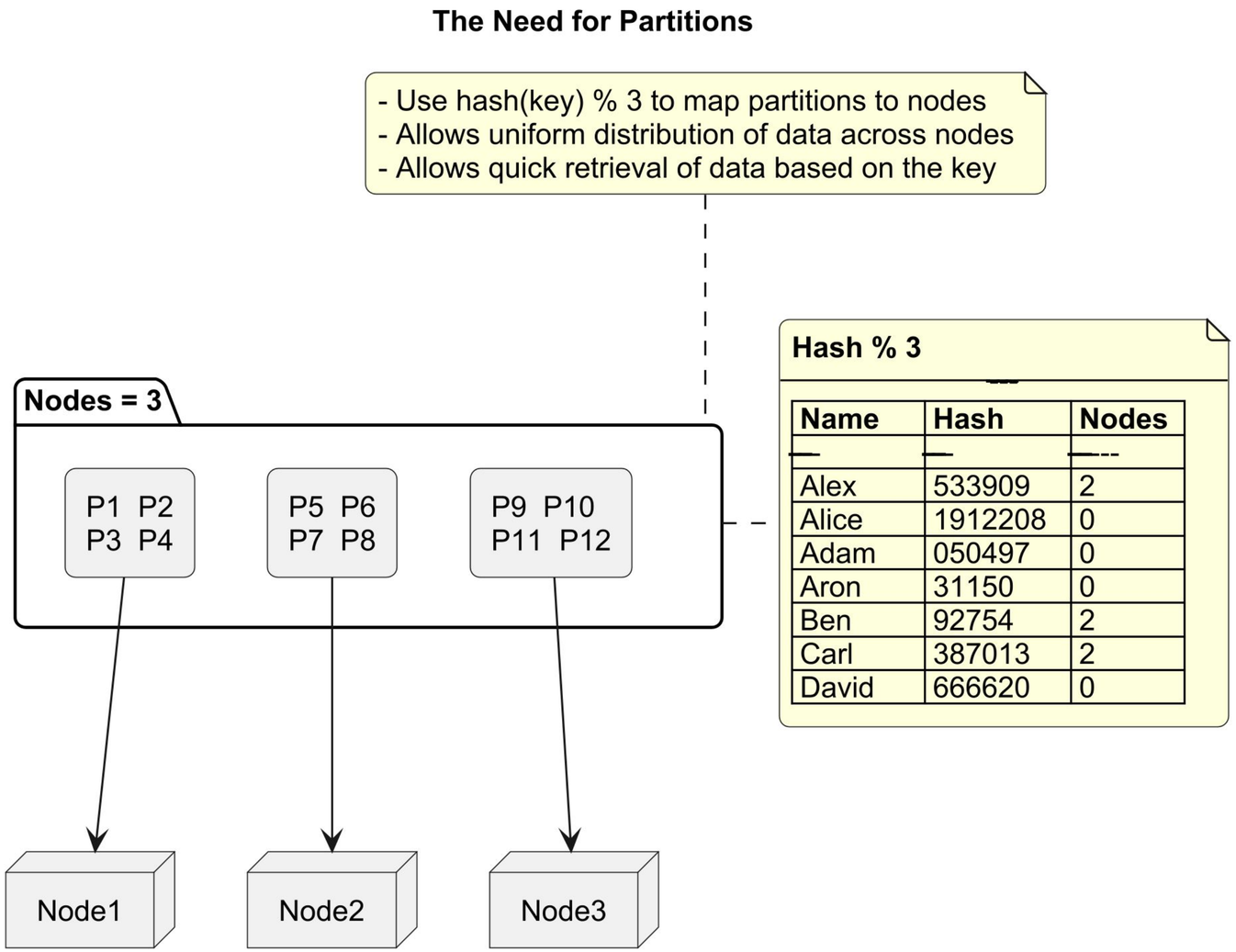
Hashing keys with modulo

- Apply hash function (e.g. MD5 / Murmur3) to key for uniform distribution
- Formula: `Node = hash(key) % N`, where N is number of nodes
- Clients compute destination node locally without central coordination

What breaks when a node is added

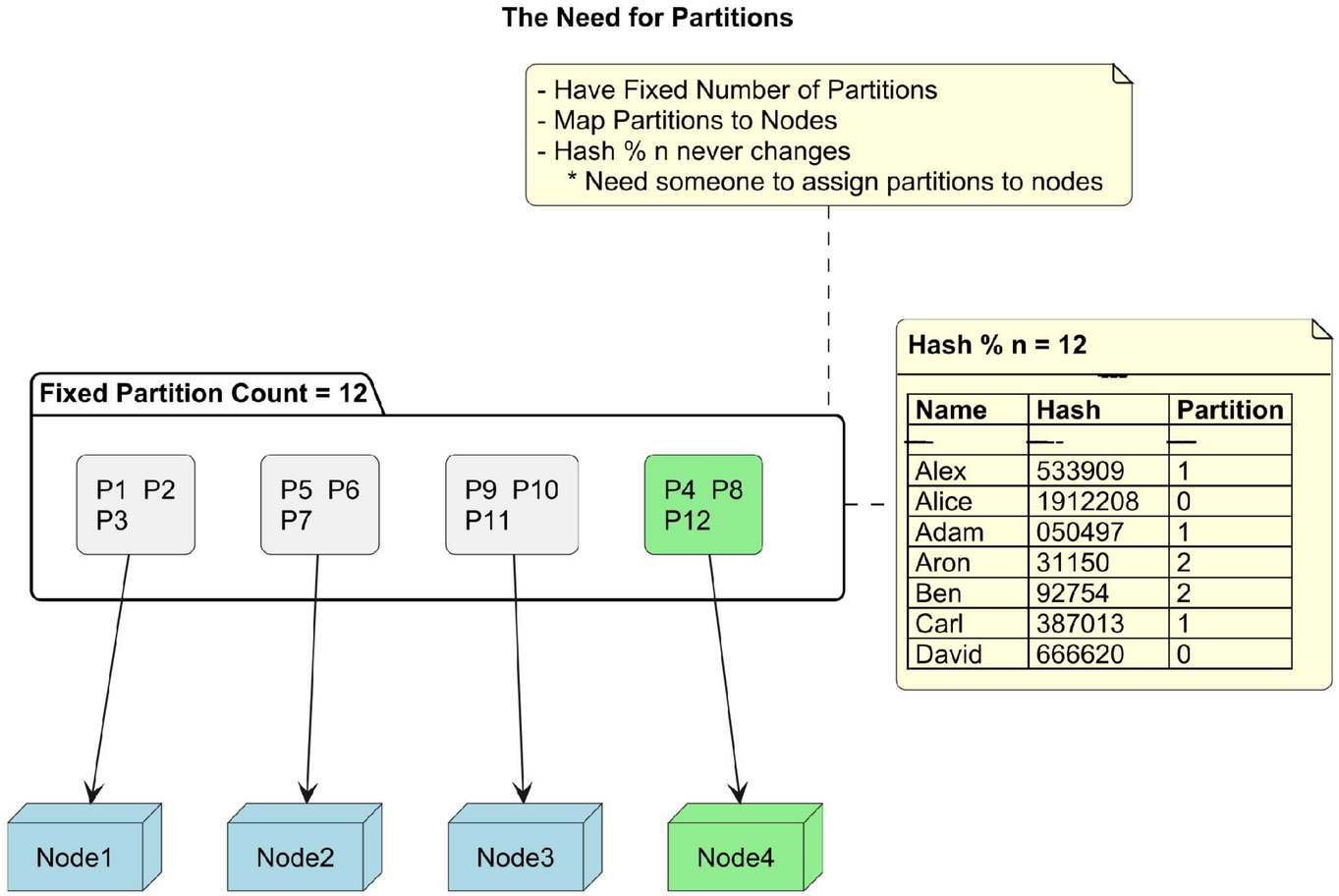
- Formula changes from `hash(key) % 3` to `hash(key) % 4`
- Almost every key hashes to a completely different node!
- Requires massive data shuffling across the entire network

The need for ‘partitions’



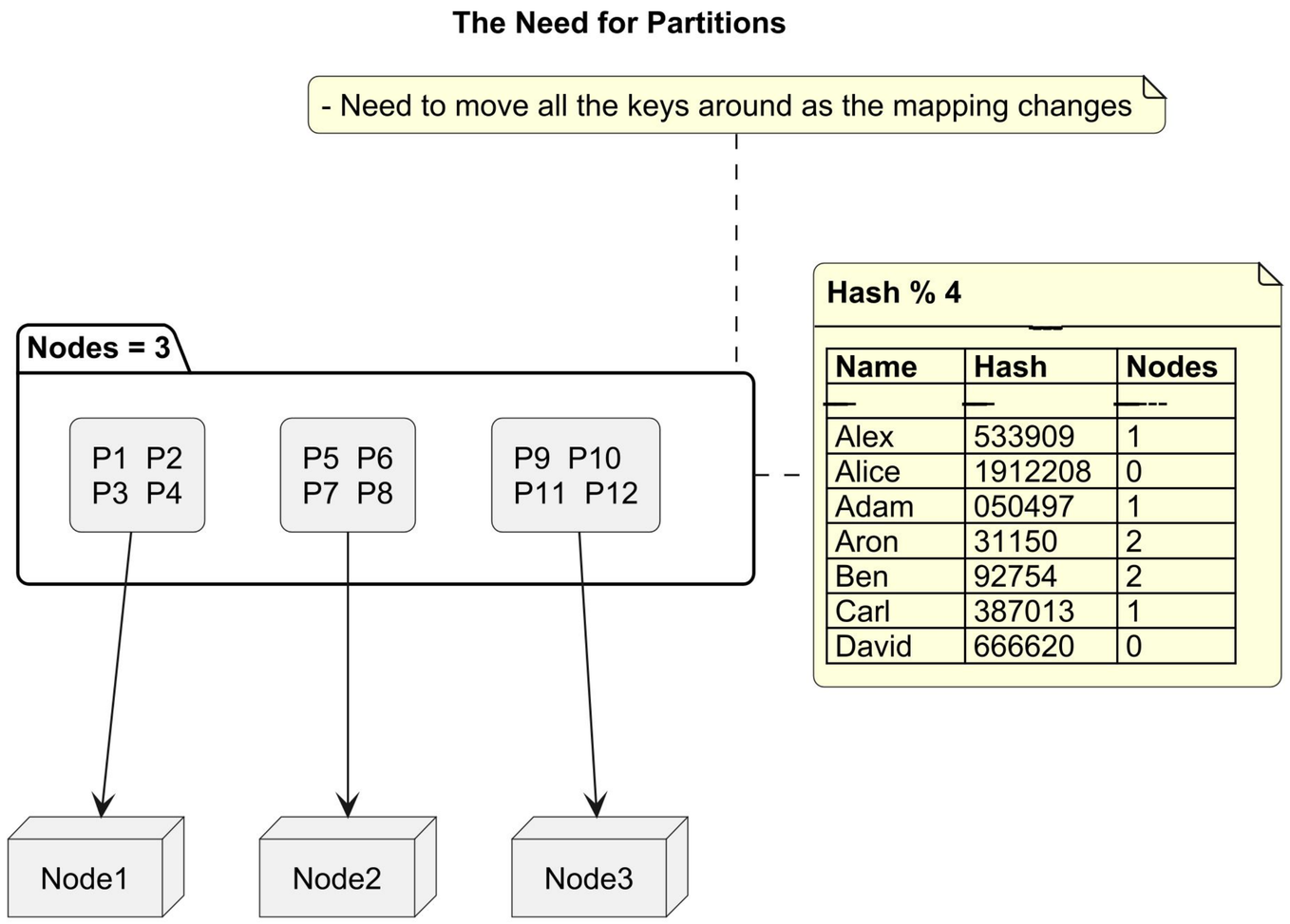
- Introduce indirection: Keys map to Partitions, Partitions map to Nodes
- `Partition = hash(key) % NumberOfPartitions` never changes when nodes change

Fixing the partition count



- Fix total partition count (e.g., 12 partitions)
- Map 12 partitions across 3 nodes (4 partitions per node)
- A coordinator assigns partitions to nodes

Move partitions, not keys



- When Node 4 joins, move entire partitions (P4, P8, P12) instead of individual keys
- Number of partitions must be larger than number of nodes
- Industry guidelines: 1024 in Ignite, 271 in Hazelcast, 10x nodes in Akka

Implications of ‘Fixed’ partition count

- Partition count is defined at topic creation and is not dynamic
- All producers and consumers use fixed hash % partition count
- Impacts scaling: to use more consumers, you must have more partitions
- Partitions do not rebalance automatically when brokers are added/removed
- Reassignment requires tooling (e.g., Cruise Control or manual scripts)
- Critical for per-key ordering guarantees (same key = same partition)

Deciding Partition Count: Ordering vs Growth

- If ordering of events for a key matters (e.g., per-user or per-session), the key must always hash to the same partition
- E.g. If all transactions for an account should be processed in order, we can have partitions per account
- Plan ahead: Have enough partitions to accommodate future growth
- Too many partitions: More open file handles, memory pressure, metadata overhead
- Too few partitions: Underutilized consumer parallelism

Deciding Partition Count: Throughput Math

- Formula: `# Partitions = Desired Throughput / Single Partition Throughput`
- Example: Message size = 1 KB, Desired = 10,000 msg/sec (10 MB/s)
- Assuming single partition throughput = 2 MB/s: `# Partitions = 10 / 2 = 5`

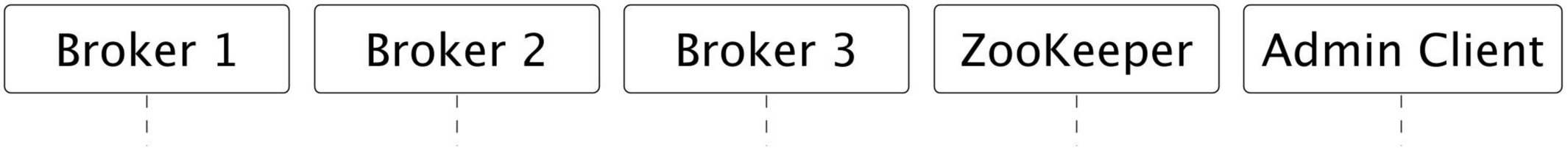
Who assigns partitions to nodes

- Nodes cannot independently guess assignment without risking inconsistency
- Consistent Core elects a single cluster Controller
- Controller computes partition placement and disseminates assignments

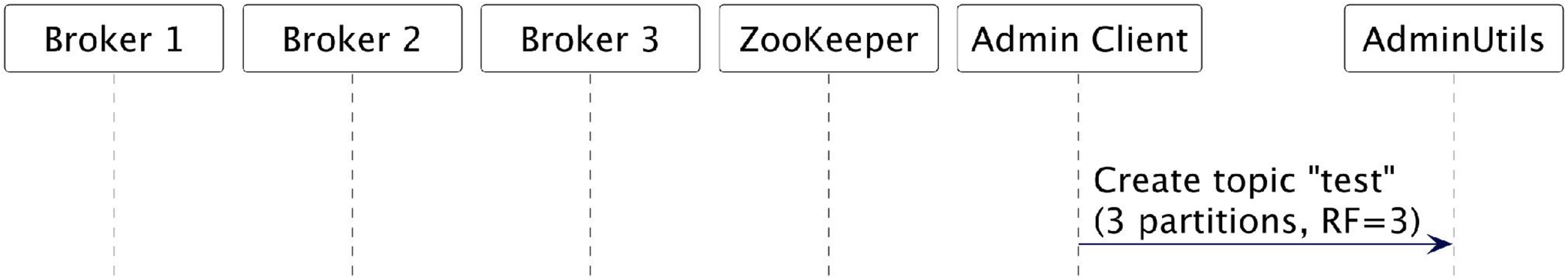
Partitions: Fundamental Storage Unit

- A topic is split into multiple parallel streams called partitions
- Fundamental unit of data storage and parallel processing
- Holds an ordered, append-only log of immutable records

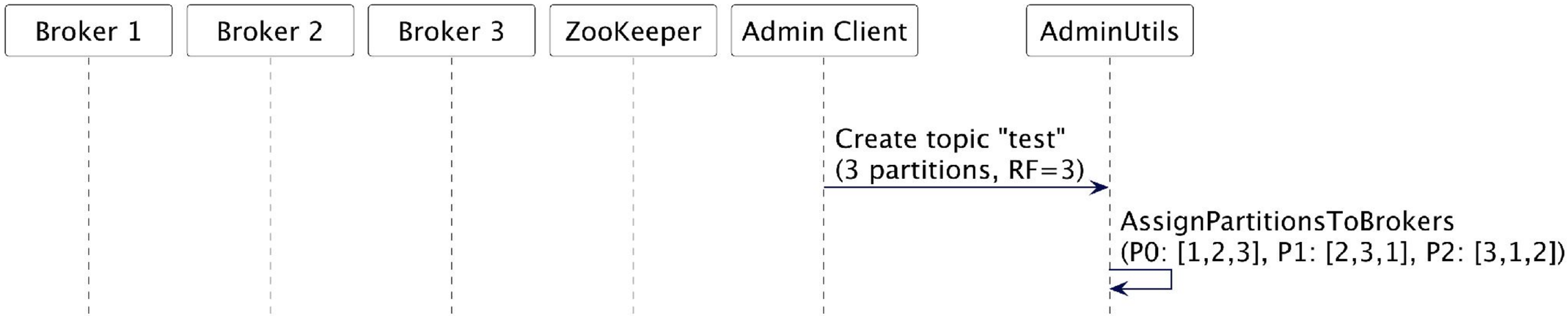
Topic Creation uses fixed number of partitions – Step 1



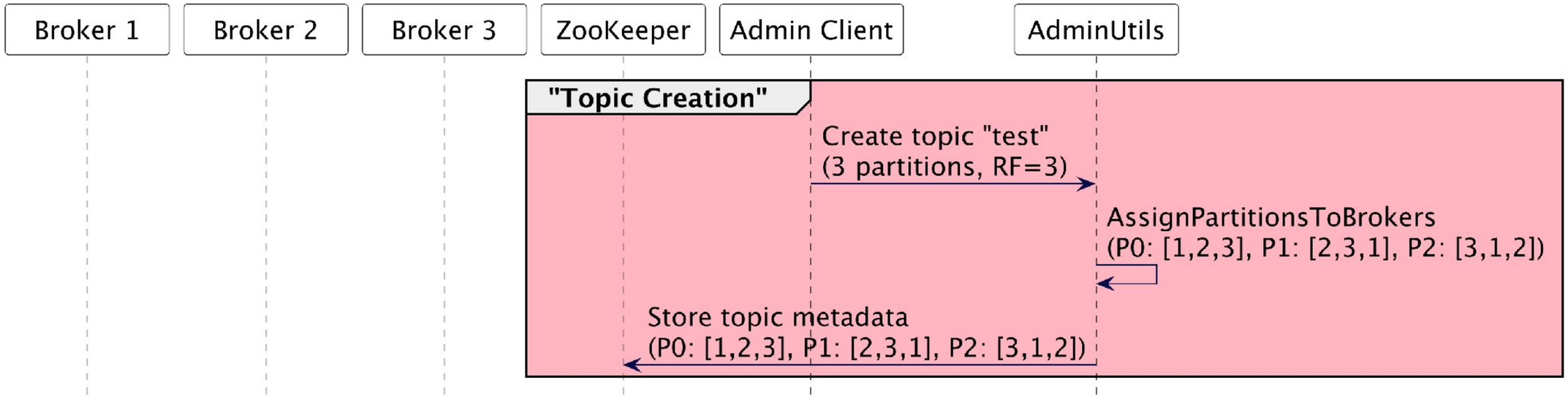
Topic Creation uses fixed number of partitions – Step 2



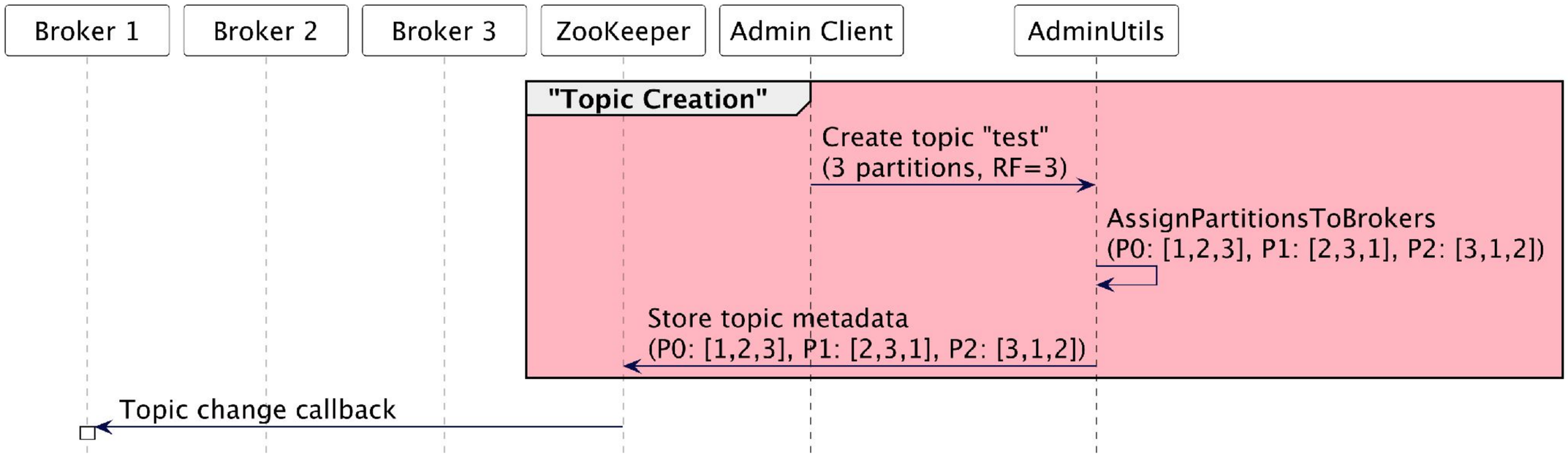
Topic Creation uses fixed number of partitions - Step 3



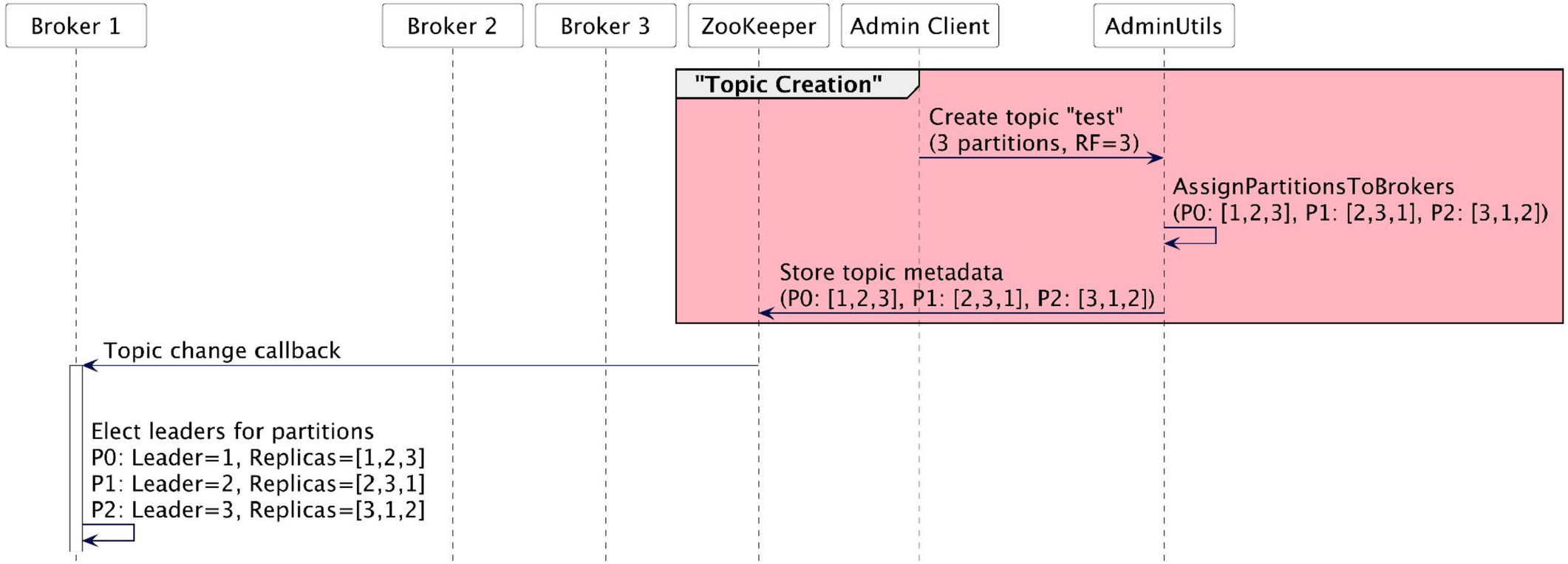
Topic Creation uses fixed number of partitions – Step 4



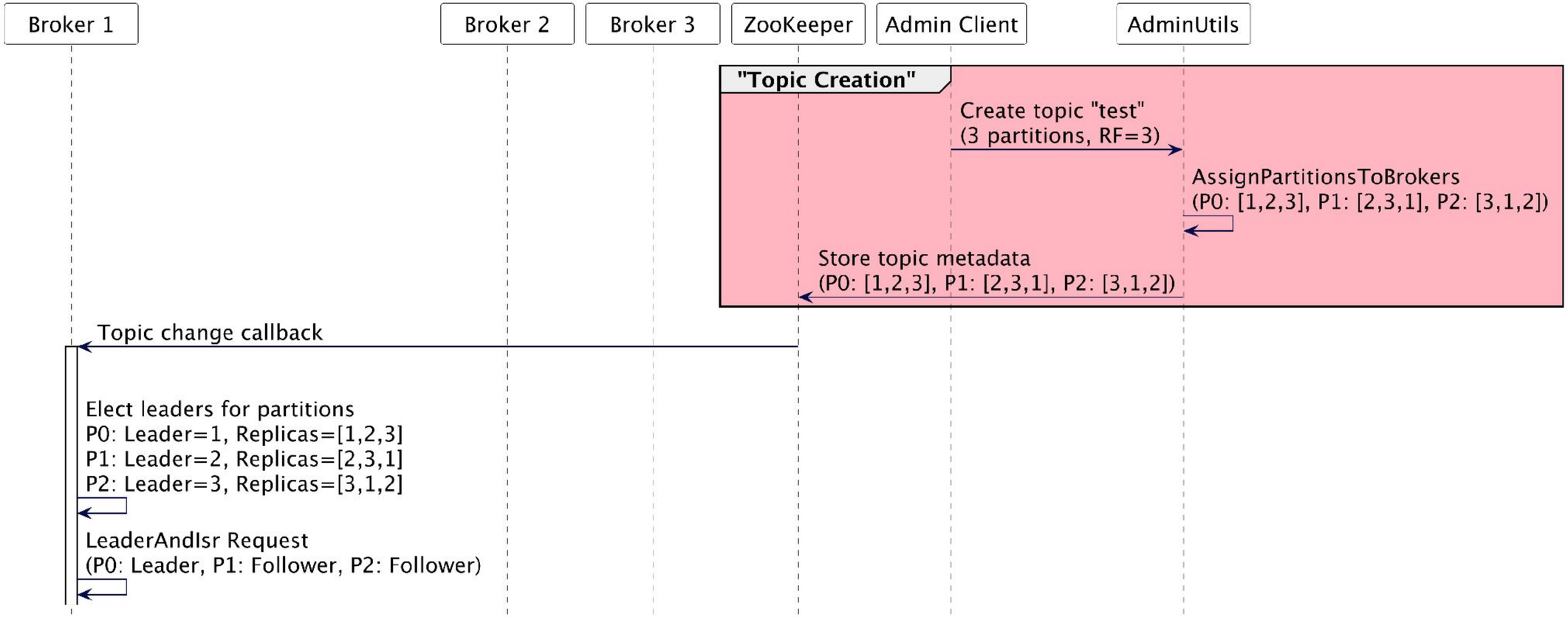
Topic Creation uses fixed number of partitions – Step 5



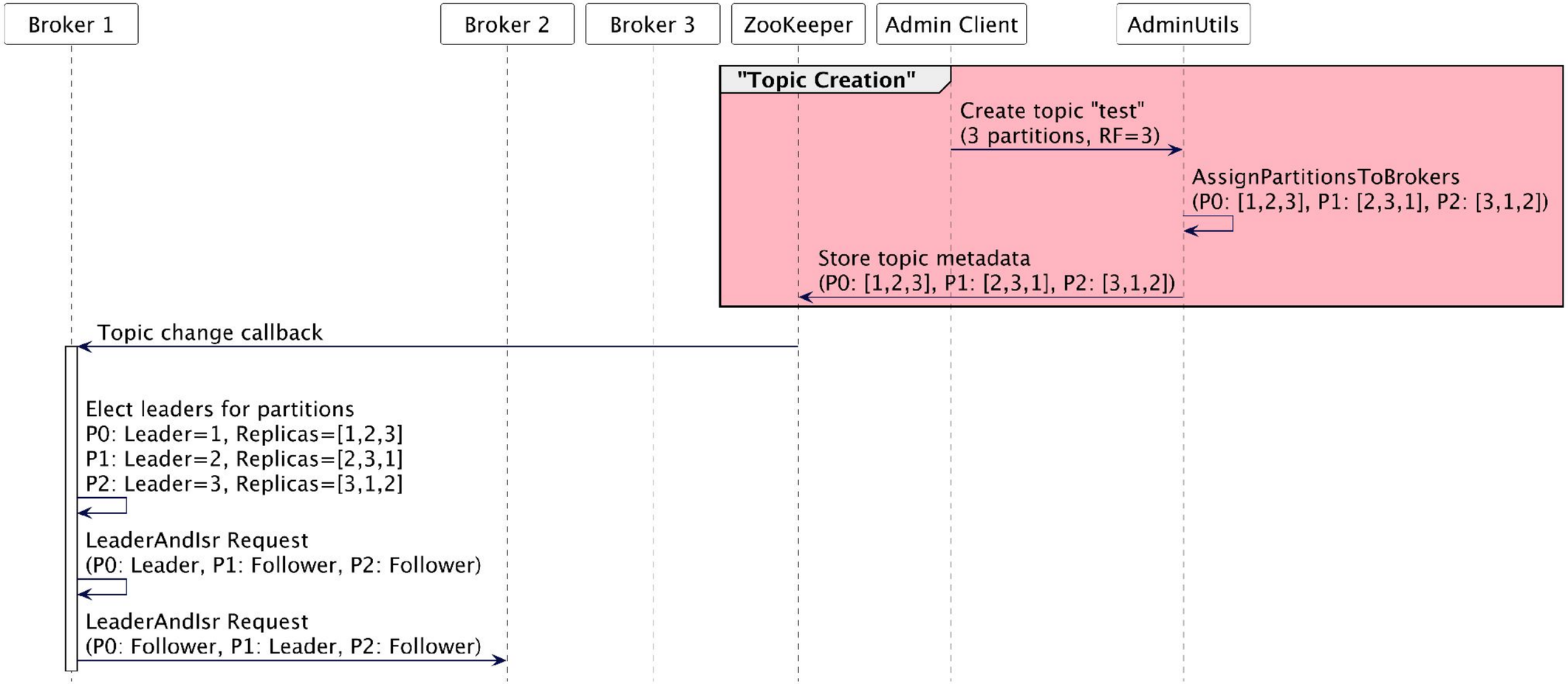
Topic Creation uses fixed number of partitions – Step 6



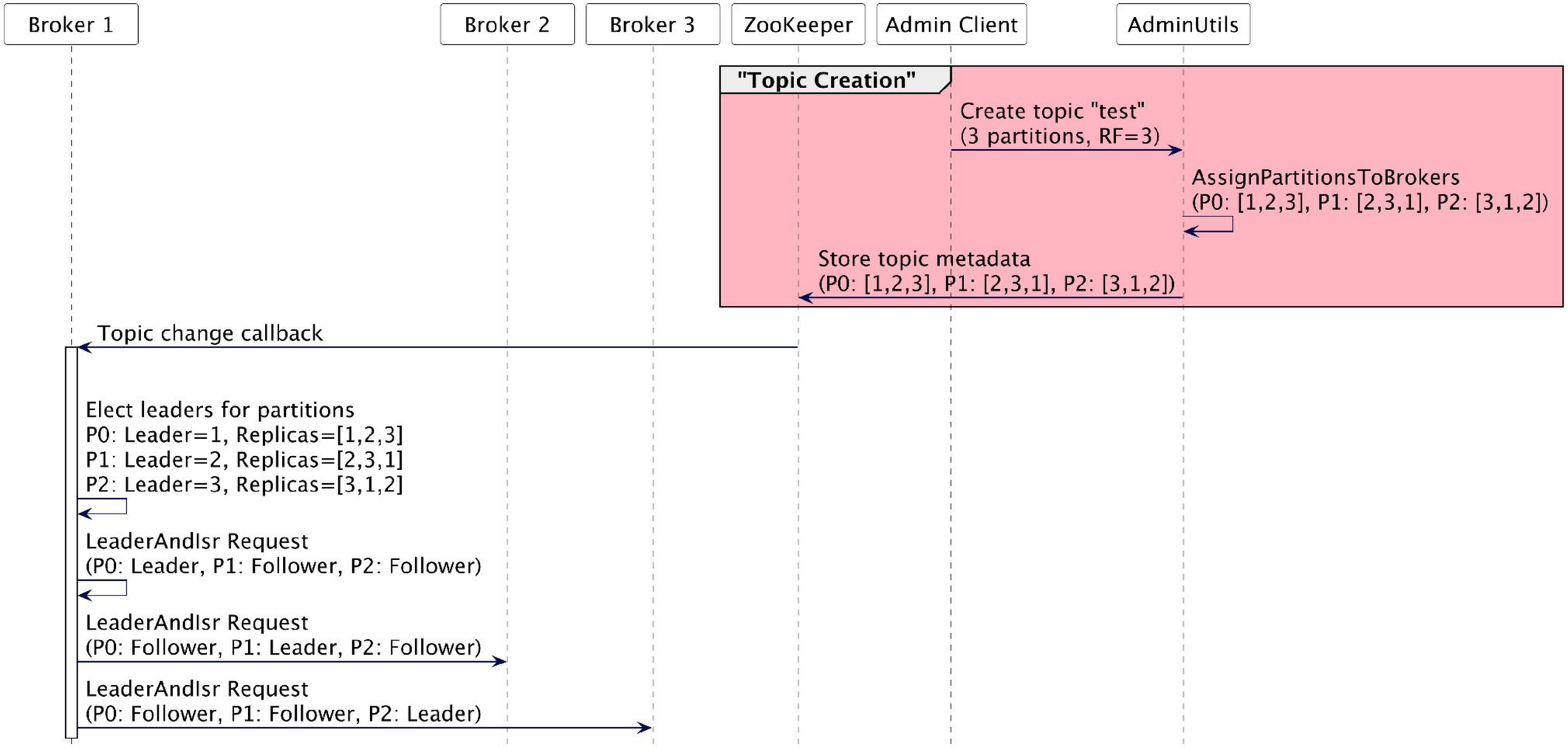
Topic Creation uses fixed number of partitions – Step 7



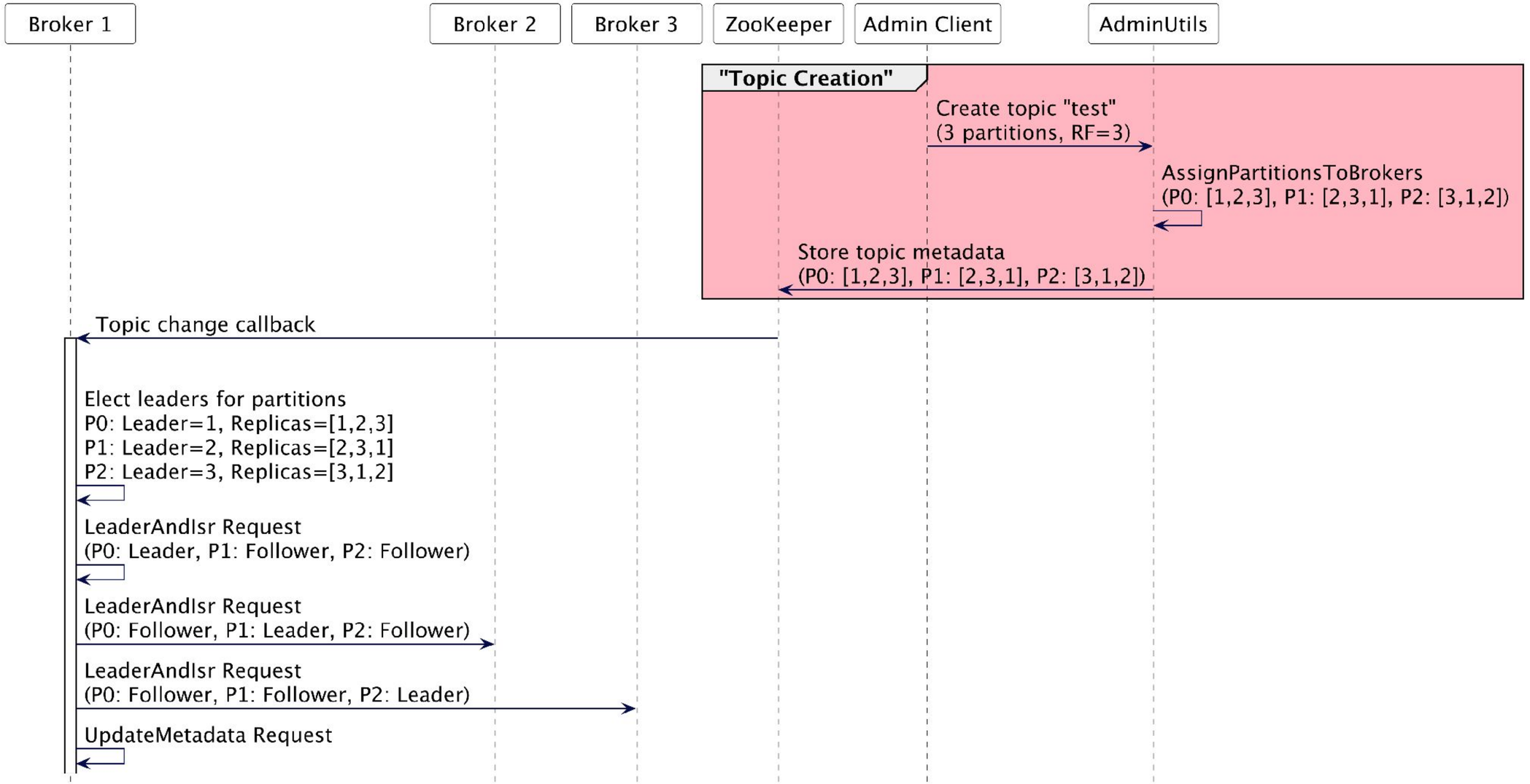
Topic Creation uses fixed number of partitions – Step 8



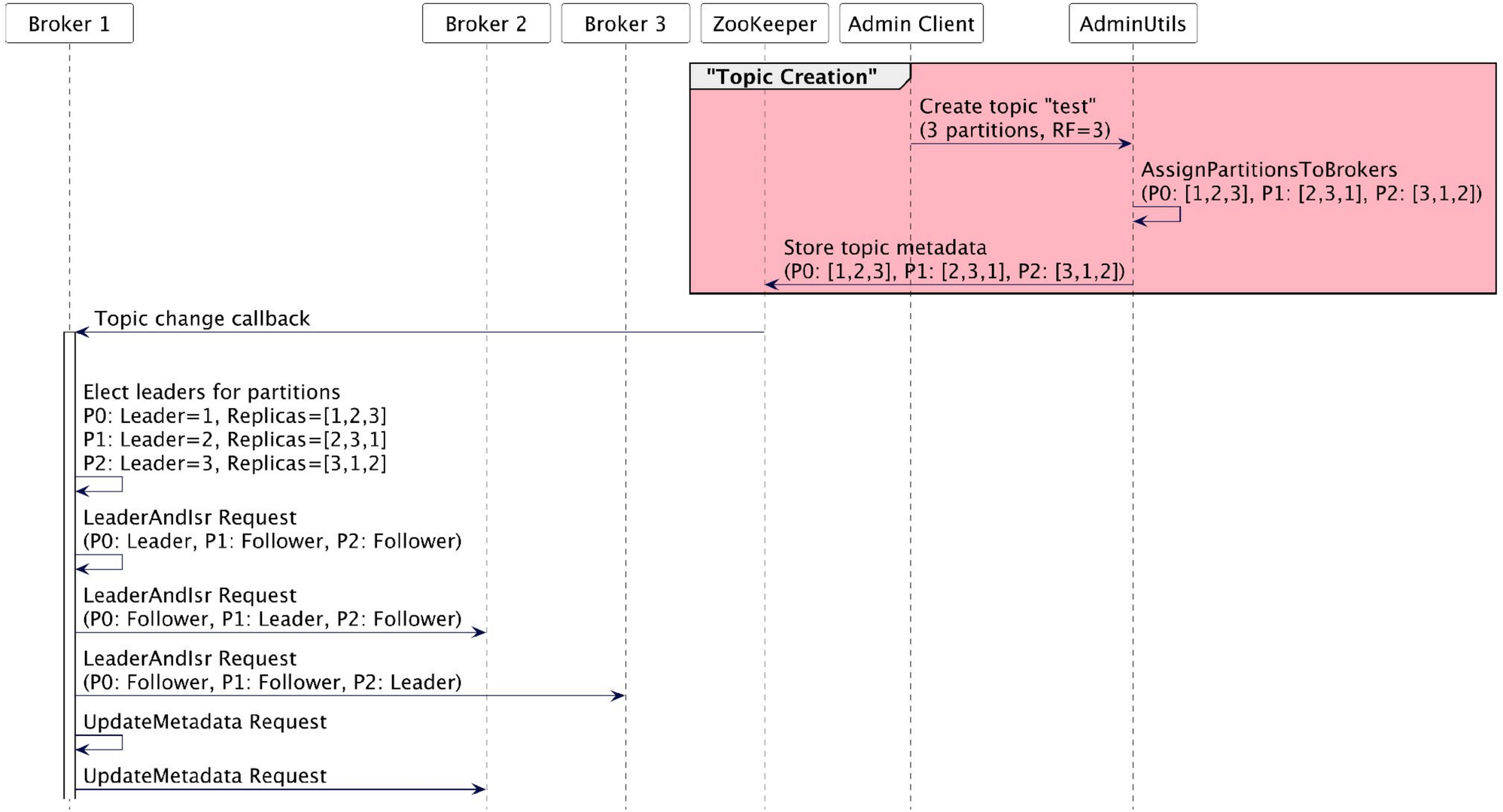
Topic Creation uses fixed number of partitions - Step 9



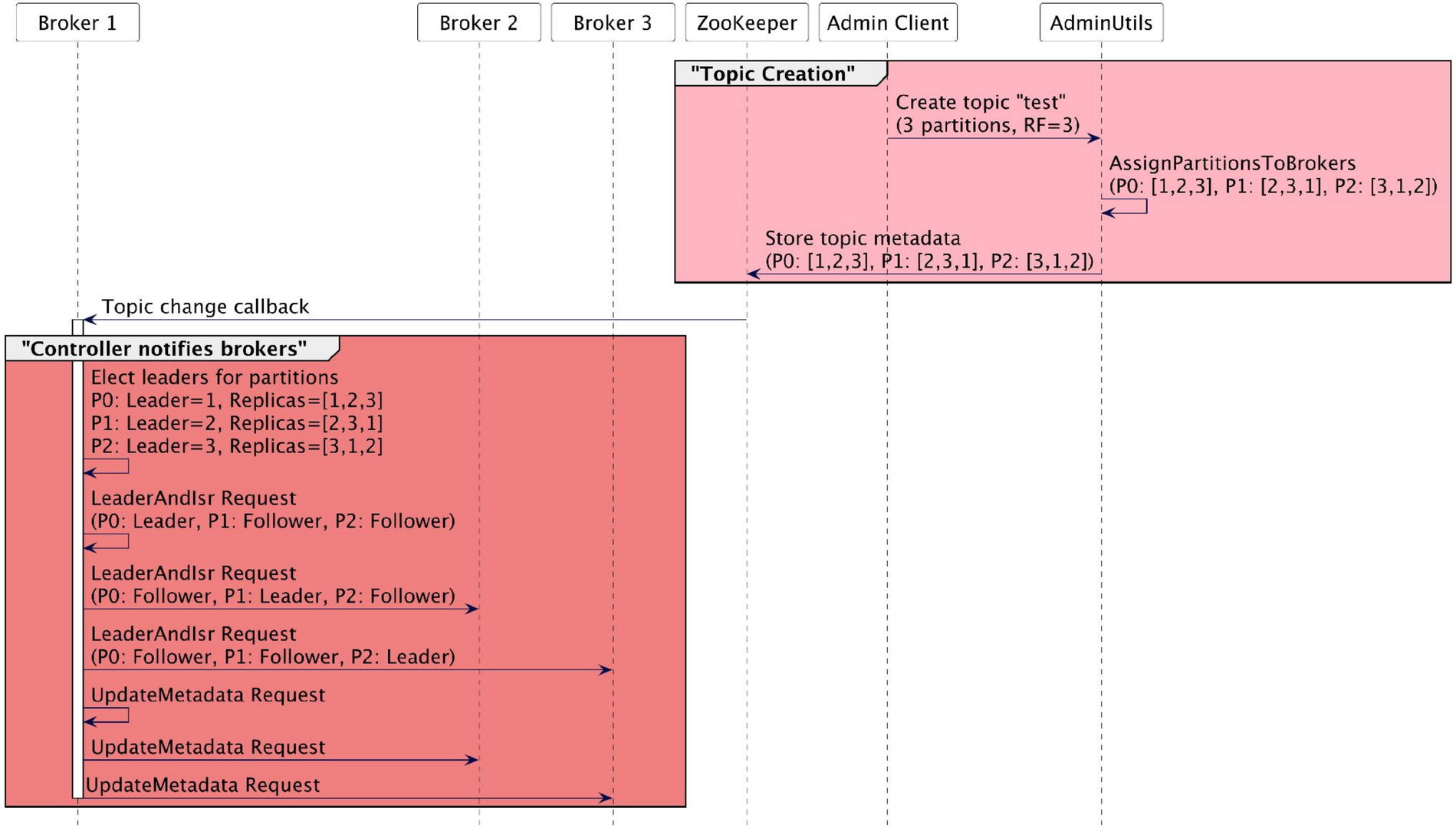
Topic Creation uses fixed number of partitions - Step 10



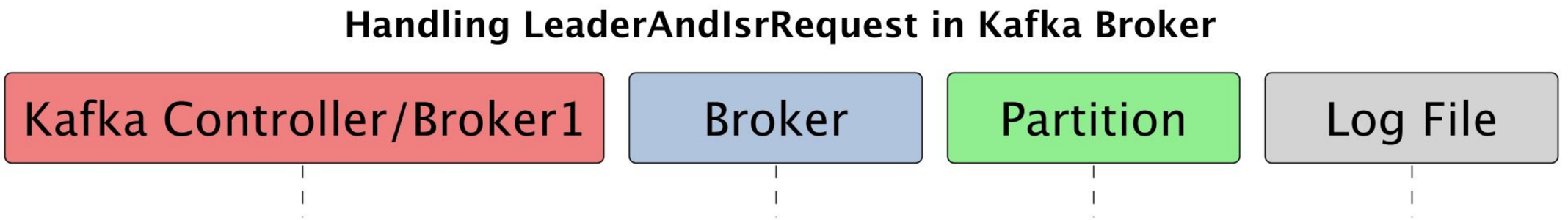
Topic Creation uses fixed number of partitions - Step 11



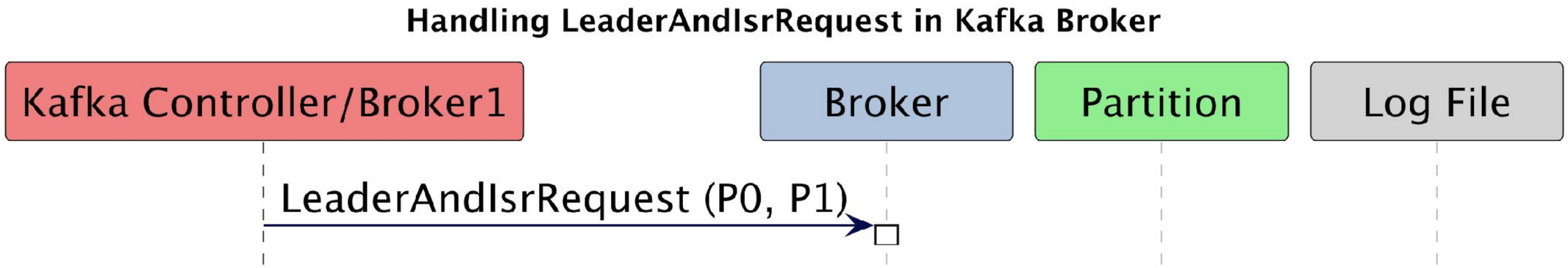
Topic Creation uses fixed number of partitions - Step 12



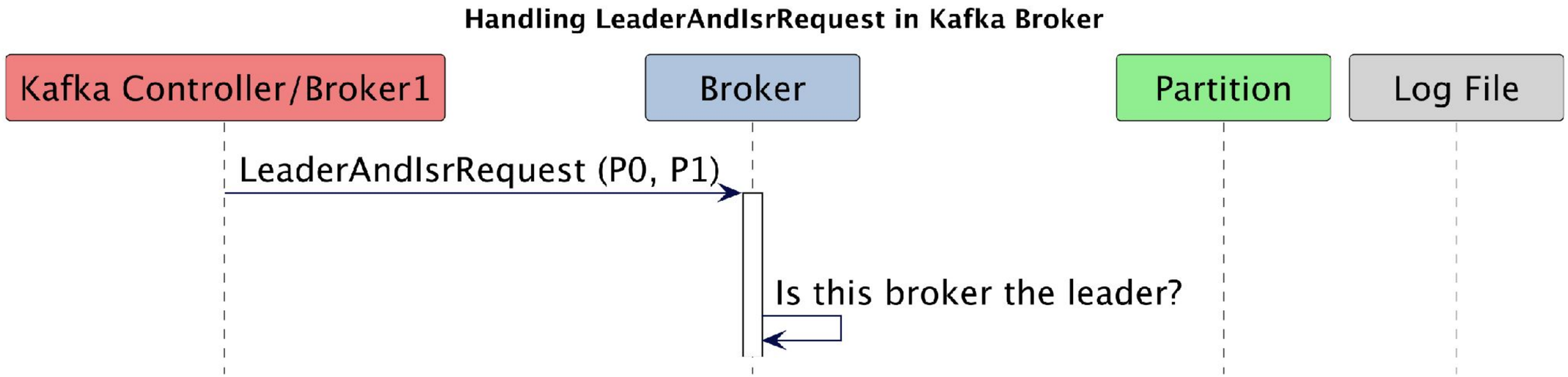
Brokers handle LeaderAndIsrRequest – Step 1



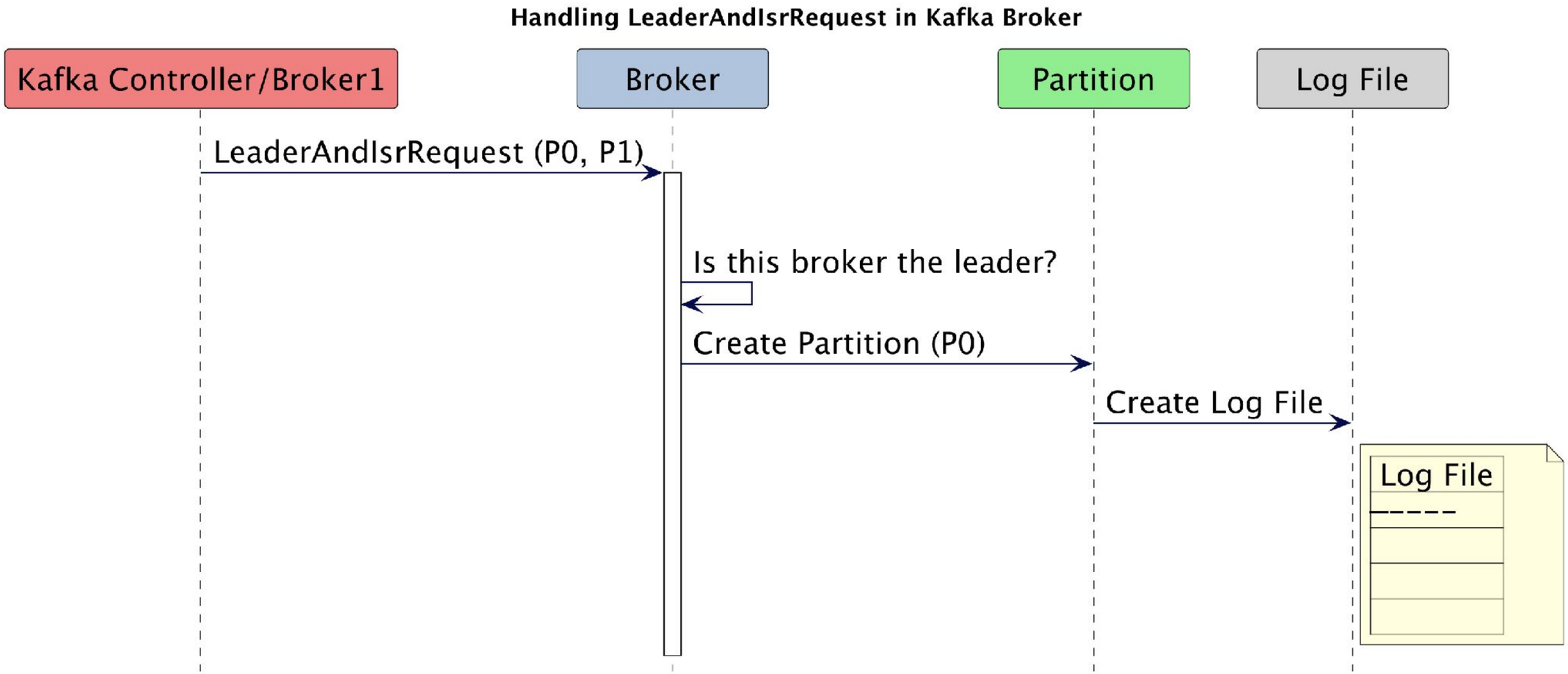
Brokers handle LeaderAndIsrRequest – Step 2



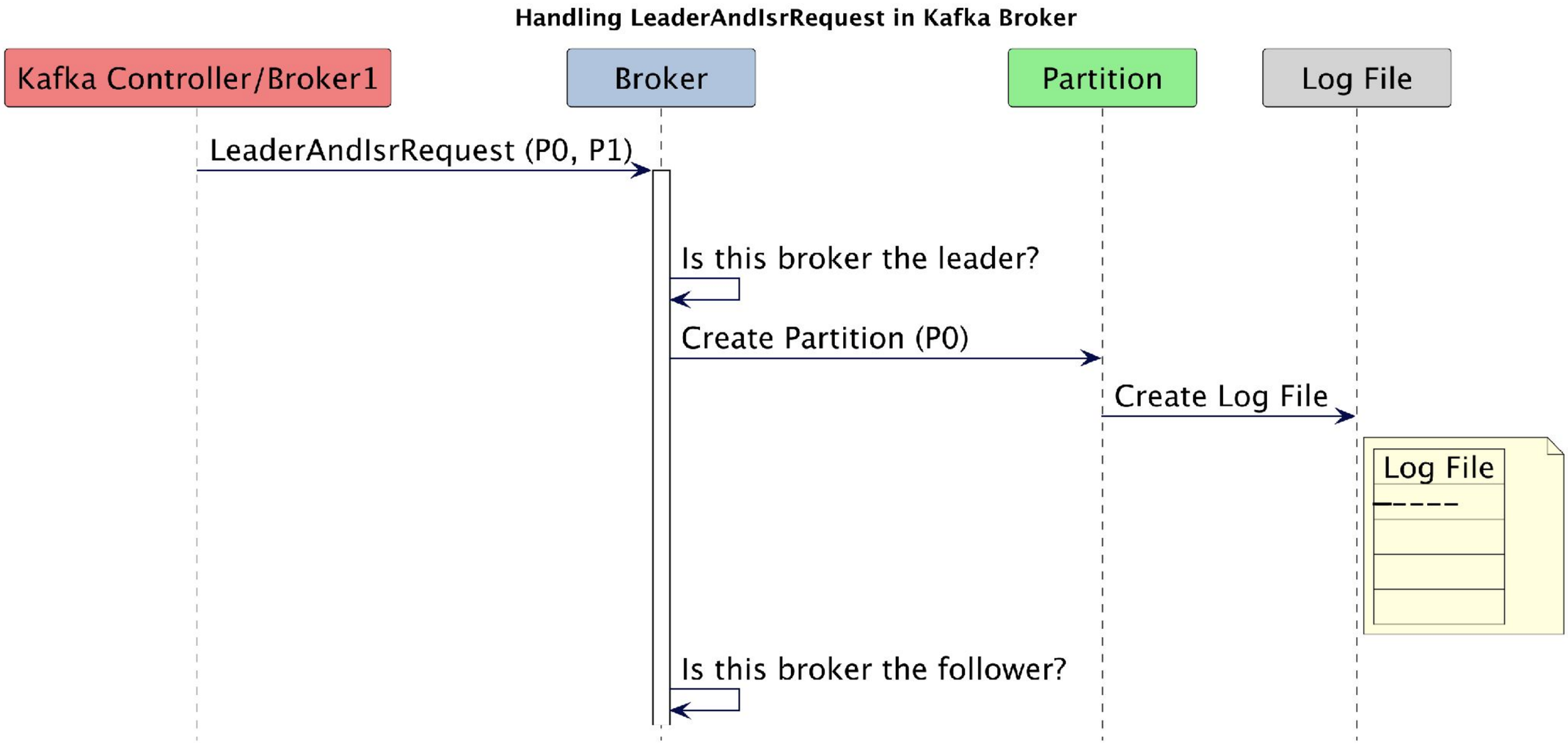
Brokers handle LeaderAndIsrRequest – Step 3



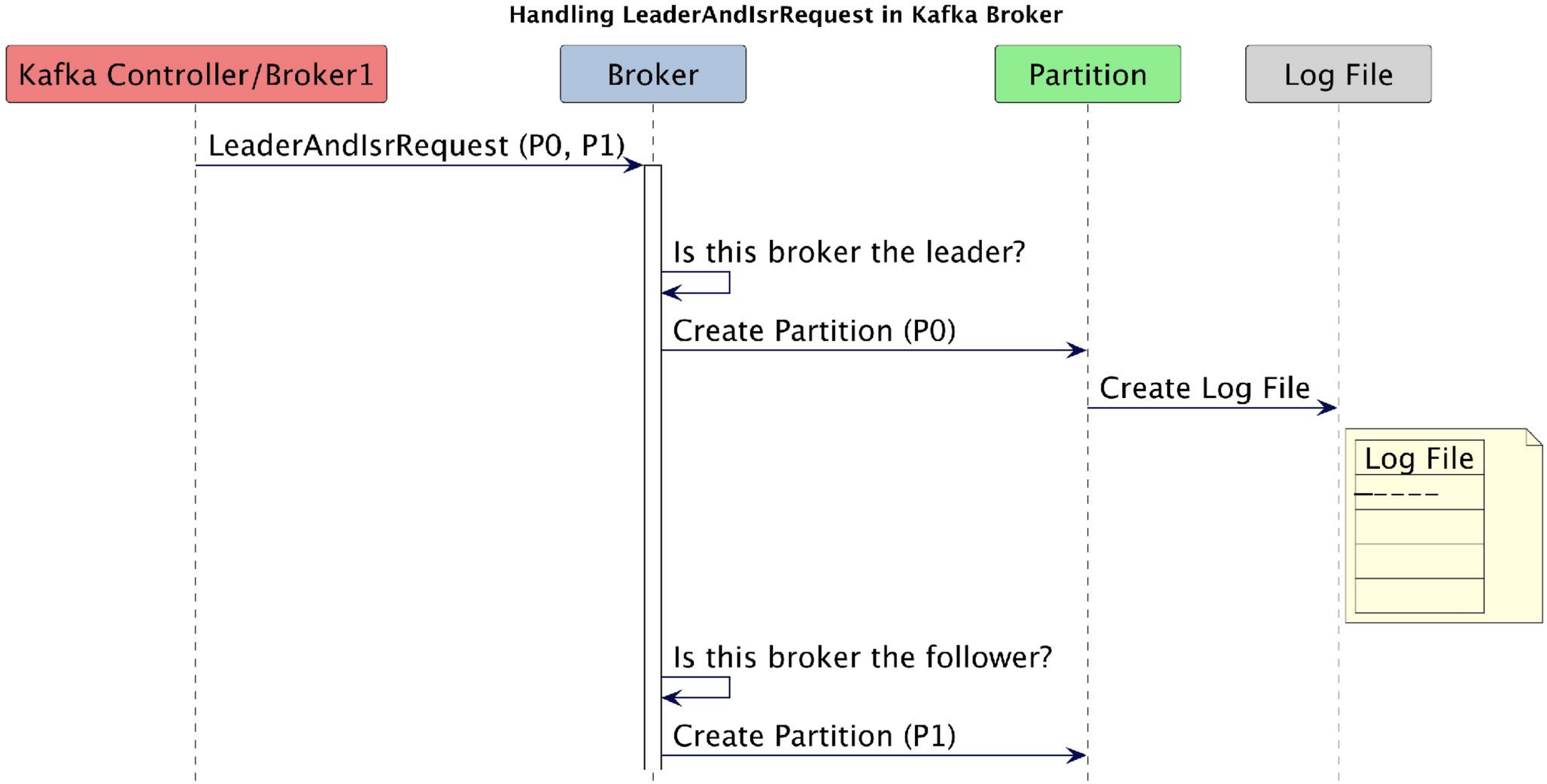
Brokers handle LeaderAndIsrRequest – Step 4



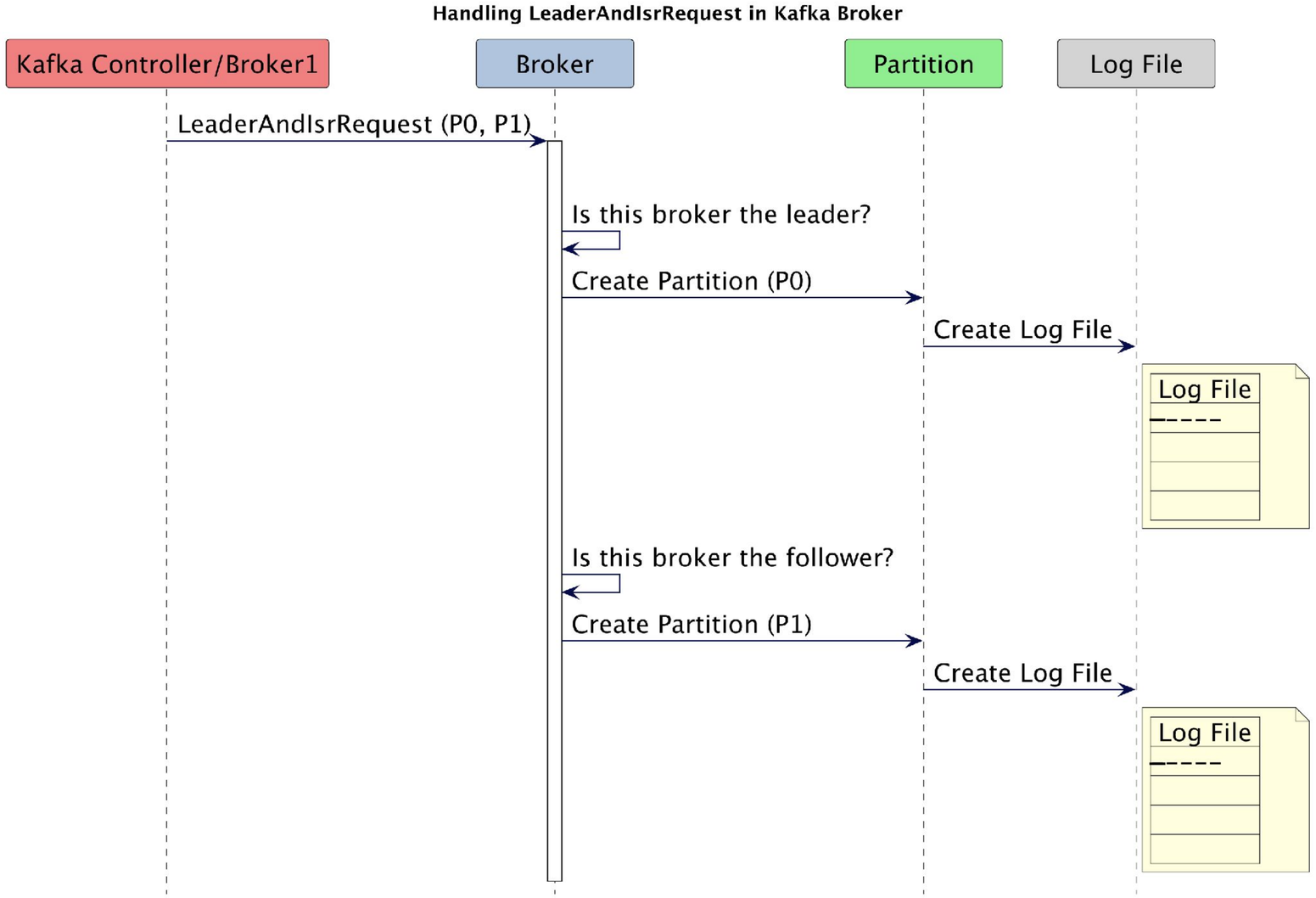
Brokers handle LeaderAndIsrRequest – Step 5



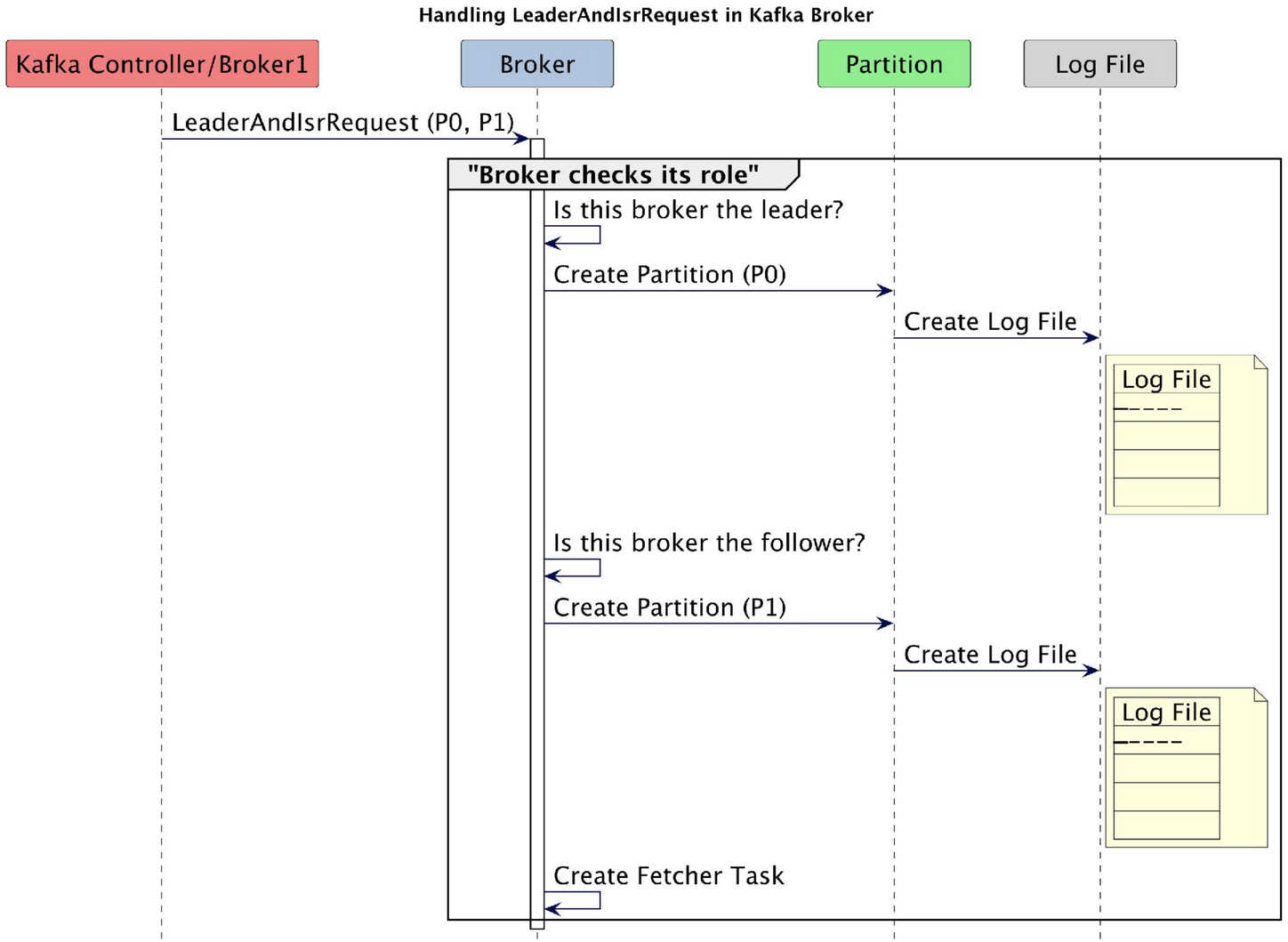
Brokers handle LeaderAndIsrRequest – Step 6



Brokers handle LeaderAndIsrRequest – Step 7



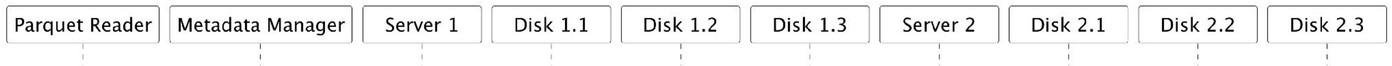
Brokers handle LeaderAndIsrRequest - Step 8



Parquet: Columnar Storage Architecture

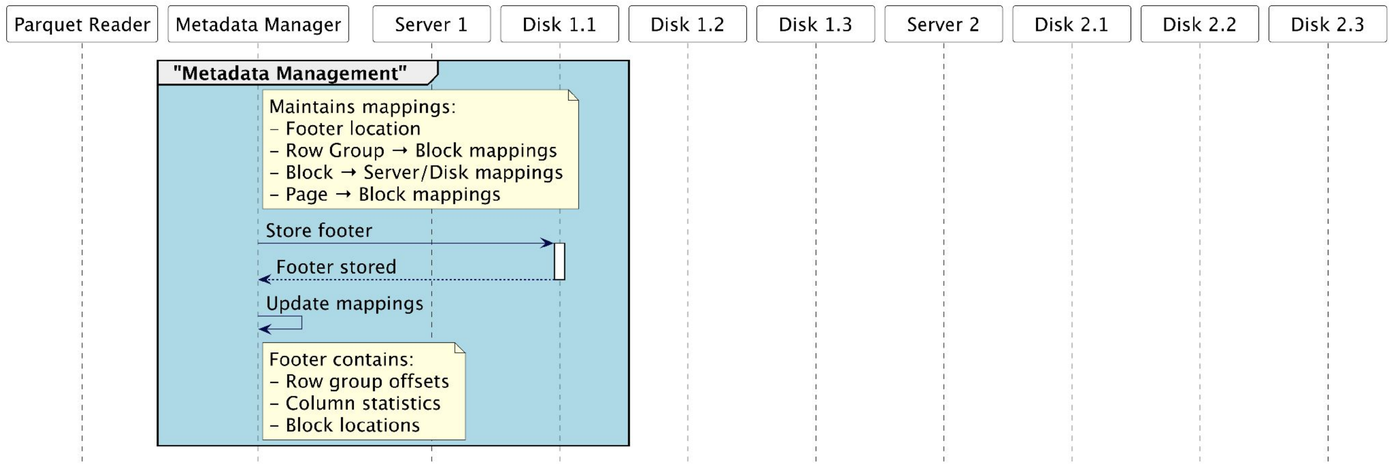
- Columnar Storage: Stores data by columns rather than rows, improving read performance for analytics
- Efficient Compression: Applies type-aware compression (dictionary, RLE, Snappy), reducing storage size
- Schema Evolution: Supports changes to schema without breaking existing stored files
- Compatibility: Native integration with Spark, Hadoop, Hive, Trino, and DuckDB
- Analytical Queries: Ideal for read-heavy operations, column pruning, and vectorization

Parquet Data Organization: Row Groups & Column Chunks – Step 1



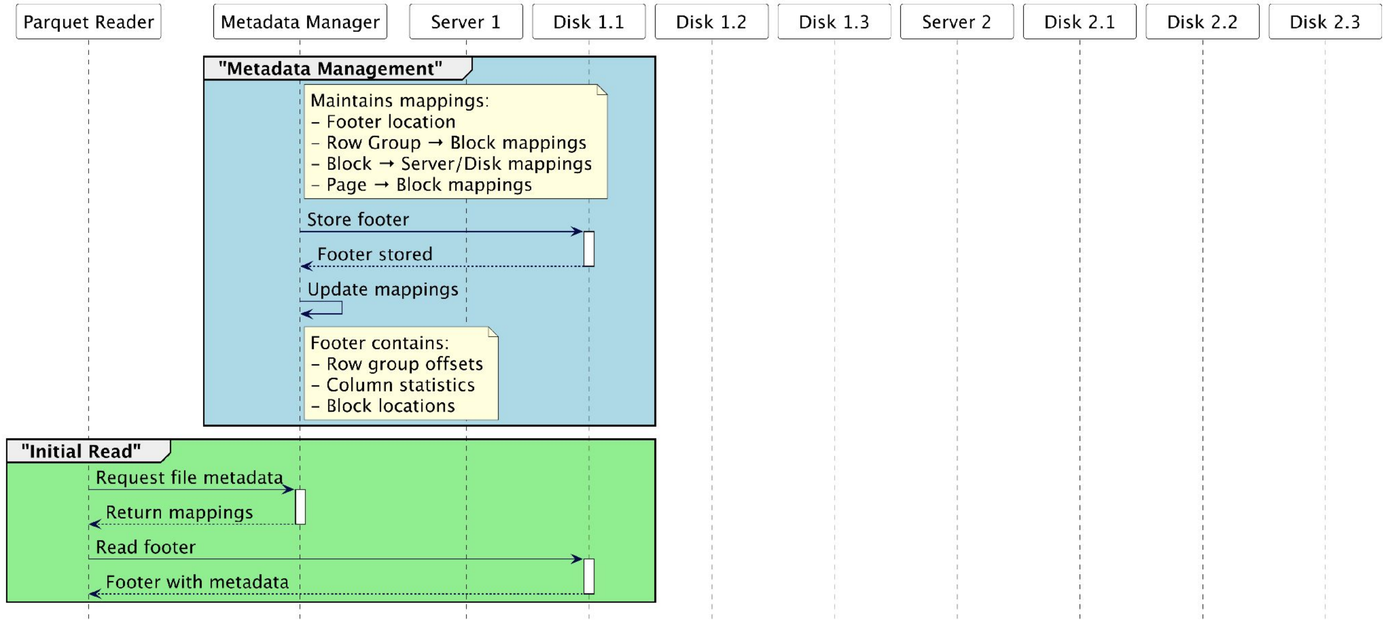
- Data divided into Row Groups representing chunks of column values for all records
- Each row group contains column chunks broken into individual data pages
- File footer contains min/max statistics per column chunk for predicate pushdown

Parquet Data Organization: Row Groups & Column Chunks – Step 2



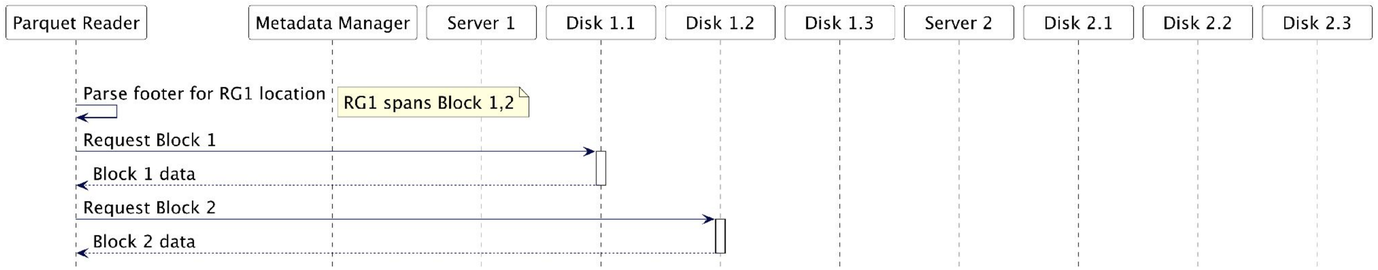
- Data divided into Row Groups representing chunks of column values for all records
- Each row group contains column chunks broken into individual data pages
- File footer contains min/max statistics per column chunk for predicate pushdown

Parquet Data Organization: Row Groups & Column Chunks - Step 3



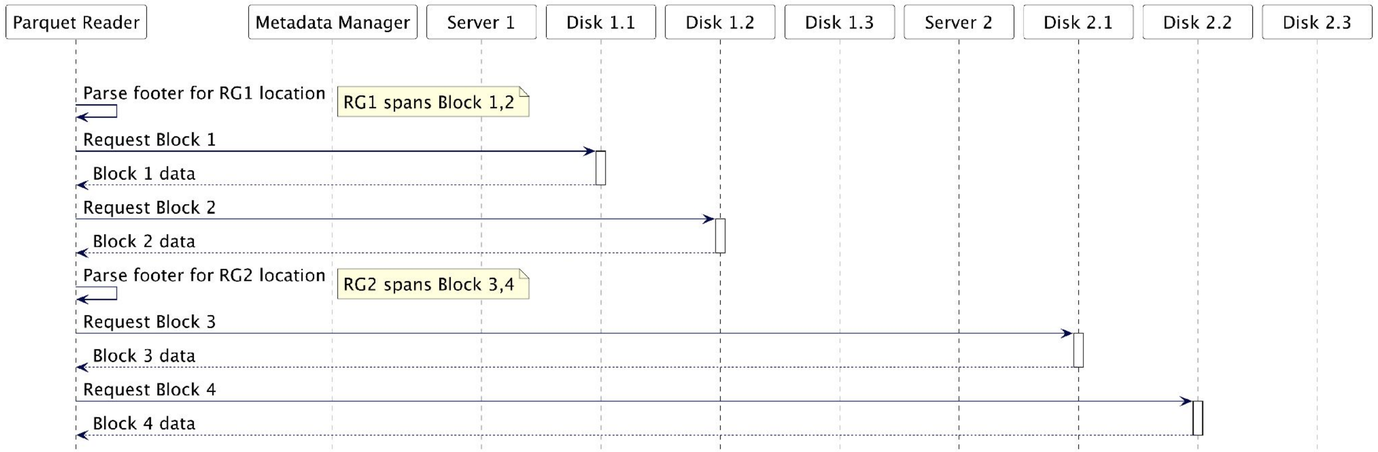
- Data divided into Row Groups representing chunks of column values for all records
- Each row group contains column chunks broken into individual data pages
- File footer contains min/max statistics per column chunk for predicate pushdown

Parquet Data Organization: Row Groups & Column Chunks – Step 4



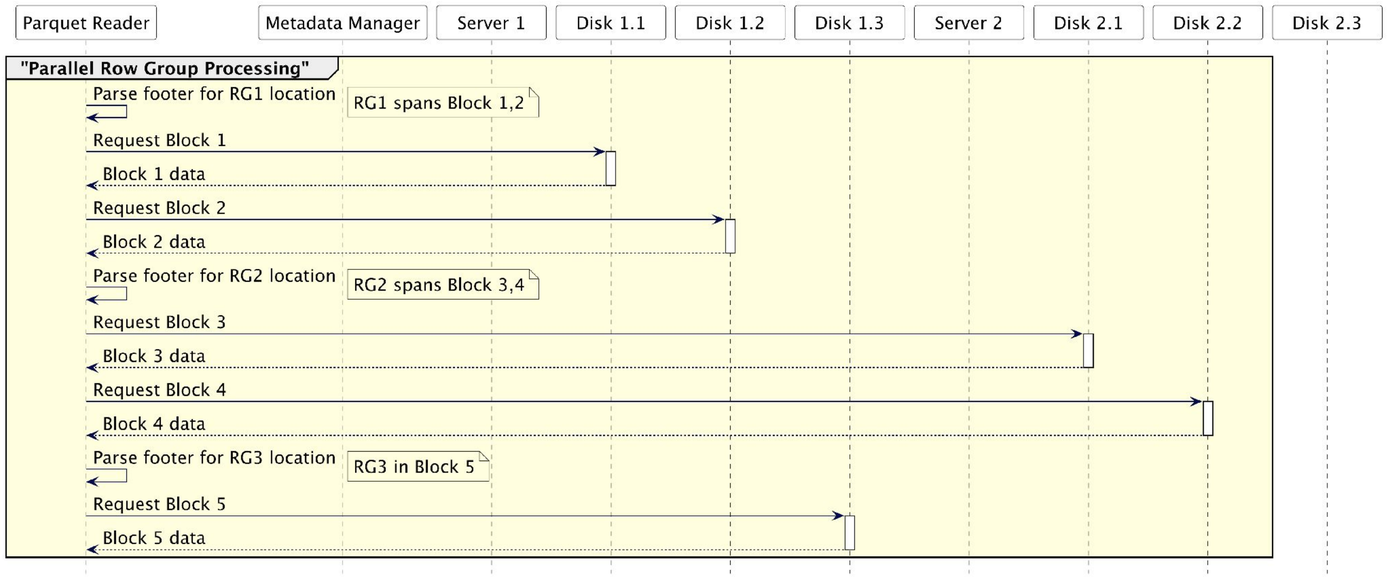
- Data divided into Row Groups representing chunks of column values for all records
- Each row group contains column chunks broken into individual data pages
- File footer contains min/max statistics per column chunk for predicate pushdown

Parquet Data Organization: Row Groups & Column Chunks – Step 5



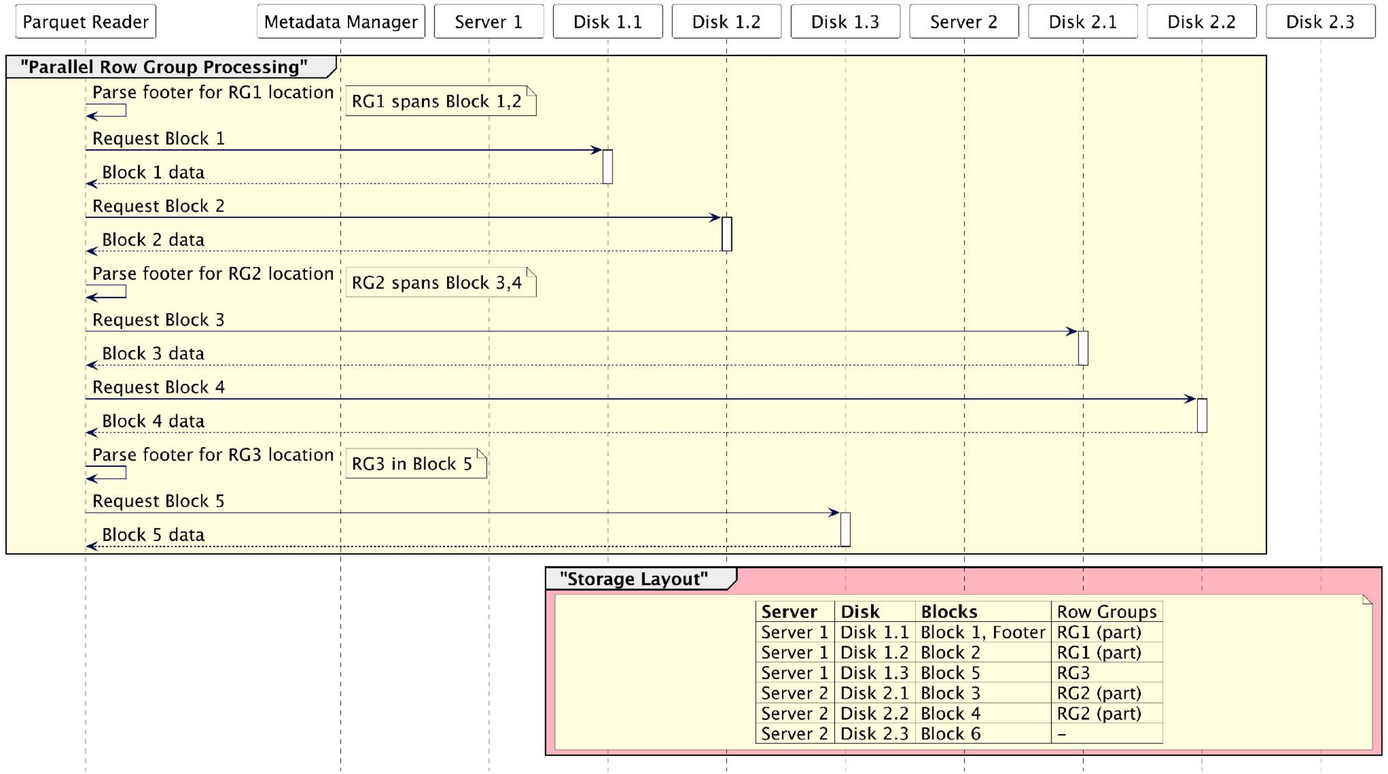
- Data divided into Row Groups representing chunks of column values for all records
- Each row group contains column chunks broken into individual data pages
- File footer contains min/max statistics per column chunk for predicate pushdown

Parquet Data Organization: Row Groups & Column Chunks - Step 6



- Data divided into Row Groups representing chunks of column values for all records
- Each row group contains column chunks broken into individual data pages
- File footer contains min/max statistics per column chunk for predicate pushdown

Parquet Data Organization: Row Groups & Column Chunks - Step 7



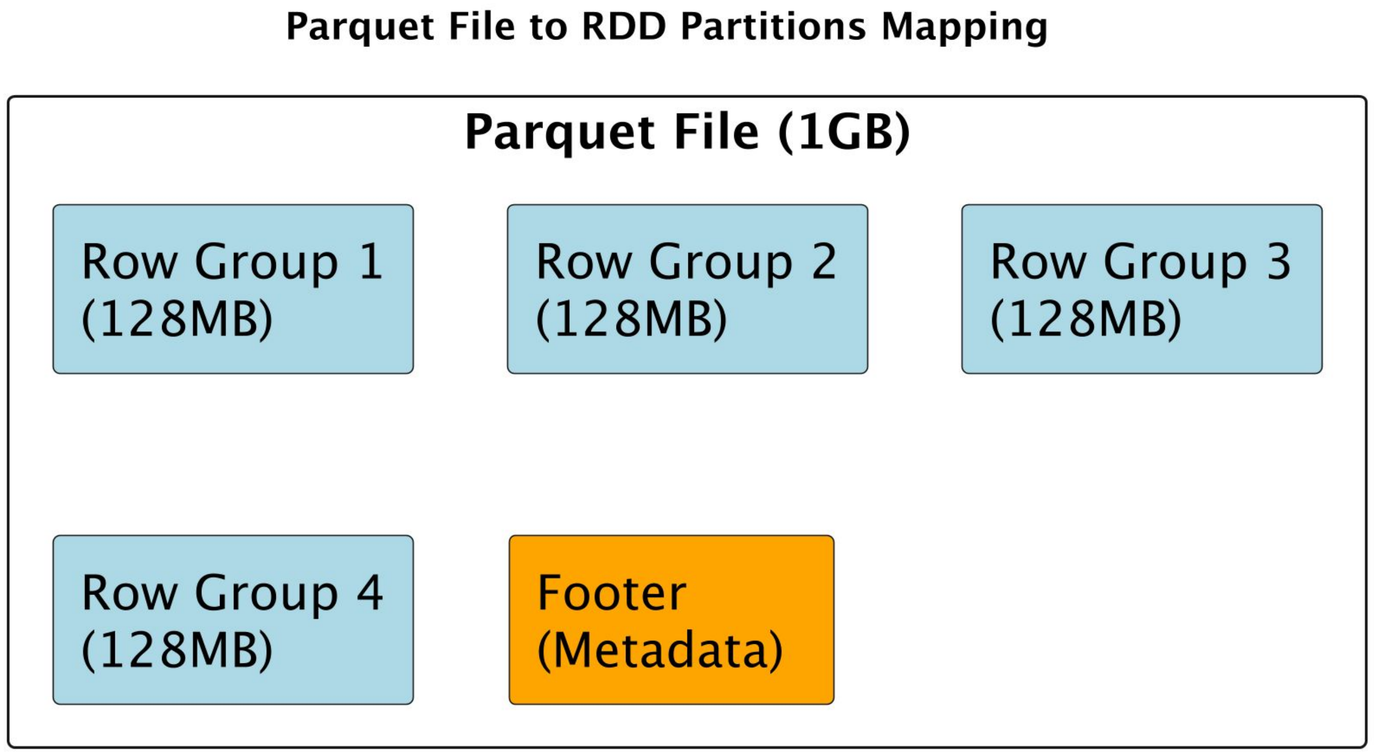
Server	Disk	Blocks	Row Groups
Server 1	Disk 1.1	Block 1, Footer	RG1 (part)
Server 1	Disk 1.2	Block 2	RG1 (part)
Server 1	Disk 1.3	Block 5	RG3
Server 2	Disk 2.1	Block 3	RG2 (part)
Server 2	Disk 2.2	Block 4	RG2 (part)
Server 2	Disk 2.3	Block 6	–

- Data divided into Row Groups representing chunks of column values for all records
- Each row group contains column chunks broken into individual data pages
- File footer contains min/max statistics per column chunk for predicate pushdown

Parquet + Distributed Processing: Perfect Match

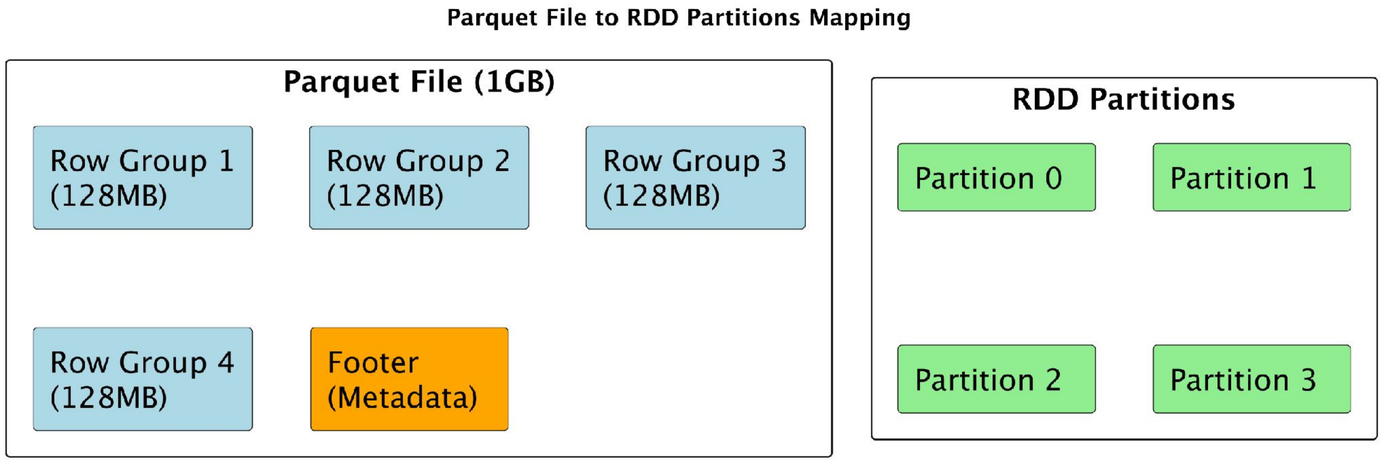
- Row Groups = Natural Partitions: Each row group becomes a task
- Self-Describing: Metadata enables independent processing
- Predicate Pushdown: Skip entire row groups using statistics
- Column Pruning: Read only necessary data

Parquet to RDD Mapping – Step 1



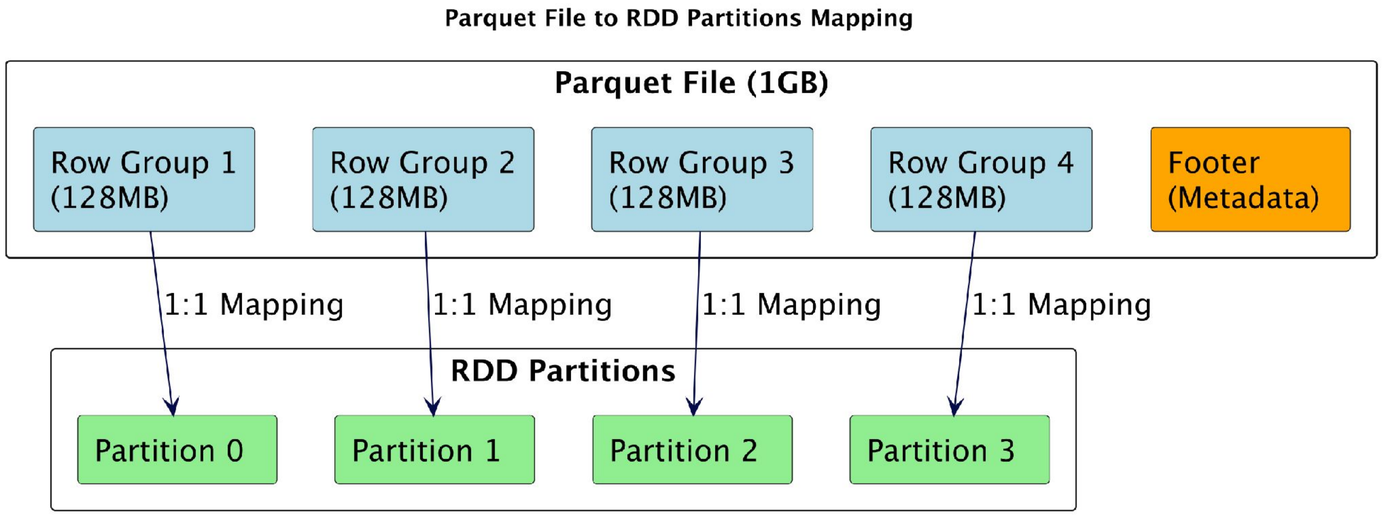
- Driver reads Parquet footer to discover row groups
- Each row group becomes one RDD partition
- Tasks scheduled on nodes with local row group data
- Parallel processing of independent row groups

Parquet to RDD Mapping – Step 2



- Driver reads Parquet footer to discover row groups
- Each row group becomes one RDD partition
- Tasks scheduled on nodes with local row group data
- Parallel processing of independent row groups

Parquet to RDD Mapping – Step 3

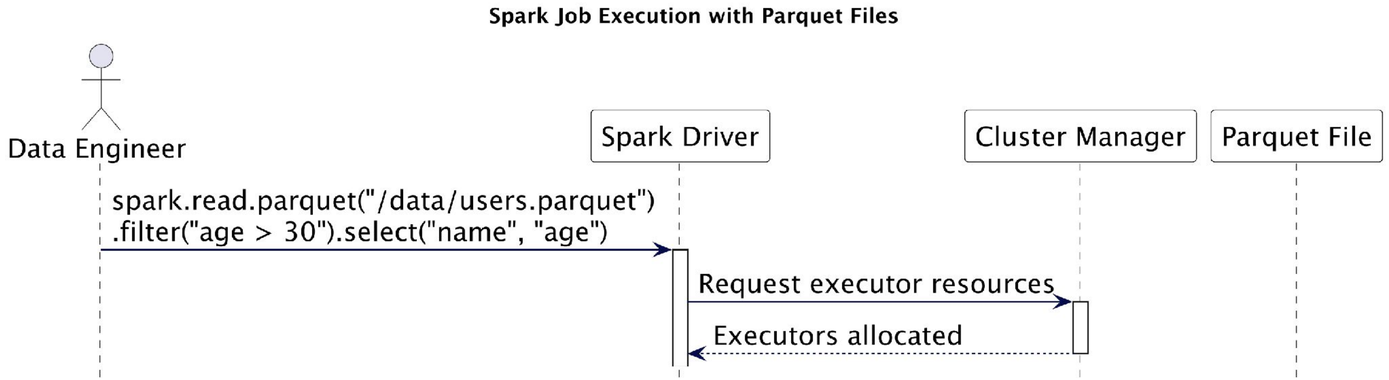


- Driver reads Parquet footer to discover row groups
- Each row group becomes one RDD partition
- Tasks scheduled on nodes with local row group data
- Parallel processing of independent row groups

Spark's Parquet Integration

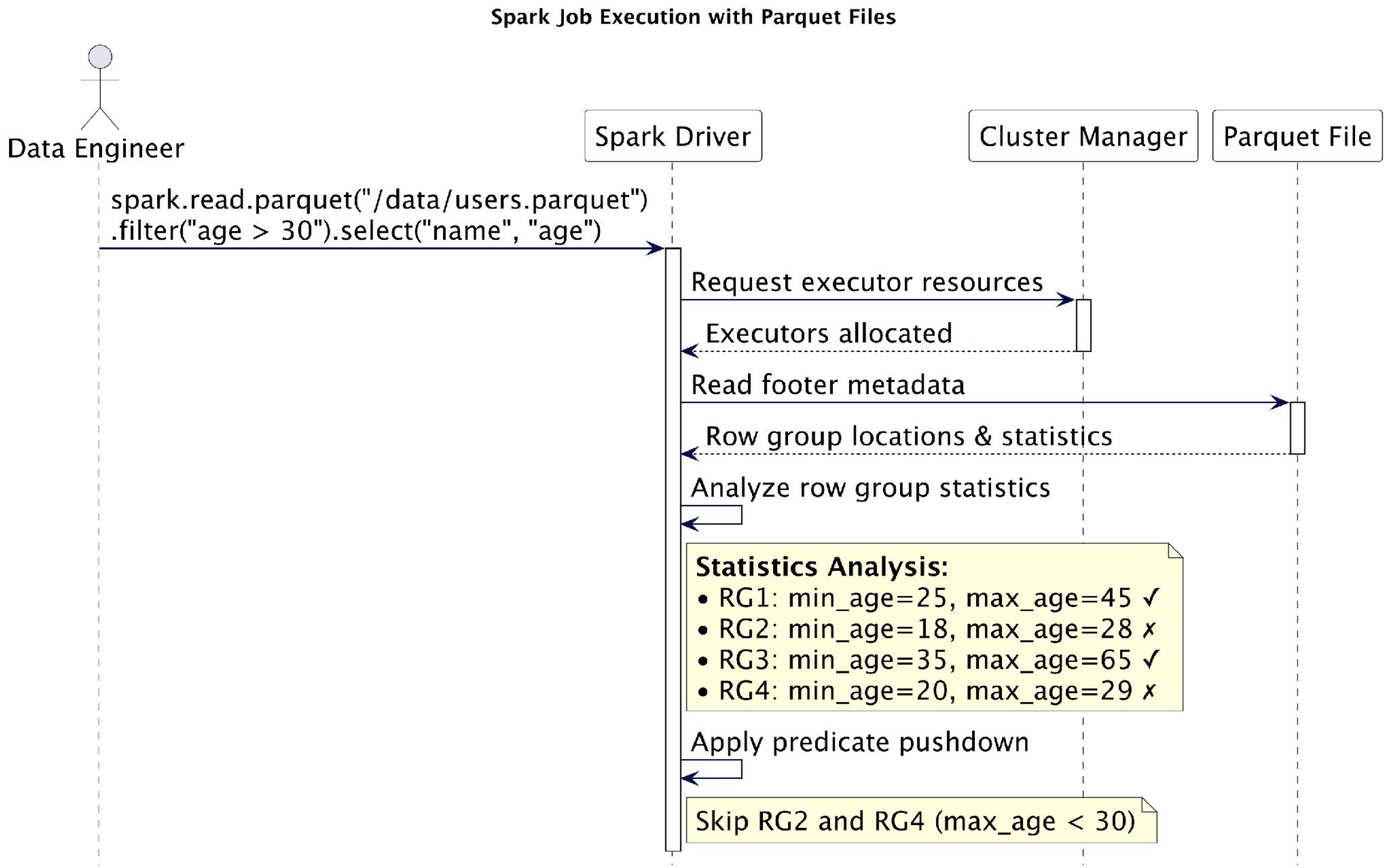
- ParquetInputFormat: Automatically creates one split per row group
- Catalyst Optimizer: Pushes predicates down to Parquet reader
- Vectorized Execution: Processes columnar data in batches
- Data Locality: Schedules tasks on nodes with row group data

End-to-End Spark + Parquet Execution – Step 1



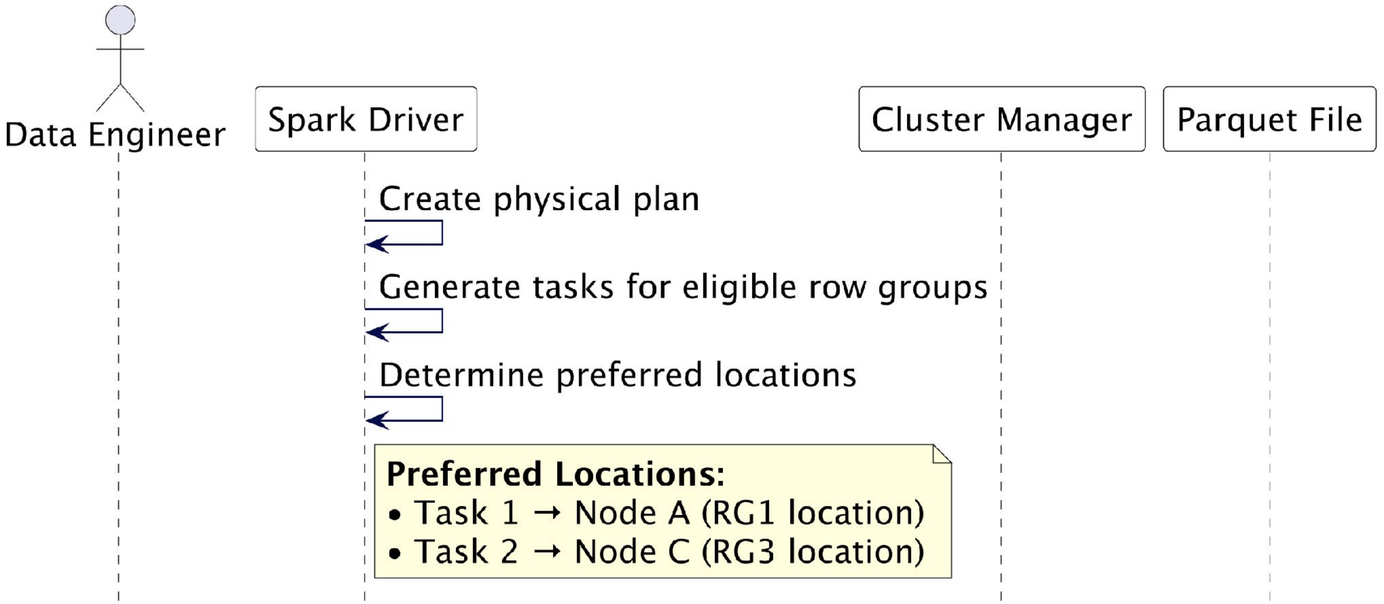
- Driver analyzes Parquet metadata and creates execution plan
- Tasks distributed to executors with row group assignments
- Parallel processing with predicate and projection pushdown
- Results collected and combined for final output

End-to-End Spark + Parquet Execution – Step 2



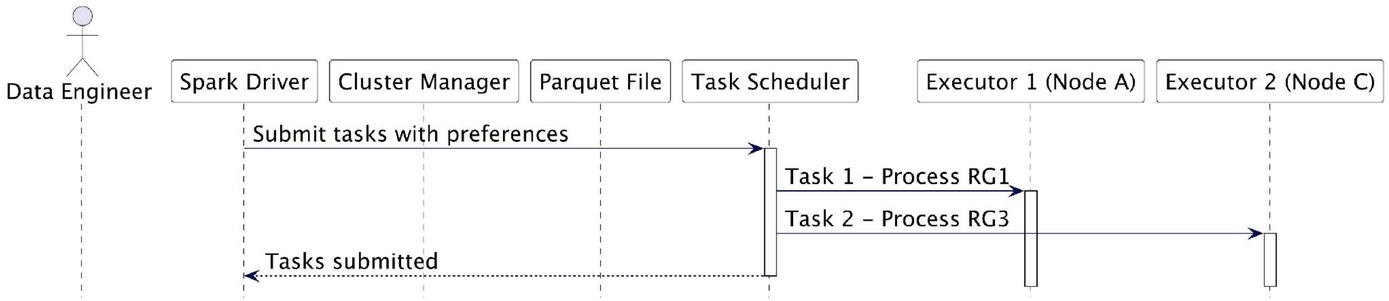
- Driver analyzes Parquet metadata and creates execution plan
- Tasks distributed to executors with row group assignments
- Parallel processing with predicate and projection pushdown
- Results collected and combined for final output

End-to-End Spark + Parquet Execution – Step 3



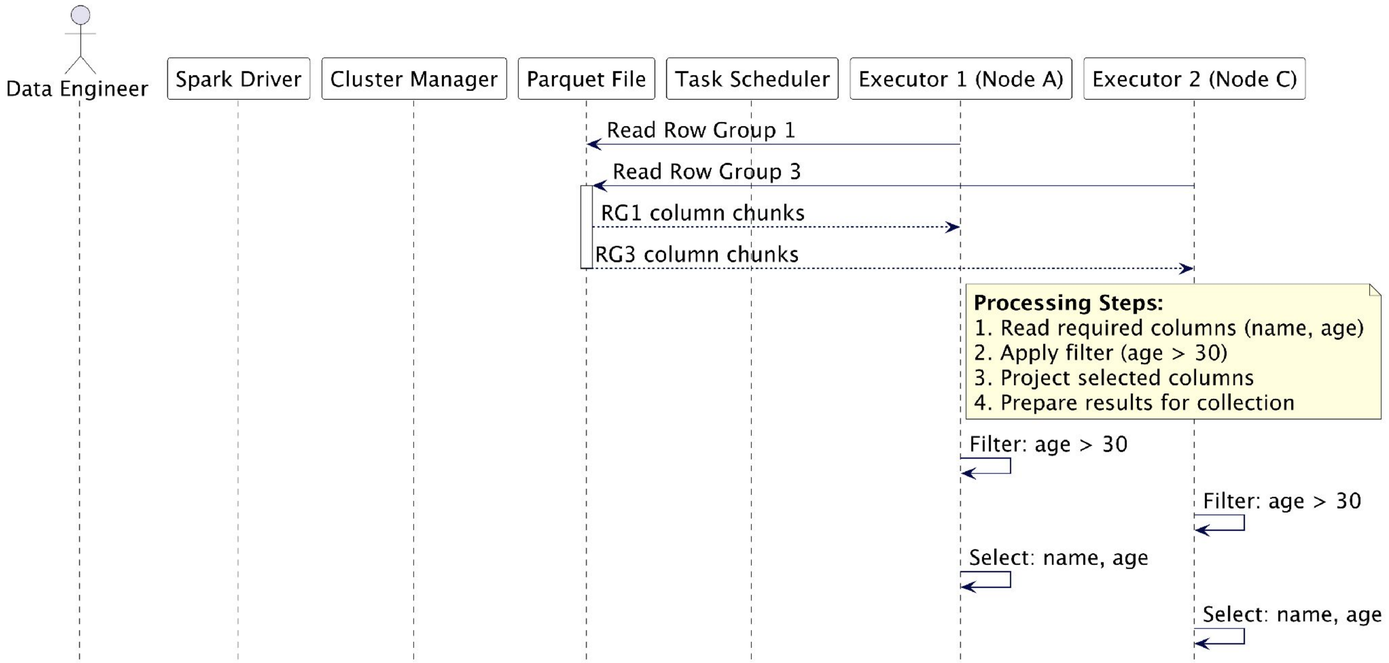
- Driver analyzes Parquet metadata and creates execution plan
- Tasks distributed to executors with row group assignments
- Parallel processing with predicate and projection pushdown
- Results collected and combined for final output

End-to-End Spark + Parquet Execution – Step 4



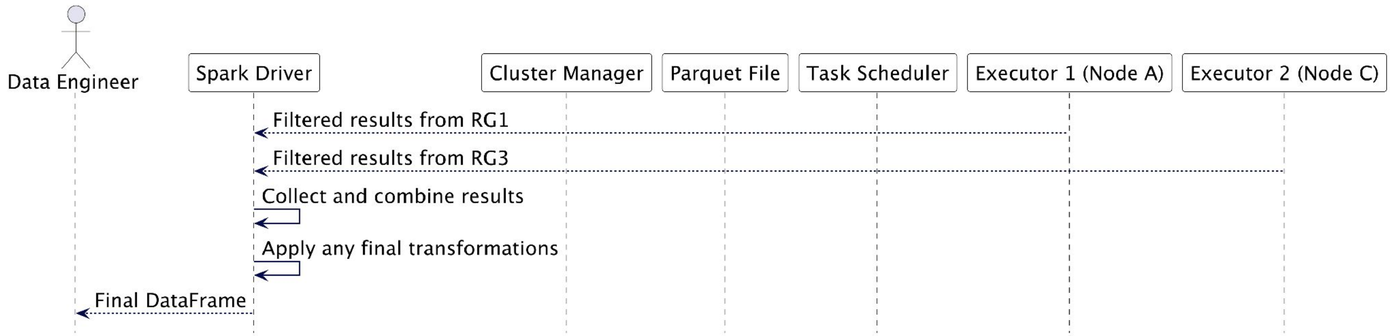
- Driver analyzes Parquet metadata and creates execution plan
- Tasks distributed to executors with row group assignments
- Parallel processing with predicate and projection pushdown
- Results collected and combined for final output

End-to-End Spark + Parquet Execution – Step 5



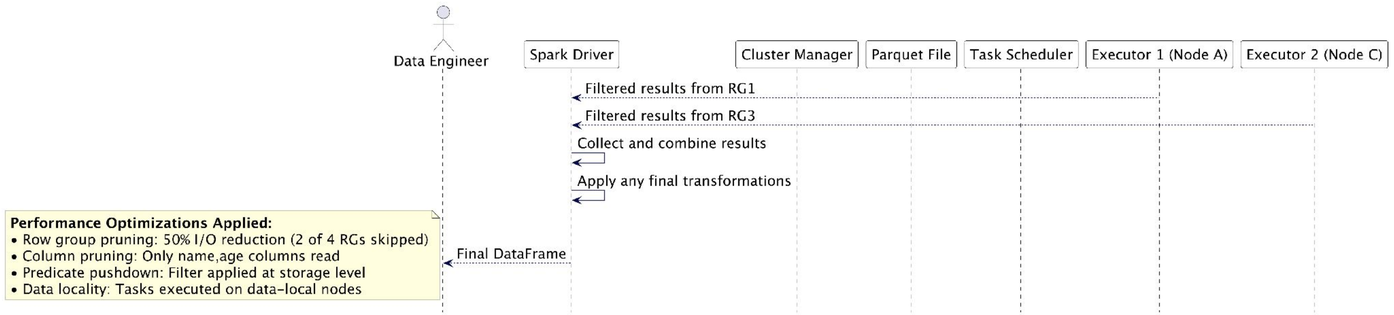
- Driver analyzes Parquet metadata and creates execution plan
- Tasks distributed to executors with row group assignments
- Parallel processing with predicate and projection pushdown
- Results collected and combined for final output

End-to-End Spark + Parquet Execution – Step 6



- Driver analyzes Parquet metadata and creates execution plan
- Tasks distributed to executors with row group assignments
- Parallel processing with predicate and projection pushdown
- Results collected and combined for final output

End-to-End Spark + Parquet Execution – Step 7



- Driver analyzes Parquet metadata and creates execution plan
- Tasks distributed to executors with row group assignments
- Parallel processing with predicate and projection pushdown
- Results collected and combined for final output

Assignment: Distributed Parquet Processing

- File: `spark-module/src/test/java/minispark/fileformat/ParquetWorkerTest.java`
- Goal: Build end-to-end distributed Parquet processing system
- Task 1: Implement FileFormatRDD with ParquetReader
- Task 2: Create multiple row groups for parallel processing
- Task 3: Use DAGScheduler to distribute tasks across workers
- Task 4: Collect and verify results from distributed execution
- Experience: Complete Spark-like distributed file processing

Real-World Performance Impact

- Query Performance: 10-100x faster than row-based formats
- Storage Efficiency: 75% compression typical
- Network Reduction: 90% less data movement through column pruning
- Cost Savings: Significant reduction in compute and storage costs

Module 4 · Consistency Across Replicas & Partitions

High-watermark and Replica Consistency

- Leader-based replication requires a consensus boundary for committed data
- How to ensure all followers maintain identical prefix of committed logs?
- How to protect clients from reading values that may later be lost on leader failover?

Replica Consistency

- Leader receives writes and appends to local log first
- Followers replicate entries from leader via fetch requests
- Entries cannot be considered committed until stored on a majority of replicas

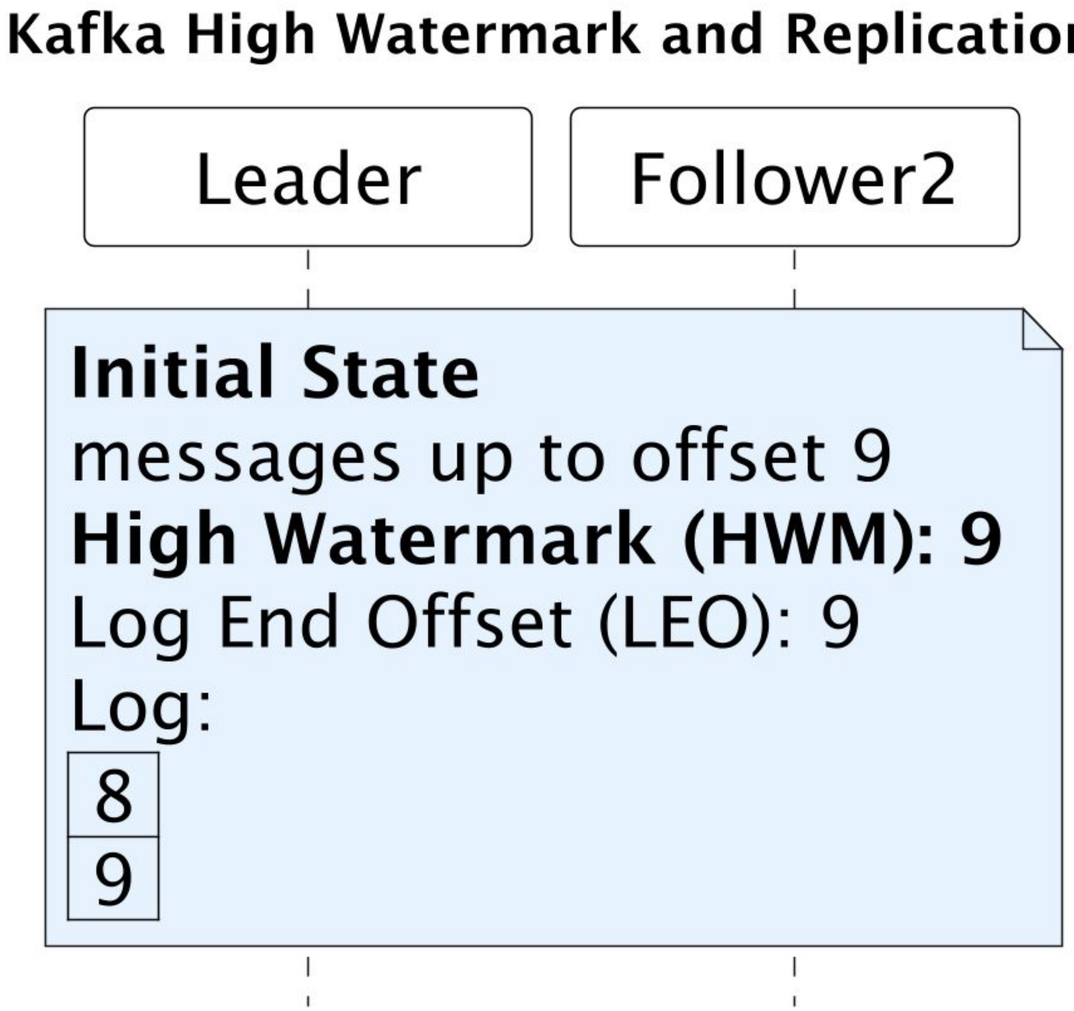
Leader Based Replication

- Producer sends produce request for `<M1>` on `/Transactions, P0` to Leader (Broker 1)
- Follower brokers (Broker 2, Broker 3) send Fetch requests to pull data
- Leader responds to fetch requests with new log entries
- Consumer reads from leader

Features of leader based replication

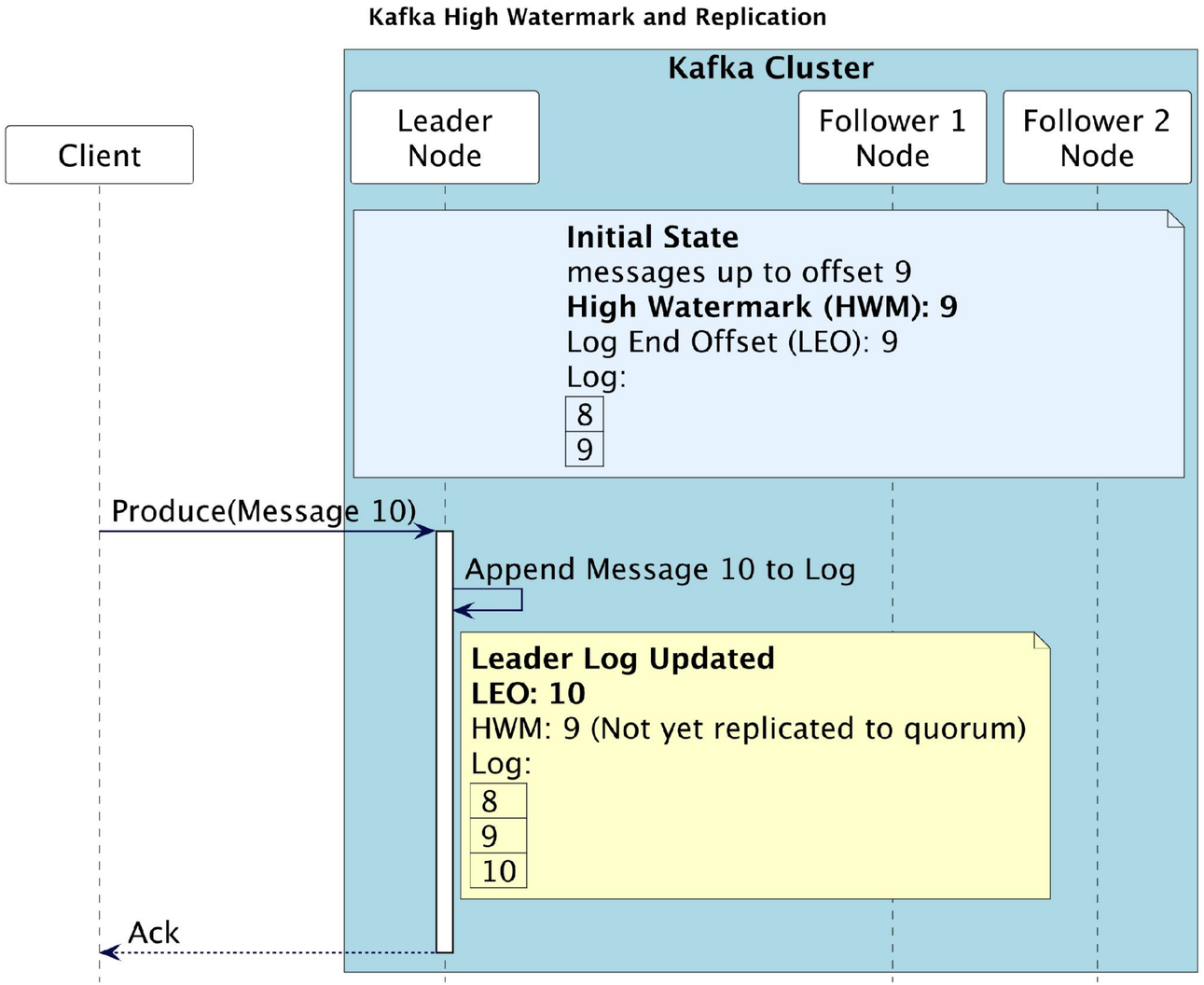
- Strict Ordering of messages/events/state changes
- The server itself is responsible for coordinating data copy across replicas
- Stronger consistency guarantees as every Reader/Writer will always go through the leader
- Push vs Pull for replication: Pull is needed anyway when follower fails and rejoins
- The follower/consumer can decide rate of pull

Consistency - HighWaterMark - Step 1



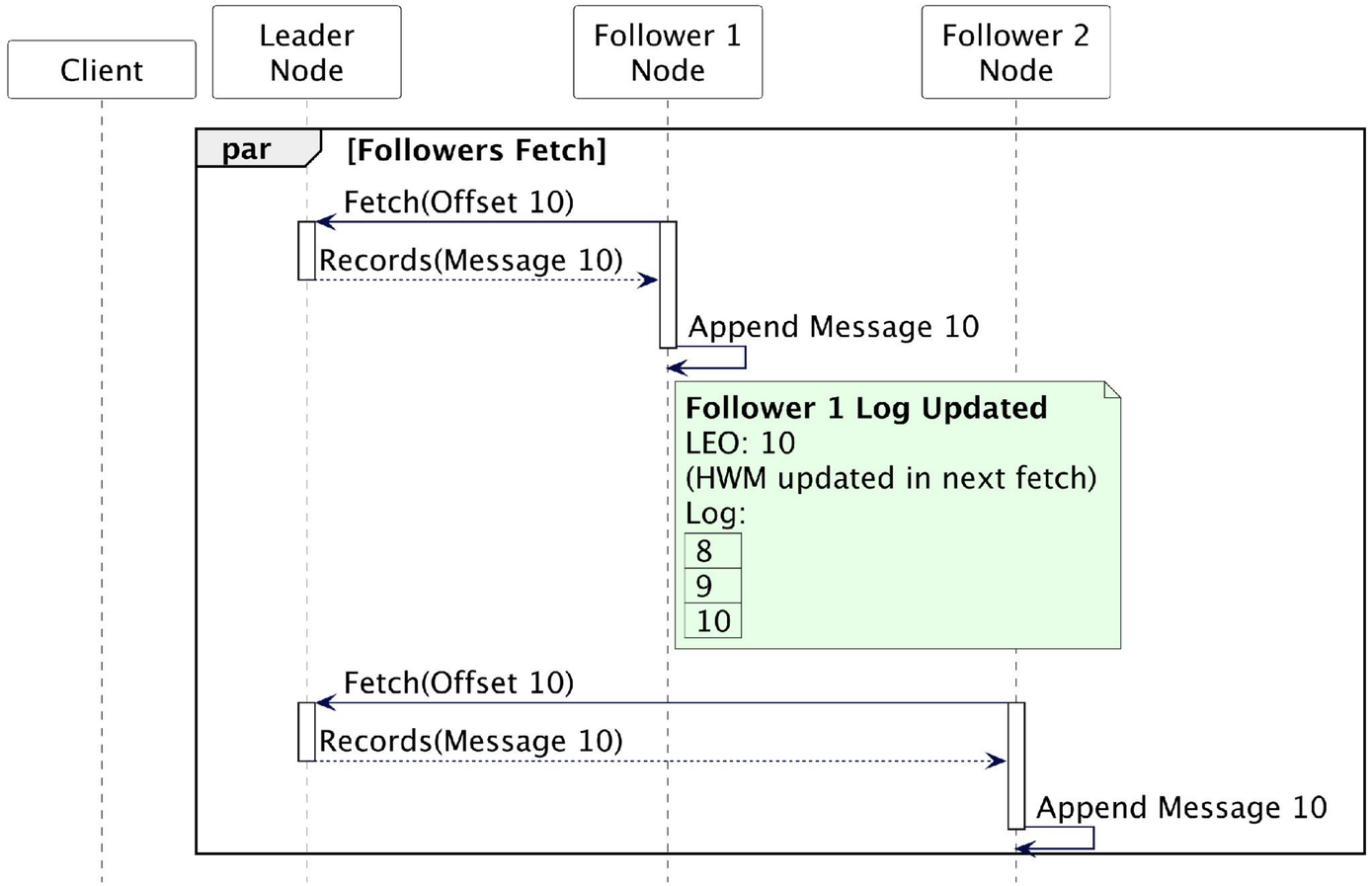
- Leader receives Produce(Message 10) and appends locally
- Followers pull records via Fetch(Offset 10) and update local logs
- Quorum reached for Offset 10 -> Leader updates HighWatermark = 10
- Followers update HighWatermark to 10 on subsequent fetch
- Clients can consume messages only up to HighWatermark

Consistency - HighWaterMark - Step 2



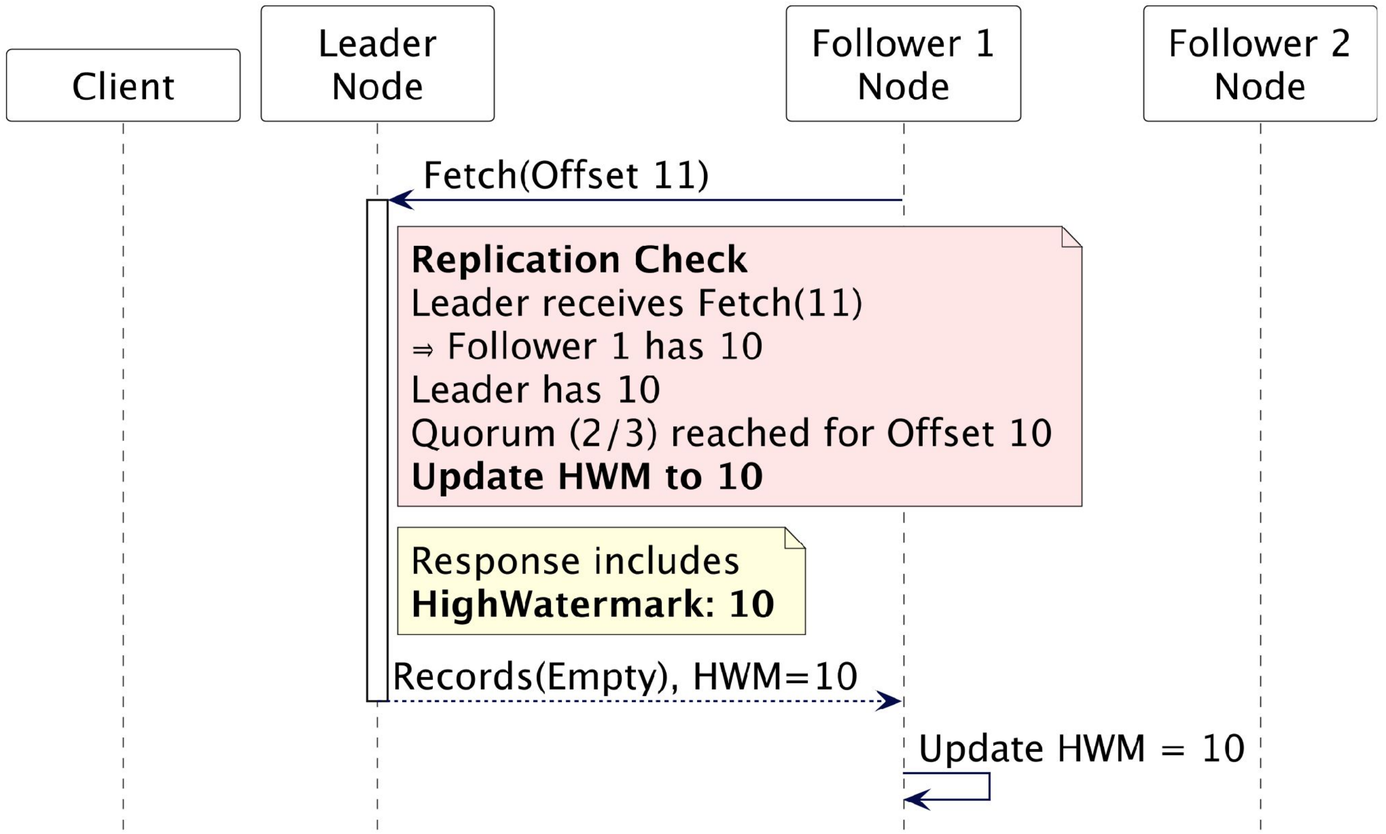
- Leader receives Produce(Message 10) and appends locally
- Followers pull records via Fetch(Offset 10) and update local logs
- Quorum reached for Offset 10 -> Leader updates HighWatermark = 10
- Followers update HighWatermark to 10 on subsequent fetch
- Clients can consume messages only up to HighWatermark

Consistency - HighWaterMark - Step 3



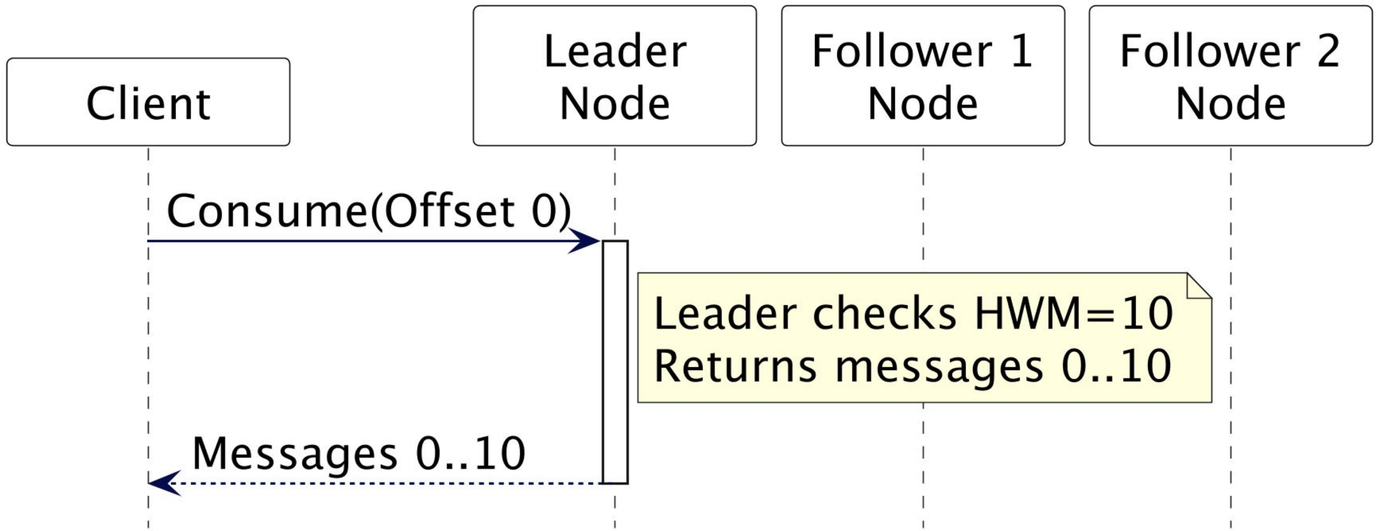
- Leader receives Produce(Message 10) and appends locally
- Followers pull records via Fetch(Offset 10) and update local logs
- Quorum reached for Offset 10 -> Leader updates HighWatermark = 10
- Followers update HighWatermark to 10 on subsequent fetch
- Clients can consume messages only up to HighWatermark

Consistency – HighWaterMark – Step 4



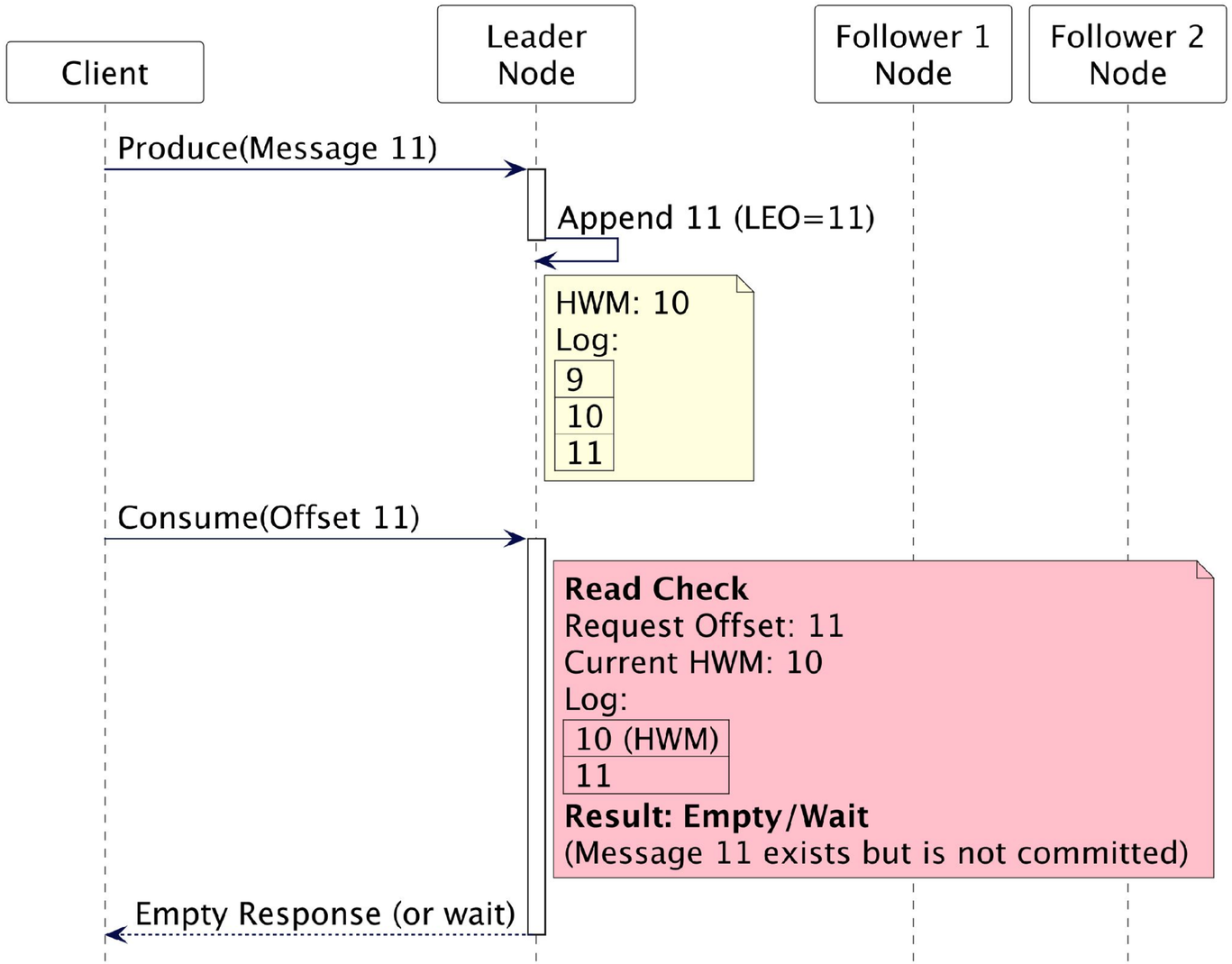
- Leader receives Produce(Message 10) and appends locally
- Followers pull records via Fetch(Offset 10) and update local logs
- Quorum reached for Offset 10 -> Leader updates HighWatermark = 10
- Followers update HighWatermark to 10 on subsequent fetch
- Clients can consume messages only up to HighWatermark

Consistency - HighWaterMark - Step 5



- Leader receives Produce(Message 10) and appends locally
- Followers pull records via Fetch(Offset 10) and update local logs
- Quorum reached for Offset 10 -> Leader updates HighWatermark = 10
- Followers update HighWatermark to 10 on subsequent fetch
- Clients can consume messages only up to HighWatermark

Consistency – HighWaterMark – Step 6



- Leader receives Produce(Message 10) and appends locally
- Followers pull records via Fetch(Offset 10) and update local logs
- Quorum reached for Offset 10 -> Leader updates HighWatermark = 10
- Followers update HighWatermark to 10 on subsequent fetch
- Clients can consume messages only up to HighWatermark

Writes above the high-watermark

- A leader can append entries and then fail before followers catch up
- Entries above the high-watermark were never replicated to a majority
- The new leader may have a shorter log, so those entries are truncated on failover
- This is correct behaviour rather than data loss: they were never acknowledged as committed
- It is exactly why clients must not read past the high-watermark — they could observe a value that later disappears

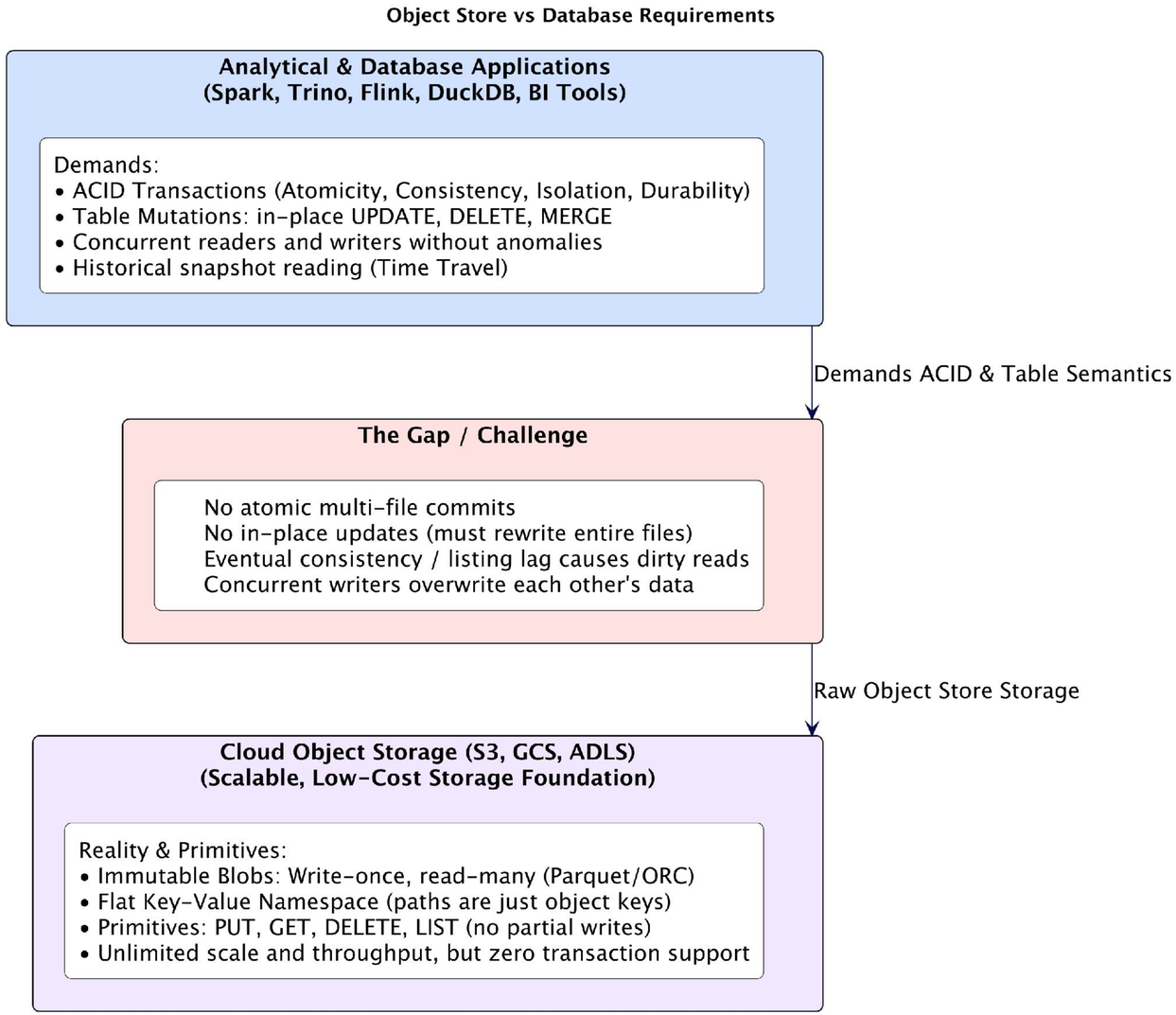
Assignment: Replication & High-Watermark

- Task 1: Implement Follower to fetch messages from Leader
- Task 2: Implement High-WaterMark tracking on Leader
- Task 3: Update High-WaterMark only after majority of followers have replicated
- Task 4: Ensure clients cannot read beyond High-Watermark

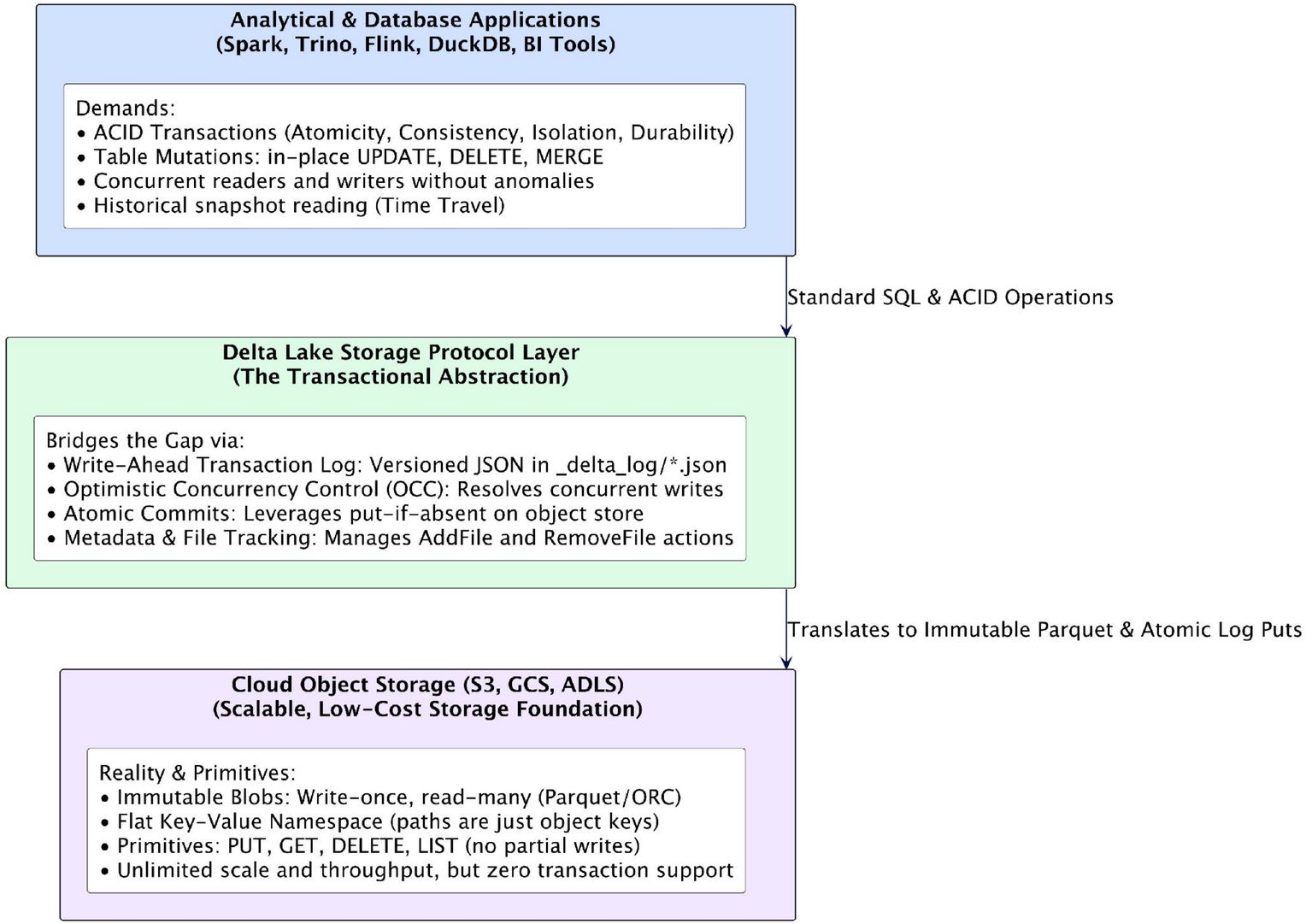
The Object Store Problem

- Object stores: Immutable blobs with key-value access (S3, Azure Blob Storage, GCS)
- No atomic updates: Cannot modify existing files in-place
- No transactions: Multiple operations across files are not atomic
- Eventual consistency and listing delays can expose incomplete writes
- Challenge: How to build ACID databases on immutable storage?

Object Store vs Database Requirements – Step 1



Object Store vs Database Requirements – Step 2



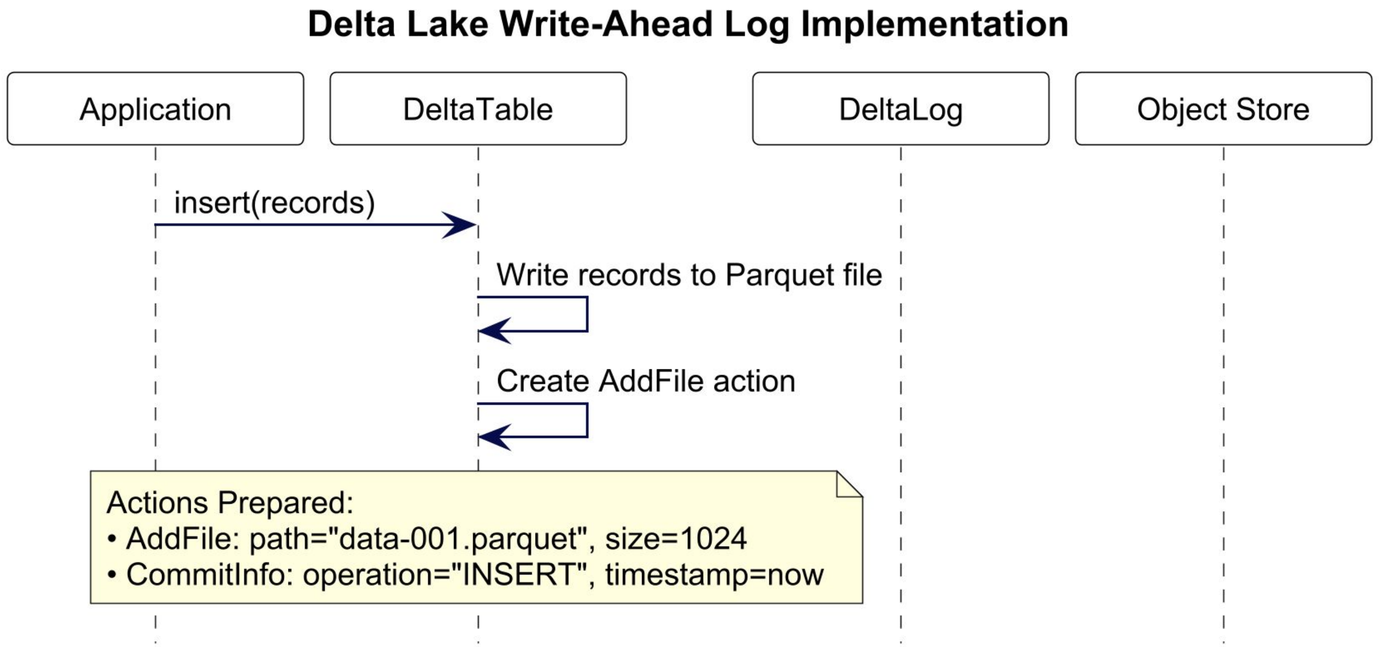
Transaction Log Solution

- Core idea: Log all changes as immutable JSON files in `_delta_log/`
- Each commit creates new numbered file: `00000000000000000001.json`
- Files contain list of Actions describing changes (AddFile, RemoveFile)
- Table state = replay all log files in order up to version N
- Example: `delta-module/DeltaLog.java` shows log file creation

Action Types – The Building Blocks

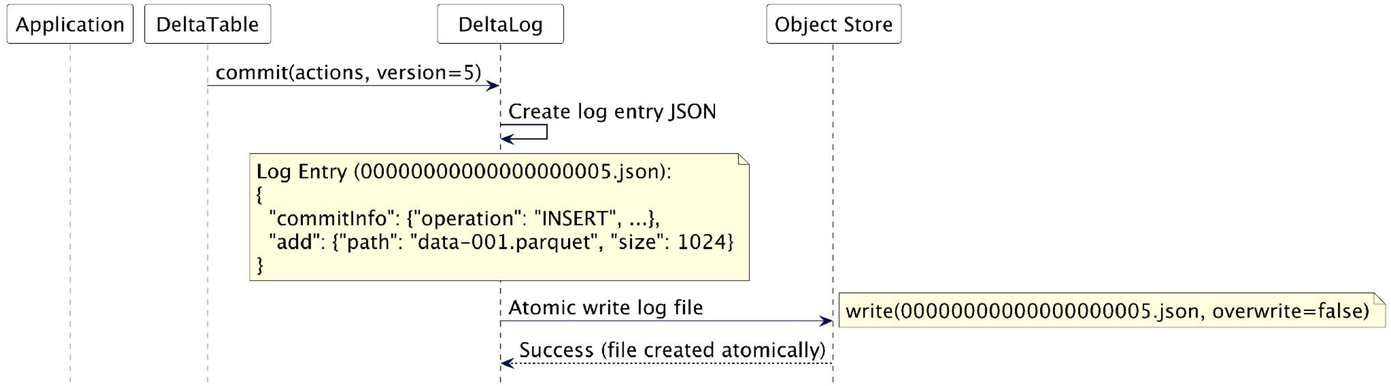
- AddFile: Adds new Parquet data file to table
- RemoveFile: Logically removes file from table (soft delete)
- CommitInfo: Metadata about the commit (timestamp, operation, engine)
- Metadata: Schema, partitioning, and table properties
- Protocol: Version compatibility information and reader/writer requirements

Write-Ahead Log Implementation - Step 1



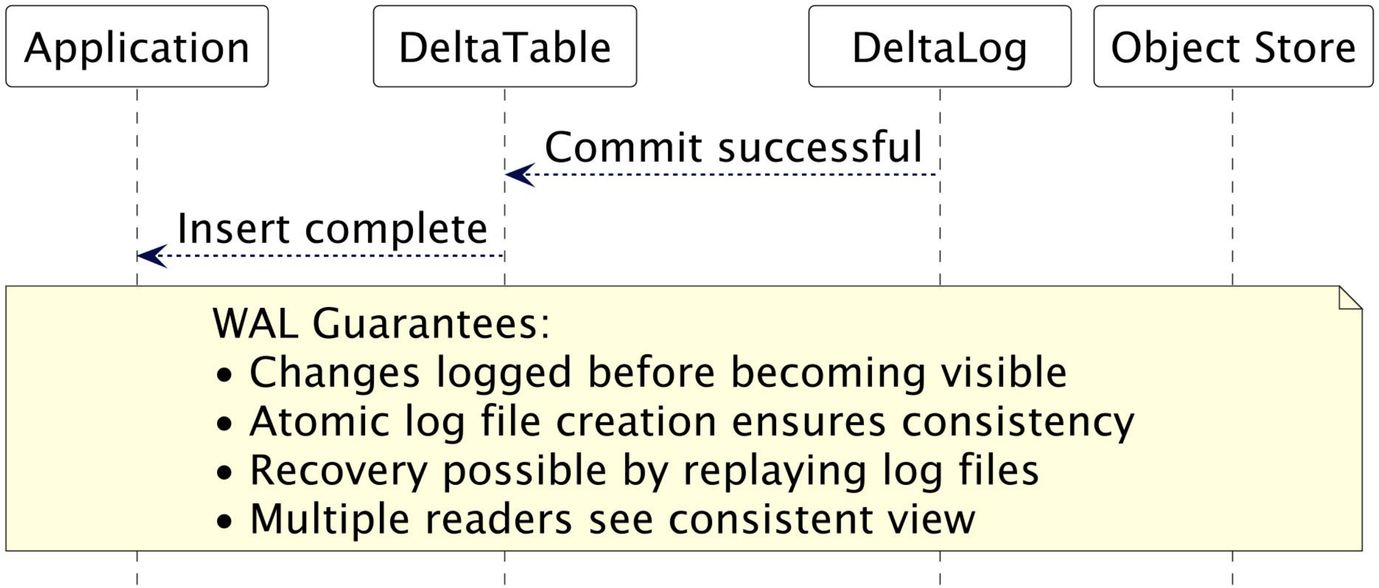
- Changes logged before becoming visible to readers
- Atomic log file creation ensures consistency
- Recovery possible by replaying log files
- Multiple readers see consistent point-in-time view

Write-Ahead Log Implementation - Step 2



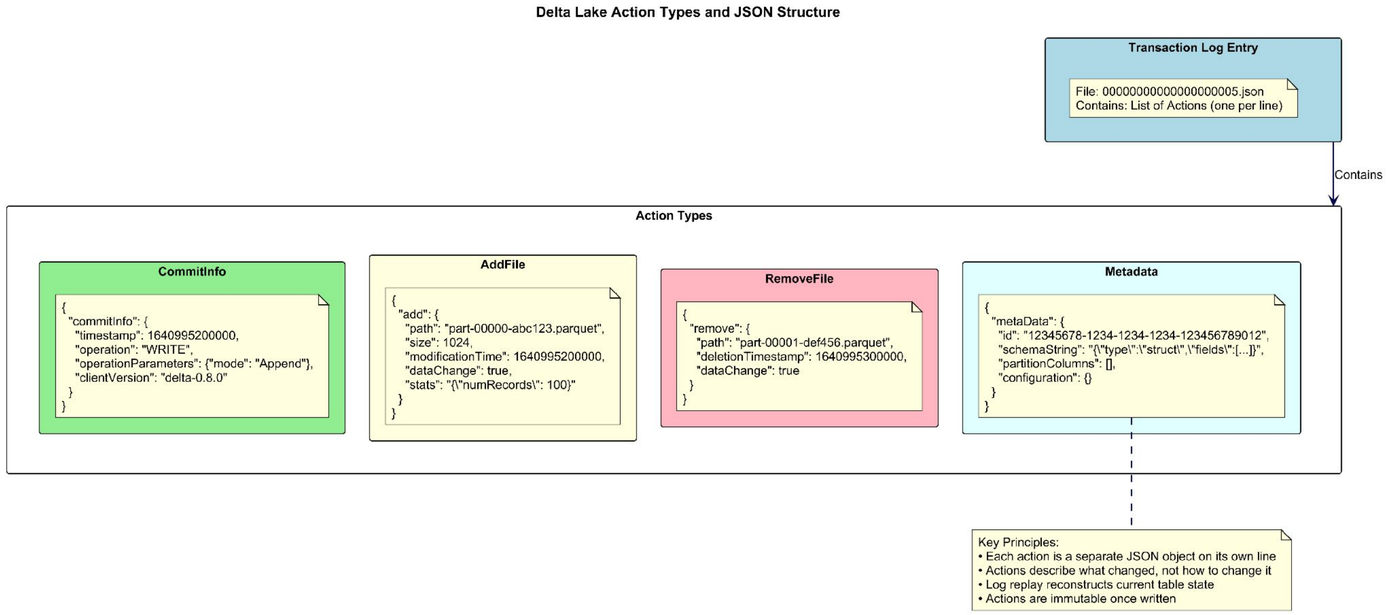
- Changes logged before becoming visible to readers
- Atomic log file creation ensures consistency
- Recovery possible by replaying log files
- Multiple readers see consistent point-in-time view

Write-Ahead Log Implementation - Step 3



- Changes logged before becoming visible to readers
- Atomic log file creation ensures consistency
- Recovery possible by replaying log files
- Multiple readers see consistent point-in-time view

Delta Lake Action Types and Structure



- Each action is a separate JSON object
- Actions describe what changed, not how to change
- Log replay reconstructs current table state
- Actions are immutable once written

Snapshot-Based Reads

- Snapshot = point-in-time view of table state
- Created by reading log up to specific version
- Immutable once created - no changes during read
- Multiple readers can share same snapshot
- Example: `delta-module/Snapshot.java` implementation

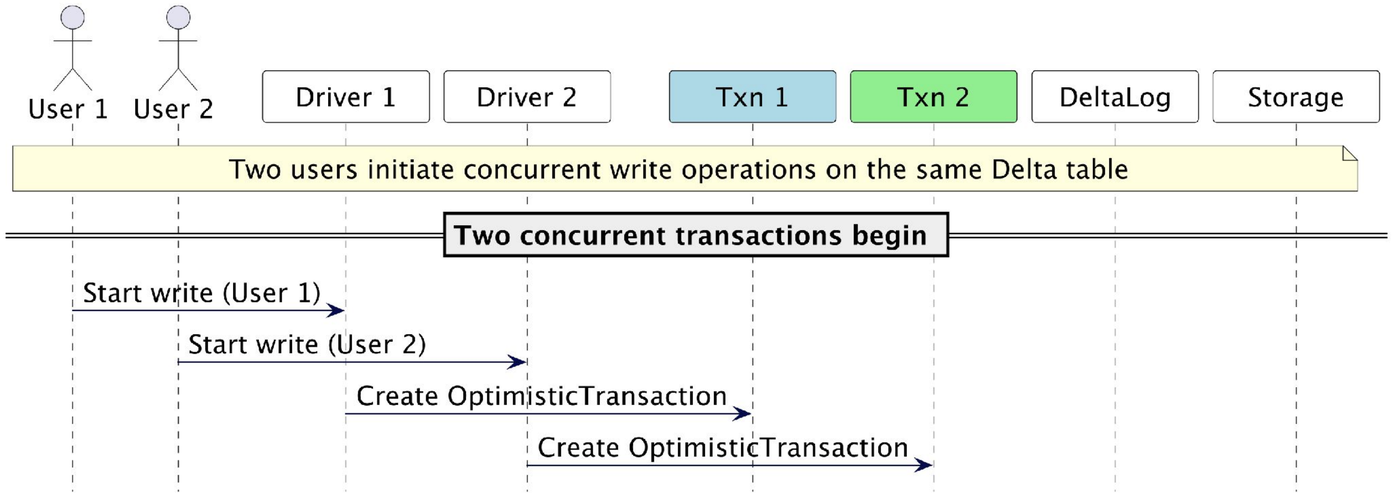
Time Travel Queries

- Read table as it existed at version N or timestamp T
- SQL syntax: `SELECT * FROM table VERSION AS OF 42`
- Use cases: Debugging, auditing, reproducible analytics, rollback
- Cost: Only metadata scan, no data reprocessing
- Assignment: Implement version-based reads

The Concurrent Writers Challenge

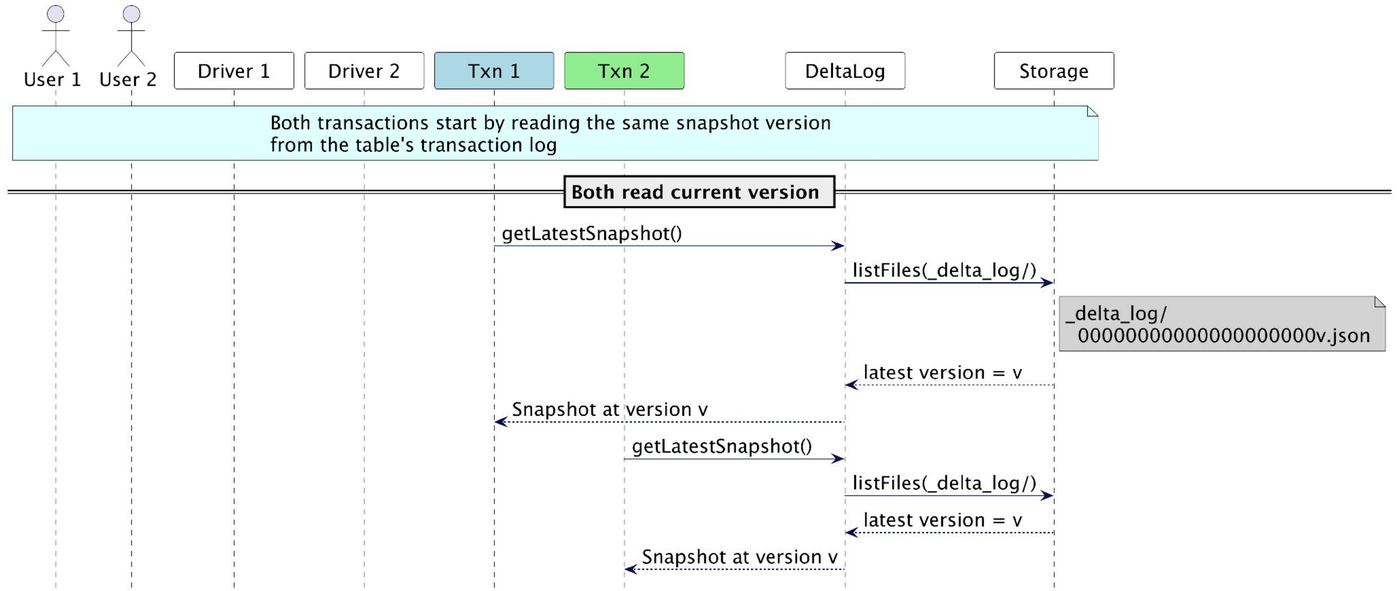
- Problem: Multiple writers, single log sequence
- Traditional solution: Pessimistic locking (doesn't scale on object storage)
- Delta Lake solution: Optimistic concurrency control (OCC)
- Key insight: Most transactions don't actually conflict
- Reference: `delta-module4/OptimisticTransaction.java`

Optimistic Transaction Flow – Step 1



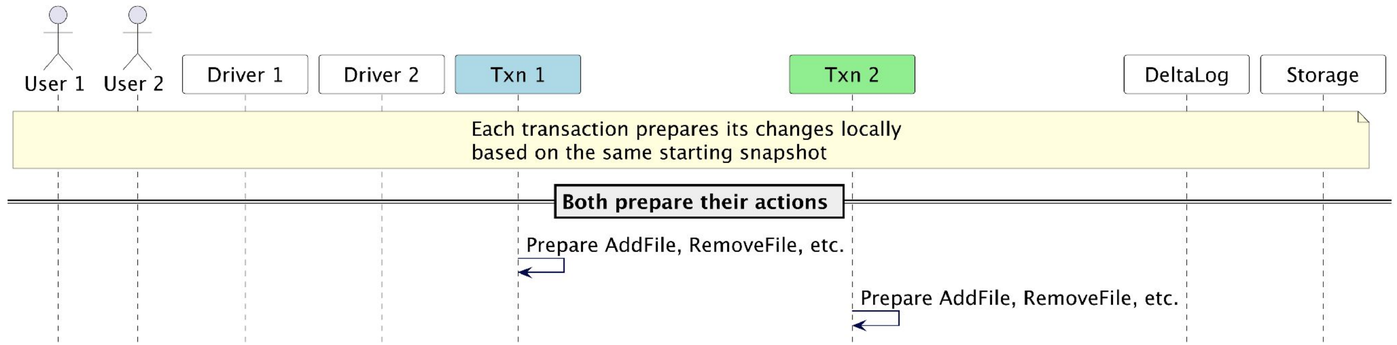
- Both transactions read version V
- Both prepare changes independently
- First to commit wins (creates V+1)
- Second detects conflict, retries or fails

Optimistic Transaction Flow – Step 2



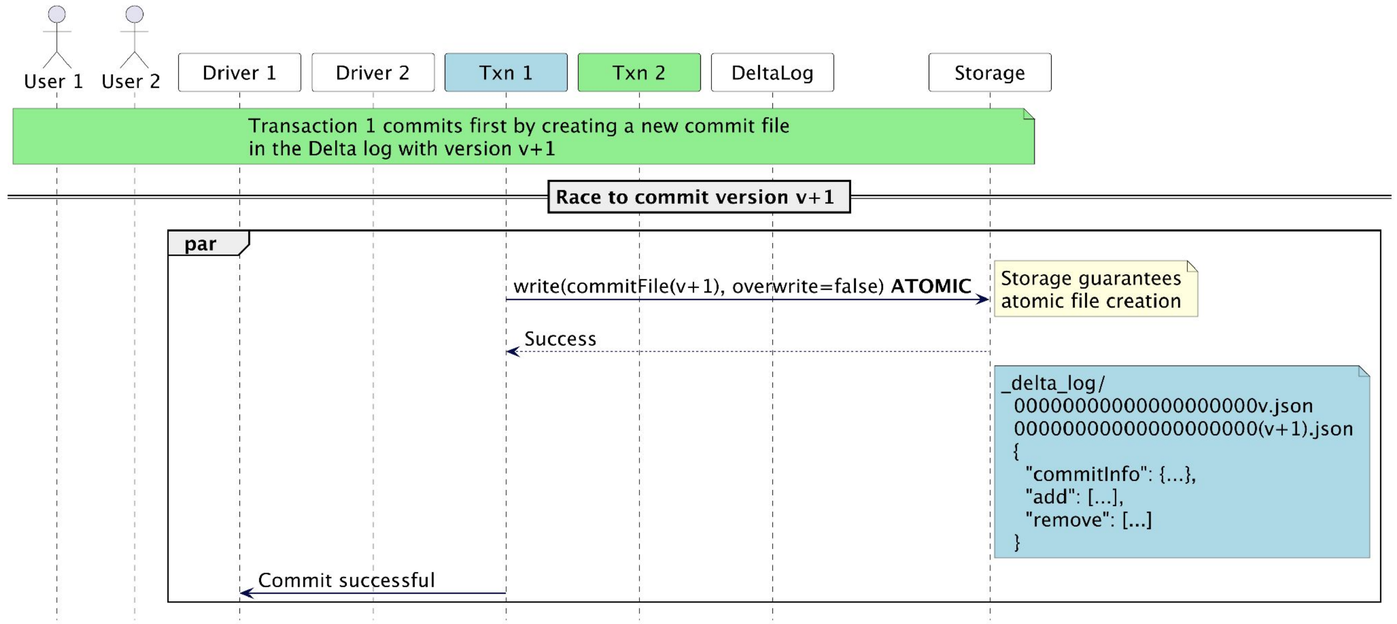
- Both transactions read version V
- Both prepare changes independently
- First to commit wins (creates V+1)
- Second detects conflict, retries or fails

Optimistic Transaction Flow – Step 3



- Both transactions read version V
- Both prepare changes independently
- First to commit wins (creates V+1)
- Second detects conflict, retries or fails

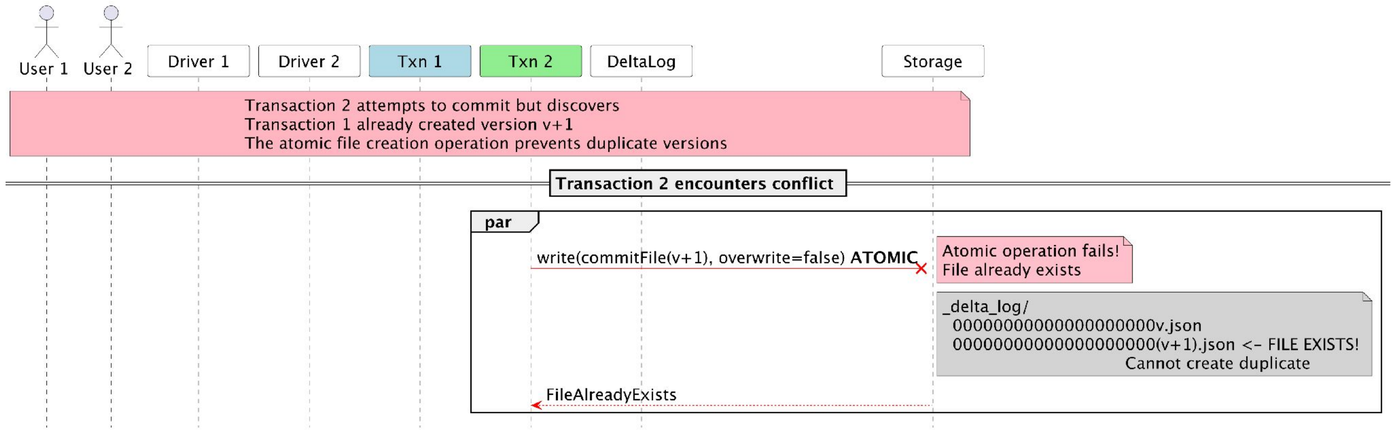
Optimistic Transaction Flow – Step 4



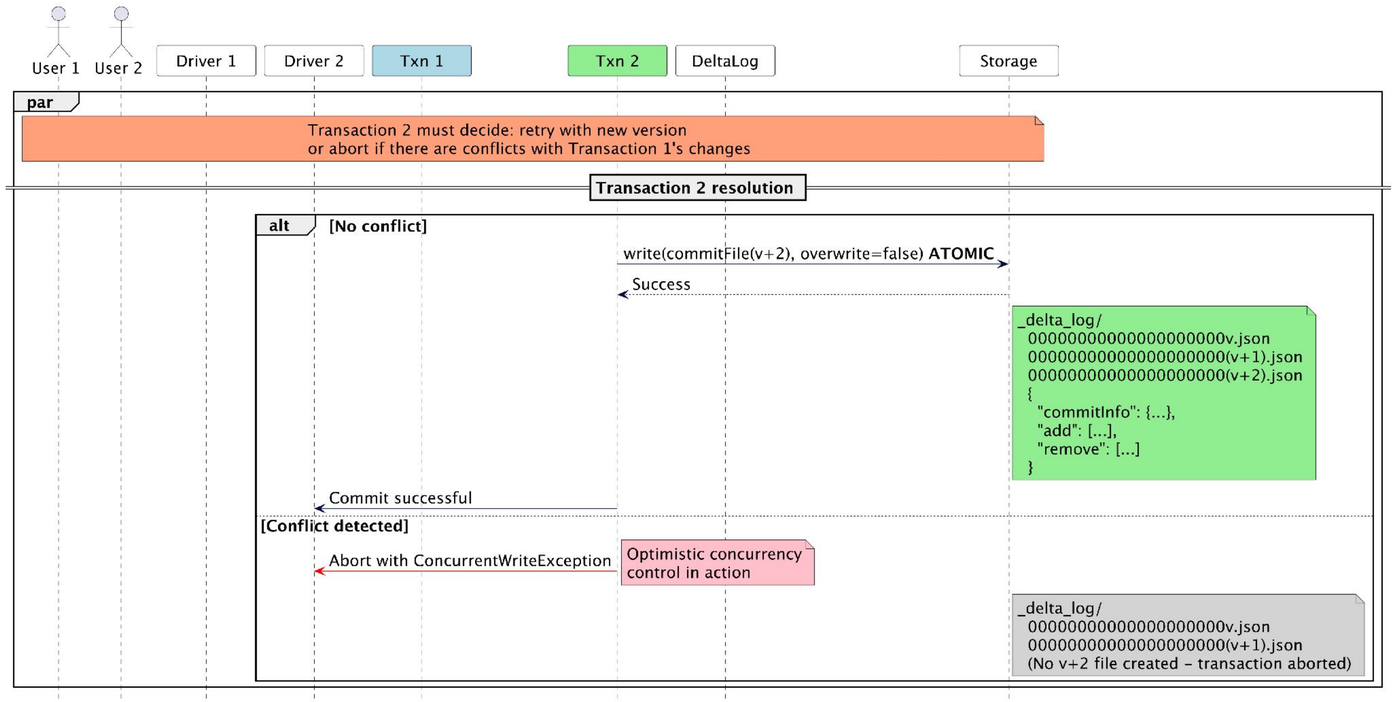
- Both transactions read version V
- Both prepare changes independently
- First to commit wins (creates V+1)
- Second detects conflict, retries or fails

Optimistic Transaction Flow – Step 5

- Both transactions read version V
- Both prepare changes independently
- First to commit wins (creates V+1)
- Second detects conflict, retries or fails



Optimistic Transaction Flow – Step 6



- Both transactions read version V
- Both prepare changes independently
- First to commit wins (creates V+1)
- Second detects conflict, retries or fails

Conflict Detection and Resolution

- Read version when transaction starts
- At commit: check if any new versions exist
- Analyze conflicting actions for logical conflicts
- Retry if safe, abort if true conflict
- Assignment: Implement `OptimisticTransaction.commit()` method

Assignment 1: Basic File Operations

- Repository: `delta-module/`
- Goal: Understand object store limitations through hands-on experience
- Task 1: Implement `DeltaTable.insert()` to write Parquet files
- Task 2: Implement `DeltaTable.readAll()` to read from all files
- Key Tests: `testReadAllRecords()`, `testConsecutiveInserts()`
- Challenge: How to track which files belong to which insert?
- Learning: Why simple file-based approaches don't scale

Assignment 2: Transaction Log Implementation

- Repository: `delta-module/`
- Goal: Build atomic commits using transaction logs
- Task 1: Implement `DeltaLog.write()` for versioned JSON commits
- Task 2: Implement `DeltaTable.insert()` with AddFile actions
- Task 3: Implement version-based reads with `readRecordsAtAVersion()`
- Key Test: `testReadRecordsAtAVersion()`
- Actions: AddFile, CommitInfo, Metadata, Protocol
- Learning: How transaction logs enable atomic operations

Assignment 3: Snapshot Isolation

- Repository: `delta-module/`
- Goal: Implement consistent reads via snapshots
- Task 1: Implement `OptimisticTransaction.commit()` method
- Task 2: Write concurrent transaction test
- Task 3: Verify only one transaction succeeds per version
- Key Concept: Point-in-time consistent views
- Challenge: Handle concurrent writers safely
- Learning: How optimistic concurrency control works

Assignment 4: Delta-lake with Object Store

- Repository: `delta-module/`
- Goal: Complete Delta Lake implementation with object store
- Task 1: Implement `DeltaLog.write()` with object store backend
- Task 2: Implement `DeltaTable.insert()` with Parquet + object store
- Task 3: Implement `OptimisticTransaction.commit()` with conflict resolution
- Key Tests: `testConcurrentTransactionsWithConflict()`
- Learning: Production-ready ACID transactions on object stores

Real-World Delta Lake Benefits

- ACID guarantees on petabyte-scale data lakes
- Time travel for data lineage and debugging
- Schema evolution without breaking existing queries
- Optimized file management and compaction
- Integration with Spark, Presto, and other engines

Summary

Workshop Summary

- Cluster Coordination: Small Consistent Core + Lease + State Watch enables scalable clusters
- Partitioning: Indirection of fixed partitions enables elastic load balancing without massive reshuffling
- Metadata Handling: Single controller with generation/epoch clocks prevents split-brain anomalies
- Controllers & Reconciliation: Level-triggered control loops bring self-healing to complex distributed systems
- Scheduling: Two-phase filtering and scoring decouples placement logic from execution
- Columnar Storage: Parquet row groups form natural partitions for parallel distributed computing
- Replication: Leader-follower replication with High-Watermark guarantees read consistency across crashes
- ACID Storage: Write-ahead logging over immutable object stores brings transactional guarantees to data lakes